

Trinity College Dublin
Coláiste na Tríonóide, Baile Átha Cliath
The University of Dublin

School of Engineering

LIFTED – LLVM Infrastructure for Testing, Equivalence, and Deobfuscation

Adam Niezabitowski

Supervisor: Prof. Vasileios Koutavas

April 13, 2026

A dissertation submitted in partial fulfilment
of the requirements for the degree of
MAI (Computer Engineering)

Declaration

I hereby declare that this dissertation is entirely my own work and that it has not been submitted as an exercise for a degree at this or any other university.

I have read and I understand the plagiarism provisions in the General Regulations of the University Calendar for the current year, found at `http://www.tcd.ie/calendar`.

I have completed the Online Tutorial on avoiding plagiarism 'Ready Steady Write', located at `http://tcd-ie.libguides.com/plagiarism/ready-steady-write`.

I consent to the examiner retaining a copy of the thesis beyond the examining period, should they so wish (EU GDPR May 2018).

I agree that this thesis will not be publicly available, but will be available to TCD staff and students in the University's open access institutional repository on the Trinity domain only, subject to Irish Copyright Legislation and Trinity College Library conditions of use and acknowledgement.

Signed: _____ Date: _____

Abstract

Binary obfuscation—the application of semantics-preserving transformations to compiled software in order to impede analysis—is a central challenge in security research. Automated deobfuscation tools exist, but three gaps constrain their practical utility: the most promising paradigm, SATURN's approach of lifting obfuscated machine code into LLVM intermediate representation and applying compiler infrastructure to reverse the obfuscation, has no publicly available implementation; its published algorithm does not specify how to handle inter-procedural, indirect, or recursive calls found in real-world binaries; and no comparable tool provides a mechanism for verifying that a deobfuscation transformation has preserved the original program's semantics.

This thesis presents LIFTED (LLVM Infrastructure for Testing, Equivalence and Deobfuscation), a framework that addresses all three gaps. First, LIFTED reimplements SATURN's iterative CFG-recovery algorithm within an open-source (available at `https://github.com/0xnobody/lifted`), modular architecture that decouples the obfuscator-agnostic lifting core from obfuscator-specific behaviour through a user-extensible callback interface. Second, it extends the algorithm with a callee-annotation system for inter-procedural calls and a block-splitting mechanism for newly discovered branch targets, broadening the class of binaries the pipeline can process. Third, it introduces an integrated verification infrastructure based on differential testing: lifted LLVM IR is executed via JIT compilation and compared against the original binary emulated through Unicorn, using a checkpoint-based synchronisation system and a staged, configurable pipeline.

LIFTED is evaluated on a corpus drawn from the TUM obfuscation benchmarks and GNU Coreutils, compiled in both unobfuscated and Polaris-obfuscated configurations. The lifter achieves pass rates of 94.0% and 91.8% on unobfuscated and obfuscated binaries respectively, with every non-passing case attributed to a root cause. The verifier validates LLVM's O2 optimisation pipeline as a semantics-preserving transformation with a 100% pass rate. A mutation-testing study confirms 100% specificity (no false positives) and sensitivity of 70.5% under strict comparison, establishing the verifier as a practical development oracle for deobfuscator development.

Lay Abstract

When software is distributed, the compiled program is often the only artefact available for analysis. Some software producers deliberately apply transformations to these compiled programs to make them harder to understand, a practice known as obfuscation. While sometimes used legitimately to protect intellectual property, the same techniques are also used by malware authors to hide malicious behaviour from security analysts. Reversing these transformations manually is prohibitively time-consuming, motivating the development of automated tools.

The most promising automated approach, proposed by prior research, works by translating obfuscated machine code into a higher-level representation used by compilers, then applying the compiler's own analysis tools to undo the obfuscation. However, this approach was never released as publicly available software, its algorithm does not fully handle patterns common in real programs, and there is no way to check whether the reversal process introduced errors.

This thesis presents LIFTED, a software framework that addresses these three limitations. LIFTED provides an open-source implementation of the translation process with a modular design that allows researchers to adapt it to different obfuscation strategies. It extends the underlying algorithm to handle function calls and code patterns that the original research left unaddressed. Most importantly, it includes a built-in testing system that automatically checks whether each step of the reversal process has preserved the original program's behaviour, by running both the translated and original versions on the same inputs and comparing their outputs.

Evaluation on a corpus of benchmark programs shows that the framework successfully translates over 90% of tested functions, that its testing system correctly validates known-good transformations, and that it detects the majority of artificially injected errors with no false alarms. The result is a practical tool that makes the development of deobfuscation techniques more systematic, more testable, and more accessible to the research community.

Acknowledgements

I'd like to extend my deepest gratitude to my supervisor, Prof. Vasileios Koutavas, for his invaluable guidance and support throughout this project. His insights and encouragement have been instrumental in my work on the thesis.

I also want to thank my friends and family for their unwavering support and understanding during the course of this work. Their encouragement kept me motivated and focused, especially during challenging times.

Finally, I would also like to thank Matteo Favaro for his pioneering work on SATURN, which laid the foundation for this thesis, and for his continual willingness to share insights that helped shape the direction of this research.

Contents

1 Introduction **1**

1.1 Motivation . . . 1

1.2 Problem Statement . . . 2

1.3 Contribution . . . 4

2 Background **5**

2.1 Binary Obfuscation . . . 5

2.1.1 Definition and Theoretical Foundations . . . 5

2.1.2 Motivations and Deployment Contexts . . . 5

2.1.3 Binary Obfuscation Techniques . . . 6

2.1.4 Layered Composition and Compounding Difficulty . . . 10

2.1.5 Consequences for Binary Analysis . . . 10

2.2 Binary Lifting & Intermediate Representations . . . 11

2.2.1 Intermediate Representations . . . 11

2.2.2 Binary Lifting . . . 12

2.2.3 Challenges in the Context of Obfuscation . . . 14

2.3 Binary Deobfuscation . . . 14

2.3.1 Definition and Purpose . . . 14

2.3.2 Deobfuscation Approaches . . . 14

2.3.3 SATURN: Deobfuscation by Optimisation . . . 16

2.3.4 Confidence in Deobfuscated Output . . . 20

2.4 Verification and Differential Testing . . . 20

2.4.1 Approaches to Program Equivalence . . . 20

2.4.2 Verification for LLVM IR and Binary Code . . . 21

2.4.3 Challenges in the Deobfuscation Setting . . . 22

2.5 Gaps in the Literature . . . 23

3 Design and Implementation **25**

3.1 System Overview . . . 25

3.1.1 Goals and Scope . . . 25

3.1.2 Relation to SATURN . . . 26
3.1.3 End-to-end Architecture . . . 27
3.2 The Lifting Framework . . . 30
3.2.1 Lifting Algorithm . . . 30
3.2.2 Annotations . . . 35
3.2.3 Handling External, Indirect, and Recursive Calls . . . 38
3.2.4 Block-Splitting . . . 45
3.2.5 A Composable Architecture . . . 47
3.3 The Verification Framework . . . 50
3.3.1 A Concrete Testing Approach . . . 50
3.3.2 A Randomised Memory Model . . . 52
3.3.3 The Executor Model . . . 55
3.3.4 The Checkpoint System . . . 66
3.3.5 Handling Loops . . . 71
3.3.6 Pipelining . . . 73
3.3.7 Feedback for the User . . . 75
3.3.8 A Verification Campaign in Practice . . . 76

4 Evaluation **78**
4.1 Evaluation Infrastructure & Setup . . . 78
4.2 Evaluation Goals . . . 81
4.3 Lifting Correctness Across Suites . . . 82
4.3.1 Methodology . . . 82
4.3.2 Results . . . 84
4.3.3 Discussion . . . 88
4.4 Deobfuscation Pass Correctness . . . 88
4.4.1 Methodology . . . 89
4.4.2 Results . . . 89
4.4.3 Discussion . . . 90
4.5 Fault Injection Study . . . 91
4.5.1 Methodology . . . 91
4.5.2 Results . . . 93
4.5.3 Discussion . . . 94
4.6 A Case Study . . . 95
4.6.1 Case Selection . . . 95
4.6.2 Diagnosing the Discrepancy . . . 96
4.6.3 Root-Cause Attribution . . . 102
4.7 Discussion . . . 103

5 Conclusion **106**
5.1 Summary of Contributions 106
5.2 Limitations 107
5.3 Future Work 109

A1 `numtowords::main` Code **119**

A2 Full Optimised Slice of `BB_11ed` **120**

A3 AI Statement **124**

List of Figures

1.1 A simple password checking function. . . . 2
1.2 An example obfuscated form of Figure 1.1. . . . 3
1.3 The analyst's (incorrect) deobfuscated form of Figure 1.2 4

2.1 Opaque predicate insertion 7
2.2 Control-flow flattening 7
2.3 Anti-disassembly by junk-byte insertion 8
2.4 Indirect control-flow obfuscation 8
2.5 A classical process for VM-based code obfuscation. . . . 10
2.6 Compiled IR vs Remill-lifted IR example 12
2.7 A simplified representation of Remill's State structure for x86-64 13
2.8 SATURN CFG exploration example 18
2.9 SATURN program counter (PC) slicing example 19

3.1 High-level architecture of LIFTED 29
3.2 Example of a single lifted basic-block (optimised with O2). . . . 32
3.3 Backwards and forwards stitching algorithms 33
3.4 Example of a stitched CFG 34
3.5 RAM lowering 35
3.6 An example of an annotation JSON file 37
3.7 Types of basic-block targets 39
3.8 Example of `LifterCallbacks` returning multiple targets 40
3.9 Example mappings of inter-procedural calls to stubs and annotations. . . . 41
3.10 Example of a lifted synthetic block 42
3.11 Stitched CFG with synthetic block dispatching 43
3.12 Example of a call stub generated by the `Lifter` for the System V x64 calling-convention. . . . 44
3.13 Block-splitting motivating example 45
3.14 Block-truncation motivating example 46
3.15 Skeleton of an obfuscator-specific `LifterCallbacks` subclass. . . . 48

3.16 End-to-end lifting of two functions from the same binary. 49
3.17 Example of the under-constrained analysis problem. 52
3.18 Example of the underconstrained memory problem 53
3.19 The LIFTED memory model 54
3.20 Example of human-readable memory diff output 55
3.21 The LIFTED executor model 56
3.22 Preparation for JIT execution. 58
3.23 The definition for 64-bit memory reads provided by the verification helper . . 59
3.24 The entry-point of the JIT verification intrinsics 60
3.25 TB vs BB block-splitting example. 62
3.26 TB vs BB spurious TB example. 62
3.27 Two-level hook interaction in the UnicornExecutor 63
3.28 External boundary problem under emulation. 64
3.29 Creating and executing a matched executor pair from a single `lift_result_t` 66
3.30 The checkpoint model. 67
3.31 Example of checkpoint creation on event. 68
3.32 Example of event desynchronisation before and after an IR transformation. . 69
3.33 The RAW-STRICT and OPTIMISED-RELAXED default checkpoint profiles' effects on the `combined_hash`. 70
3.34 A motivating example for a hard loop to break out of given random input due to bounding 71
3.35 A motivating example for a hard loop to break out of given random input due to a sentinel check 71
3.36 Simplified example of sentinel injection. 72
3.37 Example of a pipelined verification campaign. 74
3.38 Running a practical verification campaign from a single `lift_result_t` . . 77

4.1 EG1 lift time by suite and configuration. 85
4.2 A snippet of the `switch` statement from `numtowords::main`. 96
4.3 `lifted-cli fn` invocation and output. 97
4.4 An extract from the pipeline report for `numtowords::main`. 98
4.5 The deep-dive report for the first reported discrepancy in `numtowords::main`. 99
4.6 The switch table from the `numtowords::main` binary. 100
4.7 The `switch` dispatcher instructions from the `numtowords::main` binary. . . 100
4.8 Recovered CFG for `numtowords::main` with execution-frequency overlay. The discrepant site is the indirect jump at `0x11ed..0x1211`; the recovered graph stops at that node instead of enumerating the switch-arm successors. 101
4.9 Stitched wrapper around `BB_11ed`. 102
4.10 Snippet of optimised slice of `BB_11ed`. 103

A1.1 Full function code for `numtowords::main` [6]. 119

A2.1 Full PC slice function of `BB_11ed`. 123

List of Tables

2.1 Principal binary deobfuscation technique families and their key limitations. . 15

3.1 Component-level comparison of SATURN [33] and LIFTED. 27

4.1 Test suites used in the evaluation. 80
4.2 Mapping of evaluation goals to implementation goals. 81
4.3 Verification pipeline stages used in EG1 (RAW-STRICT checkpoint profile). . 83
4.4 EG1 lifting and verification results (raw lifted IR vs binary) across all test suites and obfuscator configurations. 84
4.5 EG1 discrepancy taxonomy. 86
4.6 EG2 lifting and verification (raw lifted IR vs O2 IR) results across all test suites and obfuscator configurations. 90
4.7 Mutation categories used in EG3. Categories are adapted from the libIRM taxonomy [20, 24]. 92
4.8 Per-suite mutation test specificity results (O2 vs O2 self-test). 93
4.9 Per-suite mutation test sensitivity results. 94
4.10 Per-category mutation test sensitivity results across all test suites. 94
4.11 Compact verdict on the evaluation goals. 104

Nomenclature

ABI	Application binary interface
AMD64	AMD64 64-bit x86 architecture and calling-convention family
API	Application programming interface
AVX	Advanced Vector Extensions
BB	Basic block
CFF	Control-flow flattening
CFG	Control-flow graph
CLI	Command-line interface
CPU	Central processing unit
CSV	Comma-separated values
DEBRA	Benchmark for evaluating deobfuscation methods on real-world programs
DRM	Digital rights management
DSE	Dynamic symbolic execution
EG1	Evaluation Goal 1: Lifting Correctness Across Suites
EG2	Evaluation Goal 2: Deobfuscation Pass Correctness
EG3	Evaluation Goal 3: Fault Injection Study
EG4	Evaluation Goal 4: Case Study
GCC	GNU Compiler Collection
GNU	"GNU's Not Unix" project
IG1	Implementation Goal 1: open-source extensible deobfuscation model
IG2	Implementation Goal 2: lifting-algorithm improvements for real-world binaries
IG3	Implementation Goal 3: integrated automated verification infrastructure
IR	Intermediate representation
ISA	Instruction set architecture
JIT	Just-in-time compilation
JSON	JavaScript Object Notation
LIFTED	LLVM Infrastructure for Testing, Equivalence, and Deobfuscation
LLVM	LLVM compiler infrastructure and intermediate-representation ecosystem
MBA	Mixed Boolean-Arithmetic
O0	Compiler optimisation level 0
O2	Compiler optimisation level 2
OLLVM	Obfuscator-LLVM
OS	Operating system

PC	Program counter
PRNG	Pseudo-random number generator
RAM	Random-access memory
RVA	Relative virtual address
SATURN	Software Deobfuscation Framework Based on LLVM
SHA	Secure Hash Algorithm
SIMD	Single instruction, multiple data
SMT	Satisfiability modulo theories
SSA	Static single assignment
SSE	Streaming SIMD Extensions
TB	Translation block
TUM	Technical University of Munich
UB	Undefined behaviour
VM	Virtual machine
WSL2	Windows Subsystem for Linux 2
x86-64	64-bit x86 instruction set architecture
Z3	SMT solver used for theorem proving

1 Introduction

This thesis presents LIFTED: an LLVM-based Infrastructure for Testing, Equivalence and Deobfuscation. LIFTED addresses practical gaps in automated binary deobfuscation, a field concerned with the reversal of obfuscating transformations applied to compiled software. This chapter motivates the work, states the specific problems it addresses, and enumerates its contributions.

1.1 Motivation

Binary software is typically distributed without source code: the compiled binary is the only artefact available to analysts. The security research field, including tasks such as vulnerability discovery and malware analysis, therefore depends on the ability to analyse binaries directly [69].

This is already a difficult problem. Compiled binaries do not contain source-level abstractions: variable names, types, and control-flow structure are lost during compilation. Dedicated binary-analysis frameworks, such as BAP, rev.ng and IDA Pro, have therefore been developed [14, 25, 35]. These are central to the work of security researchers and analysts.

However, certain actors deliberately apply *binary obfuscation*: semantics-preserving program transformations whose purpose is to impede analysis [17]. The motivation is twofold. On the legitimate side, software vendors apply obfuscation to protect intellectual property [67]. On the adversarial side, malware authors apply the same techniques to avoid detection and resist analysis [64, 78]. In practice, obfuscation can frustrate lightweight analysis tools and substantially increase analyst effort [64, 67].

Manual reversal of obfuscation is impractical in real-world applications. The techniques applied are focused on directly impeding this form of analysis. This has motivated a body of work on *automated deobfuscation*—tools that identify and undo obfuscating transformations [28, 74].

Such tools exist, but significant gaps remain. Garba and Favaro's SATURN [33] represents a paradigm shift in the research landscape: its approach allows use of the already existing, rich

compiler infrastructure for the purpose of deobfuscation. However, three limitations constrain its practical utility. First, the published work presents SATURN at a high level rather than as a released reusable artefact. Second, it does not describe the handling of inter-procedural calls or certain control-flow patterns found in real-world binaries. Third, neither SATURN nor any comparable tool provides a mechanism for verifying that a deobfuscation transformation has preserved the original program's semantics: a deobfuscation pass that introduces an error is indistinguishable from a correct one without such infrastructure.

This thesis addresses all three gaps directly. The driving question is how LLVM-based binary deobfuscation can be made more practical, extensible, and trustworthy; an answer this thesis develops, implements, and evaluates.

1.2 Problem Statement

The following example illustrates the three problems this thesis addresses. Consider a simple password checking function of the kind that may be commonly targeted by commercial obfuscation tools:

```
int check_password(int key) {
    if (key == 0xA3F1) {
        return activate();
    } else {
        return 0;
    }
}
```

Figure 1.1: A simple password checking function. The function is provided with an integer `key`; if the key matches the hardcoded value `0xA3F1`, it calls `activate()` and returns its result; otherwise, it returns zero.

A vendor wishing to protect this function may apply three simple layered obfuscating transformations: one conflating the data-flow, and two conflating the control-flow:

```
int check_password(int key) {
    static void *targets[] = { &&valid, &&invalid, &&other };
    unsigned idx = ((unsigned)((key ^ 0xA3F1) | -(key ^
       ↪ 0xA3F1))) >> 31;
    goto *targets[idx];
valid:
    return activate();
invalid:
    return 0;
other:
    return other();
}
```

Figure 1.2: An example obfuscated form of Figure 1.1. Three transformations have been applied: one encoding the conditional in a more complicated manner, one replacing the conditionals with a jump to computed targets, and one inserting code which is never executed.

This obfuscated program is *functionally equivalent* to the original, yet its structure has been deliberately obscured by three layered transformations: (1) the condition `key == 0xA3F1` is encoded as a more complex arithmetic expression, which evaluates to zero when the condition holds and one otherwise; (2) the conditionals have been replaced by a jump whose destinations are evaluated at runtime, not fixed in code; (3) a dead branch (to `other()`) has been inserted that appears reachable but is never taken. This obfuscation illustrates three challenges that existing approaches do not simultaneously address.

Problem 1. An analyst may wish to use SATURN's approach to combat this manner of obfuscation—but they cannot. No open implementation exists: the tool was never released, and hence cannot be extended or applied to real-world binaries.

Problem 2. Having reimplemented SATURN's pipeline, the analyst encounters a second limitation: the published algorithm does not specify how to handle inter-procedural calls. The function makes two such calls—`activate()` and `other()`—whose definitions lie outside the function under analysis. Without a defined treatment for these cases, the analyst must either extend the algorithm independently or restrict their analysis to functions that contain no inter-procedural calls.

Problem 3. Suppose the analyst applies three transformations in sequence, intended to *reverse* the effects of the obfuscation: simplification of the arithmetic, recovery of the conditionals, and elimination of the dead branch. The result is what appears to be a clean deobfuscated version:

```
int check_password(int key) {
    if (key == 0xA3F1) {
        return 0;
    } else {
        return other();
    }
}
```

Figure 1.3: The analyst's (incorrect) deobfuscated form of Figure 1.2. Instead of calling `activate()` when the key is correct, the function returns zero; instead of returning zero when the key is incorrect, it calls `other()`.

The deobfuscated function in Figure 1.3 differs from the original in Figure 1.1: the branches have been swapped, producing a function that differs in behaviour from the original.

Such a defect may be imperceptible on casual inspection, particularly as the complexity of both the function and transformations increase. Equally problematic is attribution: it is not clear whether the error was introduced during arithmetic simplification, conditionals recovery, or dead branch elimination. Without automated verification at each stage, the analyst must re-examine every transformation manually to isolate the fault; a time-consuming process.

1.3 Contribution

We address all three problems. First, we provide an open-source implementation of SATURN's approach, additionally extended with a modular architecture that allows obfuscation-specific behaviour to be defined without modifying the core. Second, we further extend the underlying algorithm to correctly handle cases that the original work left unspecified, making the pipeline applicable to real-world binaries. Third, we build automated verification infrastructure that checks if any deobfuscation step introduces a bug into the program—making the bug in Figure 1.3 detectable without manual inspection.

These three problems are the direct motivation for this thesis. Chapter 2 provides the necessary background. Chapter 3 describes LIFTED's design and implementation, formalising each contribution as a concrete implementation goal (IG1–IG3). Chapter 4 evaluates the result against testable evaluation goals derived from the IGs. Chapter 5 concludes, assessing the limitations and motivations for potential future work.

2 Background

The contribution of the thesis is situated at an intersection of cybersecurity, software engineering and program analysis. This chapter provides an overview of the relevant background and related work in the field, to establish a broader context for the problem space. We cover four foundational topics: binary obfuscation, binary lifting, deobfuscation and verification. The chapter concludes with a discussion of the gaps in the literature that motivate the contributions of this thesis.

2.1 Binary Obfuscation

2.1.1 Definition and Theoretical Foundations

An *obfuscating transformation* is a semantics-preserving program transformation whose purpose is to impede reverse engineering [17]. The transformed program must produce identical output to the original on every terminating input, but may be slower, larger, or structurally unrecognisable.

From a theoretical standpoint, Barak et al. prove that general-purpose *virtual black-box* obfuscation, under which an adversary with the obfuscated binary learns nothing beyond what oracle access to the program would reveal, is impossible [8]. As a result, the field instead aims to raise the economic cost of analysis beyond what a given adversary is willing to spend, rather than to provide formal protection guarantees [67]. The practical consequence is that obfuscation's resistance to analysis is an engineering constraint rather than a formal security barrier.

2.1.2 Motivations and Deployment Contexts

Obfuscation is typically deployed in two broad and often overlapping contexts. On the legitimate side, it is used to protect intellectual property in distributed binaries, to secure digital rights management (DRM) and software licensing logic against extraction, and to harden anti-tamper mechanisms—from anti-cheat systems in online games [2] to OS-level integrity systems such as Windows PatchGuard [16, 60]. On the adversarial side, the same techniques

are often applied to malware to evade signature-based detection and slow reverse engineering efforts [64]. Since these transformations are structurally identical in both cases, intent cannot be inferred from the binary alone, motivating the deobfuscation work discussed in section 2.3.

While obfuscation can be applied at various levels of abstraction—source code, bytecode, or machine code—the most practically significant obfuscation for analysis tools occurs at the *binary* level [64, 67]. Once compiled and distributed, a binary is what analysts, researchers, and tools encounter. Binary-level transformations are particularly resistant to analysis because they operate below the abstraction level of any source semantics: there are no type annotations, no variable names, and no high-level control structures available as aids. As a result, the remainder of this section focuses on the binary-level obfuscation landscape that this thesis addresses.

2.1.3 Binary Obfuscation Techniques

Loosely following Collberg et al.'s foundational taxonomy [17], binary obfuscation techniques can be grouped by the primary analysis surfaces they protect: the control-flow structure of the program, the values and computations it performs, and the model under which it executes.

Control-Flow Obfuscation

Control-flow obfuscations distort the structure of the program's control-flow graph (CFG) as seen by binary analysis tools. CFG recovery is a prerequisite for most binary analysis steps: program analysis first requires knowledge of *what* is executed [25]. We describe the common techniques that target this surface.

Opaque predicates (Figure 2.1) insert branch conditions whose outcome is always predetermined yet appear data-dependent [18]. A predicate that always evaluates to true leads to one live and one dead block; statically, however, both appear reachable. The dead block may contain garbage bytes, invalid instructions or data. The apparent CFG therefore overapproximates the true CFG, polluting reachability information and inflating the search space for any analysis that follows. This was one of the techniques applied in the motivating example in Figure 1.2 (transformation (3)).

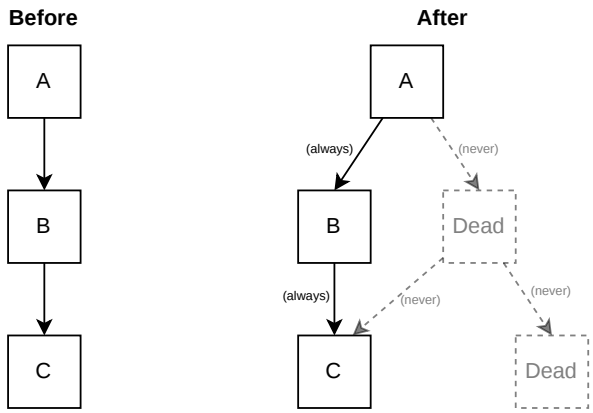


Figure 2.1: Opaque predicate insertion. An unconditional edge from Block A to Block B is replaced by a conditional branch on some predicate which always holds. The dead successors (dashed) are visible in the static CFG but never executed, causing analysis tools to reason about infeasible paths.

Control-flow flattening (CFF) replaces a structured function with a dispatching loop driven by a state variable [42]. Each original basic block becomes a case in a switch-like structure; after executing, each case updates the state variable and returns control to the dispatcher. The resulting CFG becomes flat: the original loop hierarchies, conditionals, and sequential ordering are entirely hidden behind the dispatcher.

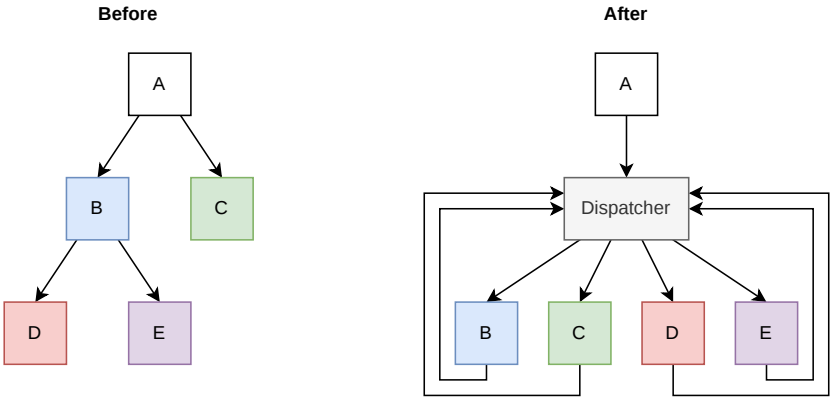


Figure 2.2: Control-flow flattening. A structured function (left) whose blocks are connected by natural conditional and sequential edges is rewritten by CFF (right): every block becomes a case dispatched by a central *dispatcher*, operating on a *state variable* that encodes the program's control state.

Anti-disassembly techniques (Figure 2.3) exploit assumptions made by binary disassembler tools [48]. Overlapping instruction sequences, misaligned branch targets, and deliberately crafted byte patterns can cause a disassembler to decode garbage from within a multi-byte instruction. The result is an incorrect disassembly: instruction, basic-block, and function boundaries are misidentified, corrupting every subsequent higher-level analysis.

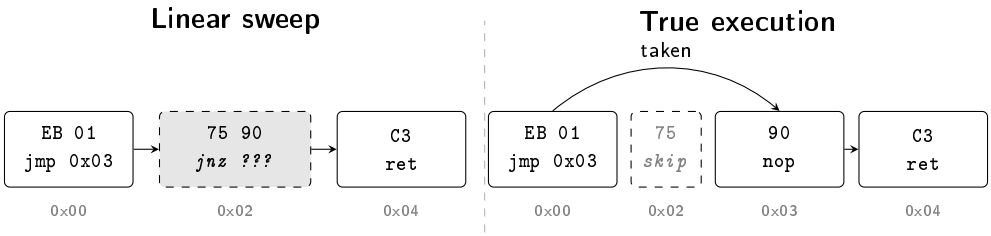


Figure 2.3: Anti-disassembly by junk-byte insertion. The byte sequence at 0x00–0x04 contains a short jump (`EB 01`) whose target is 0x03, placing one junk byte (75) at 0x02. A linear-sweep disassembler (left) decodes sequentially: after the jump it reads `75 90` as a spurious conditional branch, misaligning every subsequent instruction boundary. A control-flow-aware disassembler (right) follows the taken jump to 0x03, never decodes the junk byte, and recovers the correct sequence `nop`, `ret`. The misalignment compounds as the linear sweep continues past 0x04 [48].

Indirect control flow (Figure 2.4) replaces direct call and jump targets with addresses computed from data [76]. A simple unconditional branch to a fixed label becomes a load from a computed address, a table lookup, or an arithmetic expression over runtime state. Wang et al. show that recovering precise targets in such schemes is NP-hard in the general case [76]. Indirect calls are particularly disruptive because they sever the caller-callee relationship that inter-procedural analyses depend on, and because they interact with CFG recovery: an indirect branch whose target set cannot be resolved leaves an open edge in the graph. This was one of the techniques applied in the motivating example in Figure 1.2.

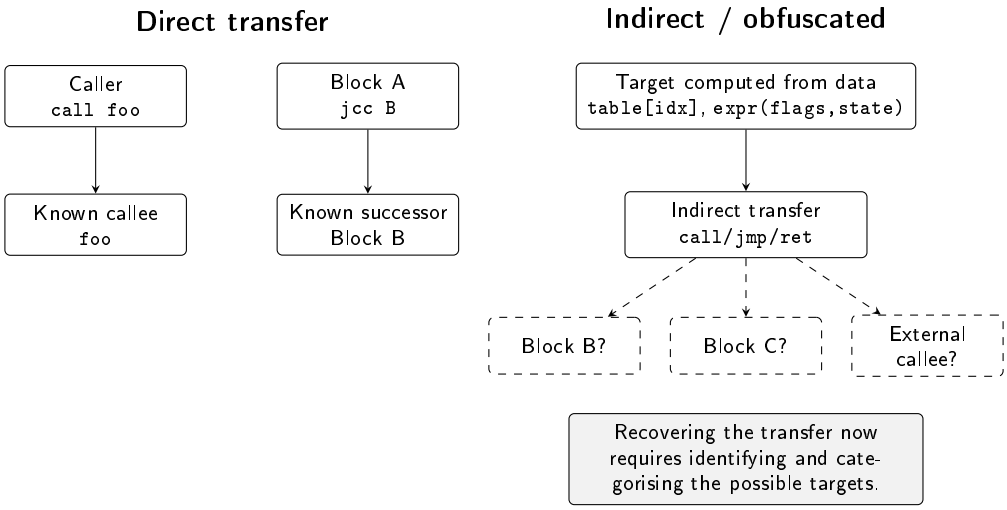


Figure 2.4: Indirect control-flow obfuscation hides explicit edges. Direct branches and calls expose their targets syntactically, but an obfuscated transfer first computes an address and then performs an indirect control transfer; static analysis therefore does not see the implicit CFG edges, and has ambiguity over whether the transfer remains intra-procedural or leaves the function [76].

Data and Value Obfuscation

Where control-flow obfuscation distorts the program's structure, data obfuscation distorts the *meaning* of its computations. Even when a CFG is correctly recovered, the values flowing through it may have been transformed beyond what standard analysis tooling can simplify.

Constant and data encoding replaces constants and expressions with equivalent but more complex variants. A literal constant or arithmetic operation is replaced by an equivalent expression that performs the same computation from less transparent operands, obscuring the original meaning [67]. An example is the use of algebraic or DeMorgan's laws to rewrite a single variable into a more complex form:

$$x \equiv \big(x \oplus (y \oplus y)\big) \wedge \neg\big(\neg x \wedge (y \vee \neg y)\big)$$

Related techniques store values in writable data sections, which intra-procedural static analysis cannot safely treat as constants without whole-binary data-flow reasoning.

Mixed Boolean-Arithmetic (MBA) transforms, are a relatively novel technique that further increases complexity by intermixing arithmetic and bitwise operations in a single expression [79]. A simple example is the identity

$$x + y \equiv (x \oplus y) \;+\; 2\,(x \wedge y),$$

which replaces an addition with a combination of XOR and AND that requires reasoning in both the Boolean and integer domains. Zhou et al. demonstrate that MBA transforms can be applied to encode constants and expressions, creating NP-hard problems for automated tools [79]. A variant of this technique was applied in the motivating example in Figure 1.2 to obfuscate the condition `key == 0xA3F1`.

Virtualisation-Based Obfuscation

Virtualisation-based obfuscation represents the extreme point on the obfuscation spectrum. Rather than distorting the native binary's structure, it translates the protected code into a *custom bytecode* for a purpose-built virtual machine (VM) embedded within the binary itself [63]. The native processor never directly executes the protected logic; instead, a VM interpreter, itself a native binary, fetches, decodes, and dispatches the bytecode at runtime. Commercial protectors such as VMProtect [75] and Themida [56] deploy this approach at scale.

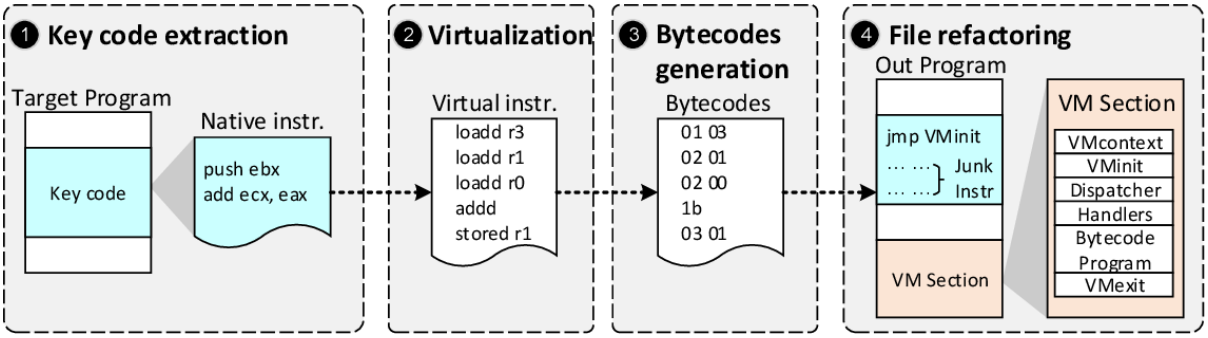


Figure 2.5: A classical process for VM-based code obfuscation. To obfuscate the code, we first dissassemble the code region to be protected into native assembly code (1). The assembly code will be mapped into our virtual instructions (2) which will then be encoded into a bytecode format (3). Finally, the generated bytecode will be inserted into a specific region of the binary which is linked with a VM library (4). Reproduced from Kuang et al. [41], licensed under CC BY-NC-ND 4.0.

Virtualisation is more resistant to analysis than structural obfuscation because it invalidates the assumption that the binary's machine-code representation corresponds to the logic being executed. Rolles illustrates that reversing a virtualisation protector requires at minimum recovering the VM architecture, dispatch loop, and instruction semantics, demonstrating this complexity [63].

2.1.4 Layered Composition and Compounding Difficulty

The aforementioned techniques are rarely applied in isolation. Real-world obfuscators routinely combine several techniques within a single binary [67, 77]. Xu et al. characterise this as *layered obfuscation*: individual transformations are composed into multi-layer protection stacks whose combined difficulty substantially exceeds that of any individual layer [77]. An MBA-derived indirect branch (as seen in Figure 1.2), for instance, requires an analyser to solve both structural and semantic problems simultaneously. This compositionality makes universal deobfuscation difficult, because tools built to defeat one technique in isolation can fail when their input has already been altered by another obfuscation [67, 77].

Toolchains such as Obfuscator-LLVM (OLLVM) [38] and Polaris [4], demonstrate this layered approach by providing several of the aforementioned techniques as compiler passes that can be applied in sequence to produce a multi-layer obfuscated binary. The same is seen in commercial obfuscators: VMProtect and Themida both apply multiple techniques in combination, including indirect branching, opaque predicates, and virtualisation [56, 75].

2.1.5 Consequences for Binary Analysis

The technique families surveyed above converge on a set of concrete analysis failure modes that motivate the following sections.

Disassembly becomes unsound. Anti-disassembly techniques cause disassemblers to misidentify instruction boundaries, producing an incorrect symbolic representation of the binary [48]. Every subsequent analysis then inherits these errors.

CFG recovery becomes uncertain. Opaque predicates introduce edges to dead or misleading blocks; indirect control flow destroys static edges; control-flow flattening complicates basic-block dependency chains. The resulting CFG over-approximates the true graph in some edges and under-approximates it in others. Analyses that reason over this graph consider impossible paths and miss real ones [64].

Symbolic execution faces path explosion and semantic opacity. Symbolic execution, often proposed as an alternative to static analysis, scales poorly against obfuscated code [5]. Opaque predicates and flattening inflate the number of paths the engine must explore; MBA expressions generate satisfiability modulo theory (SMT) queries that solvers struggle to discharge [7]. Banescu et al. demonstrate that techniques specifically designed to produce SMT-hard expressions can render symbolic exploration practically infeasible [7].

No single-technique deobfuscator suffices. Because real-world obfuscators layer transformations, a deobfuscator that focuses on one target can fail at the next. This necessitates analysis approaches that concurrently tackle all protections at once, as opposed to individually [67, 77].

These failure modes collectively motivate the remainder of this chapter.

2.2 Binary Lifting & Intermediate Representations

2.2.1 Intermediate Representations

A compiler *intermediate representation* (IR) is a language-independent, semantics-carrying program representation that sits between a source language and a target machine [1]. In the compilation domain, IRs enable analysis passes—such as dead code elimination, constant propagation or loop transformations—to operate on a uniform representation irrespective of source language or target architecture. Reverse engineering has a similar need. Machine code exposes little structure for automated reasoning, whereas a semantics-rich representation allows for the same family of analyses that compilers apply to source code [69]. The remainder of this section establishes which IR, and which lifting toolchain, satisfies these requirements, and provides the framework used throughout this thesis.

LLVM IR is a typed, language-independent low-level representation in static single assignment (SSA) form, where each variable is assigned exactly once, making data-flow information immediately available and simplifying many classical analyses and optimisations [1, 43]. Its type system exposes primitives common to high-level language semantics while remaining suf-

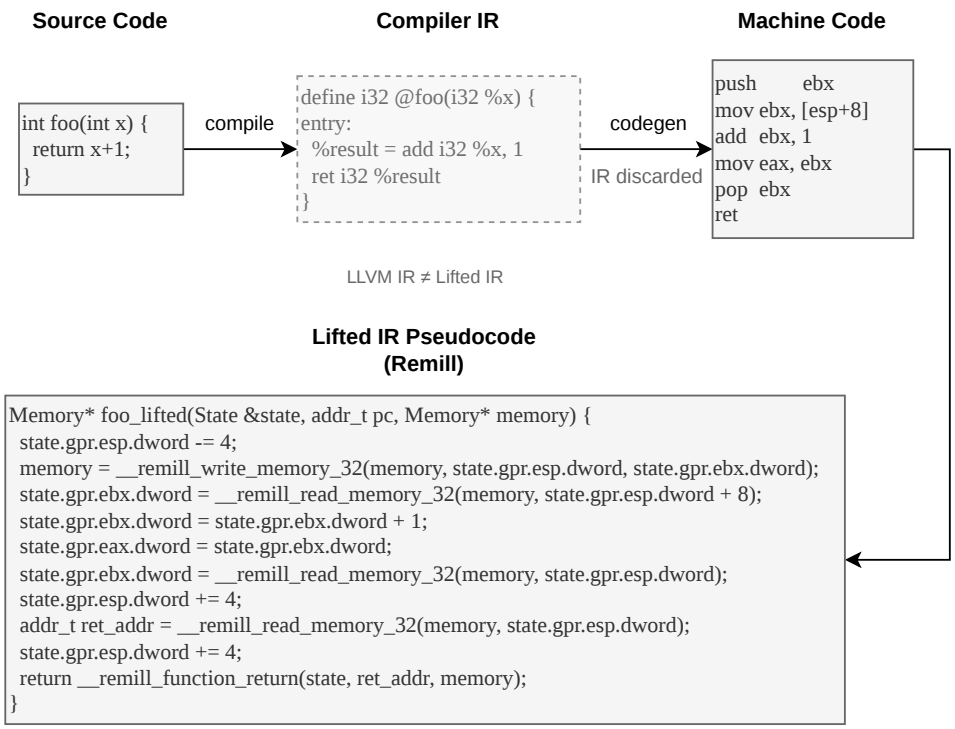


Figure 2.6: Compilation discards the compiler IR once machine code is emitted; the binary carries no IR. Lifting via Remill reconstructs LLVM bitcode instruction-by-instruction: every instruction's semantics are modelled explicitly (modelling of flags and instruction-pointer updates are omitted for brevity). The resulting IR is highly verbose and structurally different from the original compiler IR, and does not recover source-level abstractions. Lifted IR presented in C-like pseudocode for readability; the actual IR is in LLVM bitcode.

ficiently low-level to represent machine-level operations accurately. The pass infrastructure allows transformations to be composed and sequenced over this representation, irrespective of the language from which it was originally generated [43, 51]. These properties make LLVM IR an attractive transformation framework not only for compilation but also for obfuscation. Obfuscator-LLVM (OLLVM) and Polaris apply several variants of obfuscations as *LLVM compiler passes* operating directly on LLVM IR, inheriting language and architecture independence as a result [4, 38].

2.2.2 Binary Lifting

Binary lifting is the process of translating a compiled executable into a compiler-framework IR so that existing compiler analyses and tools can be applied to code that no longer has source [39, 72]. Practical applications include binary rewriting, sanitisation, fuzzing instrumentation, and binary diffing—tasks for which mature LLVM-based tooling exists and which become accessible once a binary is lifted [50, 72]. Lifting to LLVM IR does not, however, recover the original compiler IR. Lifters produce a reconstructed machine-semantics representation, generated *instruction-by-instruction* from the binary's observable behaviour rather than

from a source-level program (Figure 2.6) [39]. Liu et al. confirm this distinction empirically: modern static lifters can produce functionally correct LLVM IR even for complex binaries, and the resulting code is well-suited to binary similarity analysis and decompilation, yet it remains poorly suited to rigorous static analyses such as pointer analysis, due to the loss of source-level abstractions during the compilation process [50].

Remill is an instruction-semantics lifting library that translates machine-code instructions to LLVM bitcode [73]. Its architecture separates two concerns: Remill lifts and models individual instruction semantics precisely, while downstream consumers—such as McSema [72]—are responsible for CFG recovery and whole-function assembly.

The entire register state of the modelled processor is represented as a single `State` structure, passed by pointer into every lifted IR function. Intrinsics are injected for various effects that cannot be modelled directly by the `State`—such as memory accesses, flag updates, and system calls—so that operations that cannot be represented by the `State` alone remain distinguishable from the IR which hosts it.

```
// Simplified for illustration purposes
struct Reg {
    uint32_t dword; // e.g. EAX (32-bit)
    uint32_t _pad;
    uint64_t qword; // e.g. RAX (64-bit)
};

struct GPR {
    Reg rax, rbx, rcx, rdx;
    Reg rsi, rdi, rsp, rbp;
    Reg r8, r9, r10, r11, r12, r13, r14, r15;
    Reg rip; // instruction pointer
};

struct State {
    ArithFlags flags; // ZF, CF, SF, OF, ...
    GPR        gpr;   // general-purpose registers
    SIMD       simd; // XMM/YMM/ZMM vector registers
};
```

Figure 2.7: A simplified representation of Remill's State structure for x86-64; the actual struct is 2688 bytes and additionally includes significantly more fields, necessary to model the full CPU state.

An opaque `Memory` pointer is chained through every intrinsic, in order to preserve memory

ordering and prevent the LLVM optimiser from re-arranging calls that depend on the memory state. A standard Remill-lifted function has the signature:

```
Memory* __remill_function(State* state, uint64_t pc, Memory* mem)
```

The `state` parameter corresponds to the function's initial CPU state and is updated with the semantics of the lifted instructions after execution, while the `pc` parameter denotes the initial value of the program counter (PC) (Figure 2.6).

2.2.3 Challenges in the Context of Obfuscation

Producing any lifted IR requires knowledge of where the program's instructions are located, necessitating CFG and function-boundary recovery [25]. Di Federico et al. demonstrate that this prerequisite is difficult to satisfy even without obfuscation: tail calls, noreturn functions, predicate instructions, and context-dependent control flow each present hard cases for static recovery [25]. In obfuscated binaries these challenges compound: control-flow obfuscations (subsection 2.1.3) deliberately resist such analyses [3, 48]. Lifter correctness is also non-trivial; testing has uncovered previously unknown semantic bugs in mature lifting tools [22, 39].

2.3 Binary Deobfuscation

2.3.1 Definition and Purpose

Binary deobfuscation is the converse of obfuscation: it seeks to recover a program representation whose control-flow structure and computational semantics are accessible to automated analysis, even when the original binary has been deliberately rendered opaque [74]. The goal is not necessarily to reconstruct original source code or compiler IR; it is to produce a functionally equivalent representation from which meaningful program properties can be extracted [67, 74].

The need for deobfuscation follows directly from the analysis failure modes established in section 2.1. Because obfuscation is by definition semantics-preserving, the obfuscated binary and its deobfuscated form compute identical results on every terminating input—yet the syntactic distance between them may be vast [17]. Deobfuscation aims to close that syntactic gap, restoring a form that binary analysis frameworks, decompilers, and other reverse engineering tools—which expect unobfuscated code—can reason about.

2.3.2 Deobfuscation Approaches

The surveyed work admits organisation into several deobfuscation families distinguished by their primary analytical strategy [28, 40, 67]; Table 2.1 presents this synthesis. Each family

makes progress on the deobfuscation problem, but none provides a general system for simultaneously recovering control-flow structure, simplifying data operations, and handling arbitrary obfuscation compositions [28, 40, 77].

Table 2.1: Principal binary deobfuscation technique families and their key limitations.

Family	Representative work	Key limitation
Static/dynamic analysis	Udupa et al. [74]	Brittle against layered and VM-based obfuscation [40, 67]
Semantics-preserving simplification	Yadegari et al. [78]	Relies on dynamic traces; limited by path explosion [5, 78]
Symbolic / DSE	Bardin et al. [9]; Li et al. [46]	Solver limits; semantic confusion in VM handlers [5, 46]
Program synthesis	Blazytko et al. [12]; Coniglio [19]	Scales poorly to complex or nested obfuscation [12, 28]
Virtualisation-targeted	Liang et al. [47]; Salwan et al. [65]	Assumes VM-specific structure; limited generality [28, 40]
Expression-level (MBA)	Eyrolles et al. [26]; Liu et al. [49]	Domain-narrow; does not address whole-program CFG recovery [26, 49, 62]
LLVM-based optimisation	Garba & Favaro [33]	Not publicly released; hard obfuscation classes remain [33]

Early Pattern- and Analysis-Driven Deobfuscation

The earliest examples of automatic deobfuscation appear in Udupa, Debray, and Madou's 2005 study, which demonstrated that a large fraction of practically deployed obfuscation could be defeated by combining standard static analyses—dataflow propagation, dead-code elimination, constant folding—with targeted dynamic observations to resolve value-dependent control flow [74]. Their results were notable for showing that standard analysis infrastructure could recover a structurally simpler binary from the range of prevalent obfuscation schemes of the time, without hand-written per-scheme rules.

This line of work established that deobfuscation is most naturally framed as a program analysis problem. Its fundamental limitation is that the analyses are pattern-driven, and become progressively less effective as obfuscation becomes deeper, more layered, or more semantically opaque [67, 74].

Symbolic and Semantics-Based Deobfuscation

A more general alternative replaces structural pattern recognition with reasoning about program's *semantics*. Yadegari et al.'s generic approach to automatic deobfuscation demonstrates

this direction: rather than matching obfuscation artefacts syntactically, it applies semantics-preserving transformations derived from observed execution traces to simplify code without assuming any particular obfuscation scheme, demonstrating effectiveness against emulation-based obfuscation, runtime code unpacking, and return-oriented programming [78].

Symbolic methods remain bounded by practical solver limits; as established in section 2.1, obfuscation can be deliberately constructed to produce SMT-hard branch conditions, rendering pure symbolic deobfuscation practically infeasible in adversarial settings [7].

Program-Synthesis-Based Deobfuscation

Program synthesis recovers an *equivalent simpler semantics* by observing input/output behaviour rather than recovering structural form. Blazytko et al.'s Syntia synthesises simpler expressions matching an obfuscated fragment's observed behaviour [12]; Coniglio combines this with symbolic execution to constrain the search space [19]. The approach is attractive where neither pattern matching nor symbolic execution can simplify an expression, but scales poorly: as fragment complexity grows, the sampling costs expand rapidly, making synthesis impractical for larger or nested obfuscation [12, 19].

Virtualisation- and MBA-Targeted Deobfuscation

Specialised tools exist for the most technically demanding obfuscation classes. Devirtualisation approaches such as those of Liang et al. and Salwan et al. combine symbolic execution with trace or taint analysis to recover native-equivalent code; Kochberger et al.'s survey confirms that such tools remain effective only in targeted scenarios [40, 47, 65].

At the expression level, MBA simplification tools such as Eyrolles et al.'s pattern-based approach and Liu et al.'s MBA-Blast provide effective simplification within their domain [26, 49]; more recent tools such as SiMBA and GAMBA have since been implemented directly as LLVM passes, illustrating the natural fit between LLVM IR and deobfuscation, as explored further in the following sections [61, 62]. All of these approaches are narrow in scope: they address specific obfuscation techniques rather than providing a general solution.

2.3.3 SATURN: Deobfuscation by Optimisation

Core Idea

SATURN, proposed by Garba and Favaro in 2019, proceeds from a direct observation about the landscape above: compiler-based obfuscation is injected at the IR level by tools such as OLLVM [38], so the natural response is to lift the binary to the same level and address the obfuscation there, rather than attempting to undo it at the binary level [33]. This makes SATURN not merely another targeted technique but the *substrate* on which targeted ap-

proaches become composable: on LLVM IR, CFG recovery is an integrated lifting step rather than a separate prerequisite, and approaches such as expression simplification, synthesis-based opaque predicate resolution, and devirtualisation can become a series of custom LLVM passes operating on the same recovered representation within the same pipeline.

Why LLVM IR Is the Enabling Substrate

The choice of LLVM IR as the target representation is not incidental. As established in the preceding section, LLVM provides a typed, SSA-form representation with a mature ecosystem of analysis and transformation passes [43]. SATURN's key contribution is not merely the idea of lifting first, but the insight that lifting into LLVM IR specifically makes the full optimisation pipeline available as deobfuscation infrastructure without requiring any of it to be reimplemented from scratch. The LLVM ecosystem's analysis infrastructure becomes available as a direct consequence of lifting to LLVM IR.

Iterative CFG Construction

A key innovation of SATURN is its iterative CFG recovery algorithm, necessitated by precisely the challenges introduced in section 2.1. A single-pass lifting strategy cannot resolve these statically, and a complete static CFG (which existing end-to-end lifting tools such as McSema depend on [72]) is not available to guide the lifting process [33].

SATURN addresses this through an iterative fixpoint loop (Figure 2.8):

1. **Lift** the current basic block to LLVM IR, starting from a known entry point.
2. **Build a partial CFG** consisting of all reachable, previously-discovered predecessor blocks.
3. **Construct a program counter (PC) slice** on this partial CFG: a new LLVM IR function that calls the original with an opaque `State` and the initial instruction's address, returning the `State`'s PC after the original's semantics are applied (Figure 2.9). The result is the PC reached *after* the lifted code executes.
4. **Optimise and simplify** the sliced function using LLVM passes. In many cases this alone resolves the branch target. When it does not, custom LLVM passes can be written to target specific obfuscation methods.
5. **Collect successors** by inspecting the return expression or constant, then enqueue discovered blocks.

When optimisation is insufficient, SATURN employs the LLVM-based SMT infrastructure available in the ecosystem to attempt to resolve the target by symbolic means. After target resolution, the resolved edges are added and the loop continues with the remaining blocks.

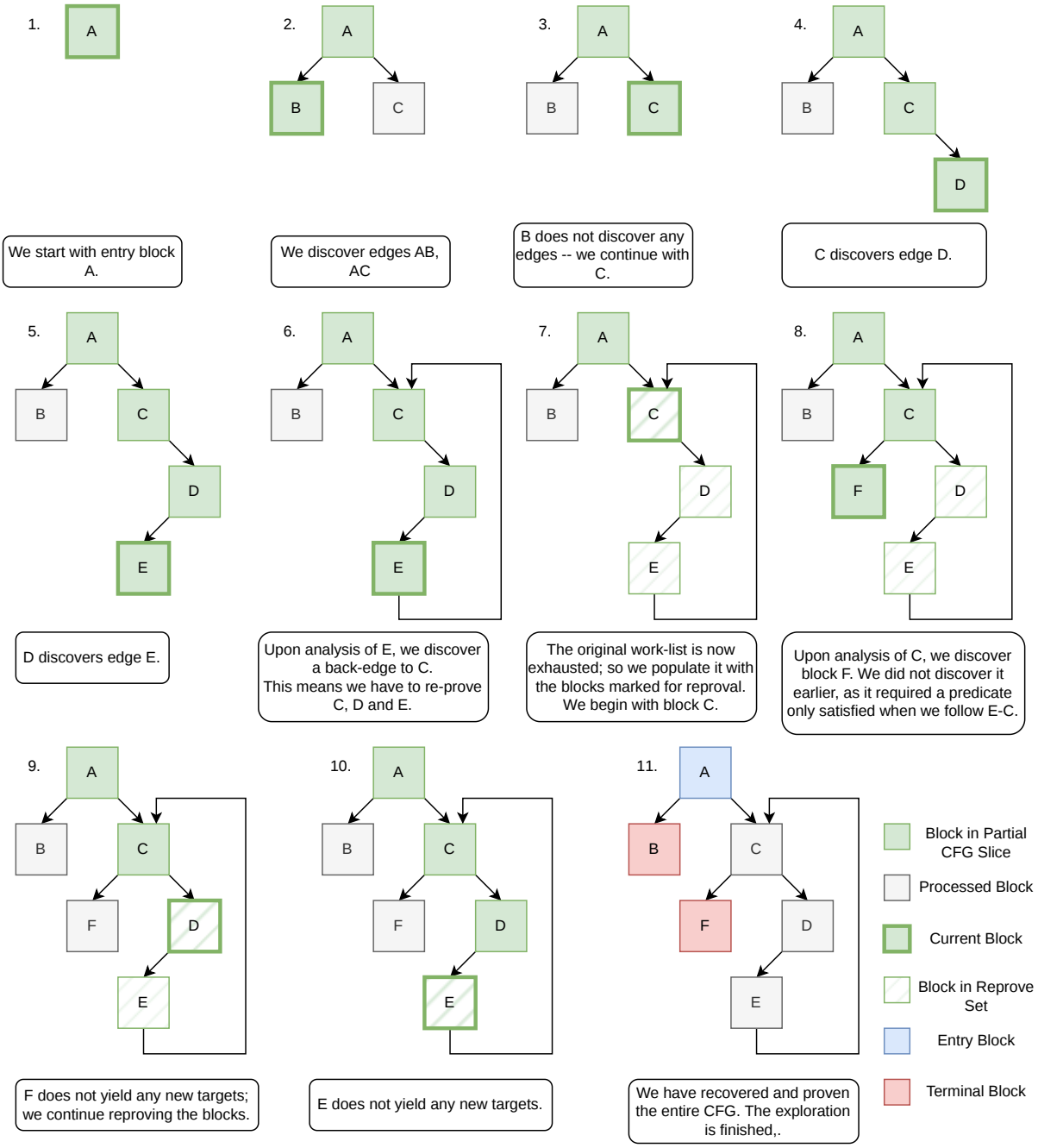


Figure 2.8: SATURN-style iterative CFG exploration: the analysis maintains a partial CFG, processes one frontier block at a time, adds newly proven successors, and revisits previously seen blocks when a new incoming edge changes what must be proved.

This algorithm provides an effective manner to explore the program's CFG while skipping opaque predicates that are not actually reachable. The loop repeats until no new successors can be discovered—the fixpoint condition.

By relying on LLVM's high-performance optimisation passes first, before resorting to more expensive symbolic techniques, SATURN's approach becomes practical for real-world obfuscation applications. LLVM's optimisation passes resolve several computationally-trivial obfuscation constructs, the solver is invoked only for the cases that require deeper reasoning. Furthermore,

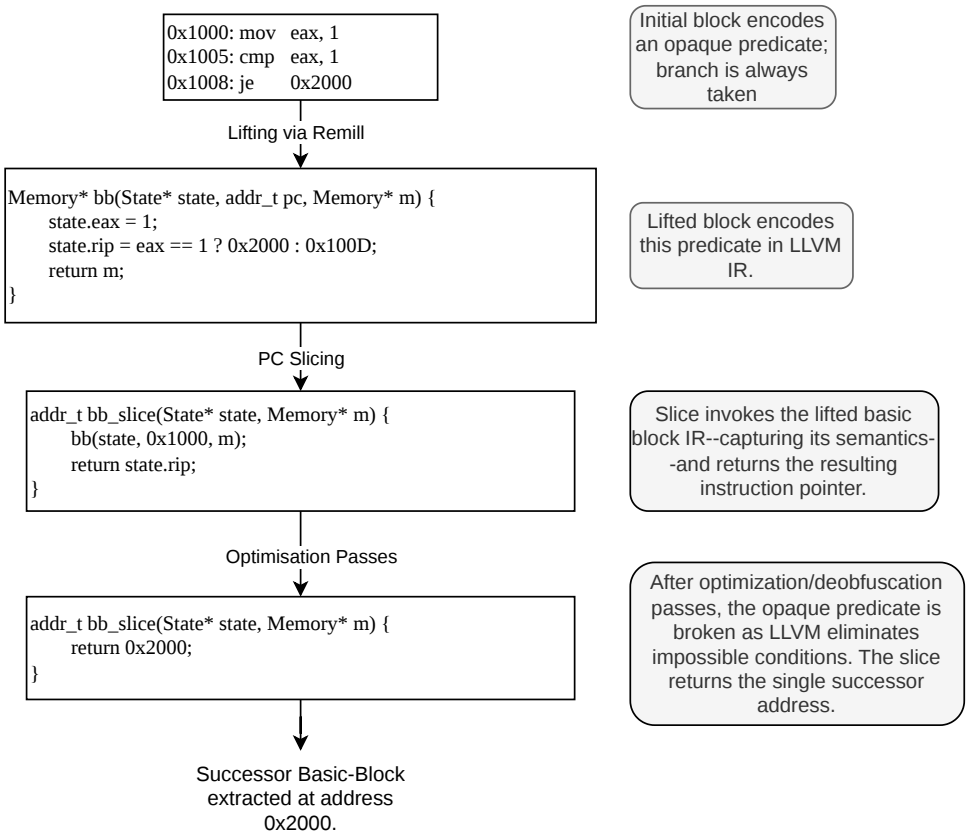


Figure 2.9: SATURN program counter (PC) slicing example. A block's successors are discovered through optimisation of a slice that computes the PC after executing the block.

since tools such as Souper [66]—a SMT-based superoptimiser for LLVM IR—are integrated directly into the LLVM infrastructure, SMT-backed simplification operates natively on the same IR as the optimisation pipeline, without a separate translation layer.

Post-Lifting Cleanup and Output

Beyond CFG reconstruction, SATURN applies a *brightening* stage that adapts the lifted LLVM IR to a more analysis-ready form, removing lifting artefacts introduced by Remill. The resulting IR can then be subjected to further LLVM deobfuscation passes and finally undergoes *recompilation* into a cleaned native binary, completing the binary-to-binary deobfuscation process. The most significant aspect for this thesis is that the lifting stage produces an LLVM IR module which models the obfuscated program's semantics and can be further used by downstream tooling.

Significance and Remaining Limitations

SATURN is significant because it demonstrates deobfuscation by optimisation in a real, evaluated framework, showing that compiler infrastructure can serve as the primary deobfuscation mechanism. Research did not present a prior framework that had unified binary lifting, iterative

CFG exploration, and LLVM-level simplification in this way.

Despite these contributions, SATURN carries several constraints of note. The implementation has not been released publicly: neither source code nor compiled binaries are available—only a set of test results has been published [32]. This has motivated the creation of open-source prototypes such as Mergen [55] and Dna [70], drawing inspiration from SATURN's design. However, these are research prototypes rather than peer-reviewed, evaluated frameworks, and neither provides the extensible, modular architecture or verification infrastructure this thesis requires.

The SATURN paper is also a high-level description of the approach rather than a detailed technical manual, and does not provide a comprehensive description of the implementation. Specifically, it does not describe the handling of certain scenarios that can arise in real-world software, such as inter-procedural control-flow.

2.3.4 Confidence in Deobfuscated Output

A separate, still under-addressed question is equivalence checking: most published deobfuscation evaluations report simplified structure, code-size changes, or agreement on selected executions rather than automated semantic-equivalence checking [28, 33, 65, 78]. This is informative but does not constitute evidence of correctness: there is no automated mechanism to detect whether a deobfuscation pass has introduced a semantic difference that the test suite did not cover. Feng and Xu's recent DEBRA benchmark demonstrates this problem empirically: virtually all existing deobfuscation tools are evaluated against small, artificially obfuscated programs rather than real-world software at scale, and real-world inputs exposed crashes and incorrect simplifications that the original tool evaluations had not surfaced [28].

The following section examines what the existing verification literature contributes to this problem, and what constraints the binary-and-lifted-IR setting imposes.

2.4 Verification and Differential Testing

2.4.1 Approaches to Program Equivalence

In this context, *software verification* relates to the problem of establishing whether a program transformation preserved the semantics of the original artefact [45]. The central notion is *semantic equivalence*: two programs are equivalent when they produce identical observable outputs on every input [1]. This is precisely the property at stake after deobfuscation: since obfuscation is by definition semantics-preserving [17], a correct deobfuscation must be as well.

One approach is to formally verify the transformation tool itself, proving definitively that it

always preserves semantics, as demonstrated for compilers [45] and individual transformation passes [53]. While this provides the strongest possible guarantees, it is rarely tractable for complex, heuristic-driven deobfuscation pipelines.

A more practical framing is *translation validation*, introduced by Pnueli et al.: rather than proving the transformation tool correct in general, one validates each individual run by checking that its output correctly implements the semantics of its input [58]. This shifts the burden from a single global proof to a series of bounded, per-transformation checks, making it well-suited to the incremental nature of deobfuscation.

Several techniques exist for discharging these per-transformation equivalence checks, differing significantly in the confidence they provide and the cost they incur.

- **Concrete and differential testing** executes the original and transformed program on the same inputs and compares outputs. It is cheap, easily automated, and fast; its fundamental limitation is that it *samples* behaviour rather than characterising it [34].
- **Symbolic execution** replaces concrete inputs with symbolic variables and uses a constraint solver to explore many execution paths simultaneously, providing stronger path coverage without enumerating every concrete input [5]. In practice it is bounded by path explosion, solver limits, and the difficulty of modelling external environments [5].
- **Concolic testing** tightly couples symbolic and concrete execution: concrete runs guide the symbolic engine, and the solver generates new inputs targeting previously unexplored paths [68]. Hybrid approaches further combine cheap fuzzing for rapid surface coverage with selective concolic execution on hard-to-reach paths [71], achieving better coverage than either technique in isolation.

Each method occupies a distinct point on the confidence-cost spectrum, and the choice between them depends on the target setting.

2.4.2 Verification for LLVM IR and Binary Code

Because LLVM IR is a typed, semantics-carrying representation with a well-defined operational model, it allows for verification tooling that raw binaries do not [43]. Crucially, however, the tractability of these tools depends on structural properties that *compiled* IR provides but that *lifted* IR does not.

The landmark example is *Alive2*, an automated bounded translation validation tool for LLVM optimisation passes [52]. Alive2 encodes the semantics of each LLVM IR rewrite into SMT constraints and automatically checks whether the output is a correct refinement of the input. Its feasibility depends on the type, alignment, and pointer-provenance information that compiled IR carries: Lee et al. show that without such metadata (as is the case for Remill-

lifted IR; see Figure 2.6), the SMT encoding must consider that each memory operation may alias with any live object, producing a combinatorial case split that rapidly overwhelms the solver [44]. Berger et al.'s *arm-tv*, the closest attempt to apply Alive2 to assembly-level lifted code, required approximately 14,400 lines of modifications to Alive2, found that byte-granularity lifted memory operations defeated Alive2's alignment-based encoding optimisation entirely [11]. These results indicate that Alive2's approach does not transfer to the machine-semantics IR that binary lifting produces.

KLEE, a symbolic execution framework for LLVM, provides complementary capability by allowing inputs to be defined as *symbolic*, enabling differential checking between original and transformed programs by comparing symbolic outputs [15]. KLEE faces the analogous structural constraint: symbolic addresses over flat, byte-granularity memory can reference any cell, and without the pointer provenance and type information that compiled IR provides, the resulting state space becomes intractable [5].

At the binary level, the correctness of the lifting step itself also cannot be assumed: as established in section 2.2, differential testing has shown that mature lifters contain semantic bugs that cause lifted IR to differ from the binary's actual behaviour on inputs that expose certain edge cases [39]. Dasgupta et al. develop a scalable validation framework that directly checks whether lifted LLVM IR correctly implements the binary's semantics, making binary-to-IR equivalence reasoning both mechanisable and practically tractable [22]. These results establish two complementary points: binary equivalence reasoning through lifting is feasible, and the lift is not free—the lifter's own correctness must be established or explicitly assumed.

2.4.3 Challenges in the Deobfuscation Setting

Deobfuscation compounds these limitations in two ways. First, lifted IR encodes machine semantics explicitly; source-level abstractions such as types, alignment guarantees, and pointer provenance are lost (see Figure 2.6) [22, 50]. As established above, these are precisely the properties that make SMT-based and symbolic tools such as Alive2 and KLEE tractable on compiled IR.

Second, function-level deobfuscation tools, including SATURN, operate on individual functions extracted from a larger binary without knowledge of the calling context [33]. The verifier therefore cannot know the valid input domain, pointer aliasing state, or memory layout of the function under analysis. Ramos and Engler address precisely this class of problem with *UC-KLEE*, an *under-constrained* symbolic execution framework that analyses functions in isolation by treating all pointer arguments and global state as unconstrained [59]. This strategy avoids the need for whole-program context, but yields over-approximation: the engine may explore states unreachable in any real calling context. UC-KLEE's unavailability, however, limits

applications—no source code or binaries have been made available.

Symbolic and concolic methods face the further burden that obfuscated lifted IR is structurally dense. Opaque predicates inflate path counts, flattening dispatchers generate deep repetitive path trees, and MBA expressions produce SMT queries that resist simplification [5]. These effects compound the path-explosion problem that already limits symbolic methods on clean code.

The reviewed verification literature thus offers useful components in isolation—translation validation for LLVM optimisations, symbolic test generation, under-constrained execution for partial-context function checking, and lifter correctness validation—but not a coherent pipeline for the binary-to-lifted-IR-to-transformed-IR scenario under incomplete calling context. Deobfuscation of lifted machine-semantics IR lies at the intersection of all three paradigms and inherits the compounding limitations of each.

2.5 Gaps in the Literature

We have established that there has been significant progress in the binary deobfuscation problem; innovative techniques (section 2.3) exist that target the exact challenges that obfuscation presents (section 2.1). SATURN demonstrated that that lifting to LLVM IR, an already well-established practice for binary analysis (section 3.2), can be used as a substrate for deobfuscation by optimisation, making available both the aforementioned techniques and the full LLVM optimisation pipeline as deobfuscation infrastructure (subsection 2.3.3). Verification literature has also shown significant progress in establishing the correctness of transformations on compiled LLVM IR (section 3.3). Significant gaps, however, remain in the literature that prevent these advances from coalescing into a practical, extensible, and trustworthy deobfuscation framework for real-world software. These gaps relate directly to the motivations established in section 1.2.

The first gap is infrastructural. SATURN is the clearest statement of deobfuscation-by-optimisation in the literature, and has shown significant promise. Yet no such framework has been publicly released: neither SATURN nor any comparable alternative is available for reuse, leaving subsequent work without a foundation to build upon. It also does not provide a detailed technical description of its implementation, leaving significant questions about *how* its algorithms are implemented unanswered.

The second gap is practical coverage. The SATURN paper demonstrates a full, binary-to-binary deobfuscation pipeline, but its published description leaves several real-program complications—such as inter-procedural calls—under-specified that subsequent analysis and transformation passes depend on. Without these details, it is impossible for the pipeline to function on arbitrary, real-world functions.

The dominant gap, however, is verification. Existing deobfuscation evaluations rely largely on simplification proxies—cleaner CFGs, smaller code, or agreement on limited test inputs. As the DEBRA results discussed in section 2.3 show, such evidence does not survive contact with real-world software [28]. As established in section 2.4, the verification literature provides useful components, but their application to the deobfuscation setting, where context is incomplete and the recovered IR still reflects machine-level semantics, remains unaddressed. Improved lifting infrastructure is itself a prerequisite for any such capability: the LLVM-based tools available are inapplicable to the machine-semantics IR that lifting produces directly, and the verification gap cannot be closed by tool reuse alone. The next chapter therefore turns from literature survey to system design, outlining an architecture that addresses these three gaps together.

3 Design and Implementation

This chapter describes the design and implementation of LIFTED, a framework for lifting and verifying obfuscated binaries, directly addressing the gaps presented in the previous chapter. We first describe the high-level architecture of the system, including the goals and scope of the implementation, and its relation to SATURN. Next, we delve into the technical details of the lifting framework, outlining our contributions and innovations in this area. Finally, we present the verification framework, describing how it integrates with the lifting framework and enables confidence in deobfuscator development.

3.1 System Overview

3.1.1 Goals and Scope

The overarching goal of LIFTED is to provide the security research community with a robust, extensible *pipeline* for deobfuscator development. We build upon prior work in the literature to address three primary gaps in the literature. We define these as our *implementation goals* (IGs).

IG1: An open-source, extensible deobfuscation model. SATURN's general unavailability proved to be a significant barrier to entry for deobfuscation work. LIFTED reimplements the core control-flow exploration algorithm of SATURN, providing the research community with an open-source framework to further iterate and build upon. Like SATURN, we make no assumptions about any particular obfuscation strategies applied; instead, providing the user with an obfuscator-agnostic core alongside obfuscator-specific overrides—an improvement SATURN explicitly mentions as future work [33]—thus allowing them to implement targeted, obfuscator-specific solutions where necessary.

IG2: Critical lifting algorithm improvements to handle real-world binaries. During development and testing, practical cases were encountered that are not fully addressed in SATURN's published description. We introduce two key improvements: a means of handling external, indirect, and recursive calls through a user-extensible callee annotation system, and

block splitting when newly discovered targets fall inside already lifted blocks. SATURN's documented algorithm does not address external or recursive calls, both of which are prevalent in non-trivial real-world software, making these improvements a prerequisite for IG3.

IG3: A tightly-integrated, extensible, and automated verification infrastructure. Verification of the deobfuscation process was identified as a critical gap. We aim to create an infrastructure that allows quick and helpful developer feedback across transformations. LIFTED aims to solve this problem by allowing a user to apply differential testing between *verification artefacts*—representations of the lifted function's semantics in either machine-code (from the original binary) or LLVM IR (as products of the lifting process). Following the limitations discussed in section 2.4, we use a randomised, concrete *differential testing* approach in order to help users gain confidence in their deobfuscation work—a practical tradeoff that focuses on *discovering* bugs rather than proving their absence.

These goals deliberately scope LIFTED to the early stages of the deobfuscation pipeline—specifically, the recovery of lifted IR from an obfuscated binary and its transformation at the IR level. Unlike SATURN, LIFTED does not target end-to-end binary-to-binary output; full recompilation from lifted IR is outside the scope of this work. This scoping decision is intentional: the infrastructural and verification gaps identified in section 2.5 originate precisely in this stage, and addressing them requires dedicated focus.

3.1.2 Relation to SATURN

LIFTED builds directly on SATURN's published design but is not a replica of it. Table 3.1 maps each component of the SATURN pipeline against its counterpart in LIFTED.

The shared foundation is Remill: both systems use it as the instruction-semantics engine, making lifted IR at the basic-block level structurally consistent between them. LIFTED implements SATURN's worklist-based, backward-slice iterative CFG recovery algorithm (Figure 2.8, Figure 2.9), using LLVM optimisation passes to explore the program's CFG. Integration of Souper—SATURN's fallback for constructs that resist compiler optimisation—are outside the current scope.

Three components represent substantive additions beyond reimplementation. First, LIFTED formalises a `LifterCallbacks` interface that decouples the obfuscator-agnostic lifting core from obfuscator-specific target resolution and per-block processing. The SATURN paper explicitly identifies a plugin system of this kind as future work [33]; in LIFTED it is a key architectural decision.

Second, SATURN's documented exploration strategy does not document how to handle inter-procedural calls. LIFTED introduces per-callee contracts driven by a user-defined annotation registry (subsection 3.2.2), in which each callsite is annotated with its calling convention and

memory effects. These are then consumed by an improved core lifting algorithm (subsection 3.2.3).

Third, LIFTED introduces a block-splitting mechanism: when a newly resolved branch target falls within the interior of an already-lifted block, that block is split at the boundary and the CFG updated in place (subsection 3.2.4). SATURN does not describe how this case is handled.

SATURN's post-deobfuscation stages—code brightening and recompilation—are not implemented in LIFTED. Both presuppose a fully finalised deobfuscated function and are beyond the scope of this work. The verification infrastructure (section 3.3) is entirely new and has no counterpart in SATURN.

Table 3.1: Component-level comparison of SATURN [33] and LIFTED.

Component	SATURN	LIFTED
Instruction-level lifting	Remill	Remill
Iterative CFG recovery	Yes	Implemented
Opaque predicate resolution	LLVM + Souper/Z3	LLVM passes only
Post-lifting IR optimisation	LLVM + Souper/Z3	LLVM passes only
Modular callback interface	Future work	New
Indirect call handling	Stops exploration	New
External / recursive call handling	Not described	New
Block splitting	Not described	New
Verification / differential testing	No	New
Code brightening	Yes	Out of scope
Binary injection / recompilation	Yes	Out of scope

3.1.3 End-to-end Architecture

LIFTED is designed as an integrated infrastructure for deobfuscator development rather than as a standalone lifter or verification tool. Starting from an obfuscated binary, the system reconstructs a *single function* into LLVM IR, applies IR-level transformation passes, and verifies the resulting artefacts throughout the process. The key architectural decision is that lifting, transformation, and verification are not treated as separate phases—instead, they are composed into a single workflow in which each newly produced artefact can be checked before it becomes the input to the next stage. This is what enables the pattern of *lift* $\rightarrow$ *verify* $\rightarrow$ *deobfuscate* $\rightarrow$ *verify*, repeated as many times as a deobfuscation strategy requires.

LIFTED is implemented as a C++ library, with a CMake build system and a command-line interface for running the end-to-end pipeline. It focuses on x86-64 binaries, but the design is architecture-agnostic and could be extended to other architectures supported by Remill.

Lifting workflow. The workflow begins with a binary image and a `BinaryContext`, which provides shared information about the binary for the rest of the system. At a basic level, `BinaryContext` owns the binary metadata needed by both lifting and verification. More importantly, it also owns the per-binary annotation registry used to describe the higher-level behaviour of functions that had been lost during the compilation process (Figure 2.6). These function annotations are a prerequisite for later handling of external, indirect and recursive calls. By placing this information in a persistent context object, LIFTED ensures that the lifting, verification, and later analysis components, use the same information.

Lifting itself is performed by the obfuscator-agnostic `Lifter` core. Its responsibility is to recover a function-level control-flow graph from the binary, lift individual basic blocks, and stitch them into a single LLVM IR function. Obfuscator-specific behaviour is isolated behind a dedicated callback interface, `LifterCallbacks`. These callbacks are consulted whenever the `Lifter` needs information that depends on the obfuscation strategy applied to the binary. This separation is central to the design: the core lifter remains reusable across binaries and obfuscators, while obfuscator-specific knowledge is injected when required.

The result of this stage is a lifted LLVM IR artefact which should model the semantics of the original binary function, instruction-by-instruction—we call this the *raw lifted (LLVM) IR*. It is accompanied by the necessary metadata required for further deobfuscation and verification. The remainder of the framework operates on this output.

Transformation and verification workflow. Verification is integrated into this flow at several levels. Following the challenges identified in section 2.4, we take a concrete, differential testing approach to verification, using randomised inputs to explore the behaviour of the artefacts under test. We call two verification artefacts *discrepant* if they produce different outputs on the same input (*Note: in the published codebase and related codebase-derived figures, we may use the term 'divergent' instead—this refers to the same concept in the context of this thesis*). The verification pipeline is designed to run two artefacts with the same input and compare their behaviour, identifying any discrepancies that arise.

We provide two *executors* for executing such artefacts: a just-in-time compilation (JIT)-based executor for lifted and transformed IR, and an emulation-based executor for the original binary. These abstract away the execution environment and allow the system to compare behaviour across different artefact types—such as IR vs binary, IR vs IR, or even binary vs binary.

The infrastructure also supports granular comparisons. The artefacts under test are expected to be synchronised at several points—at which their *intermediate states* can be compared—which means that we can make comparisons at these points *during* execution, rather than relying solely on the final state. This allows for more informative feedback when a discrepancy is detected.

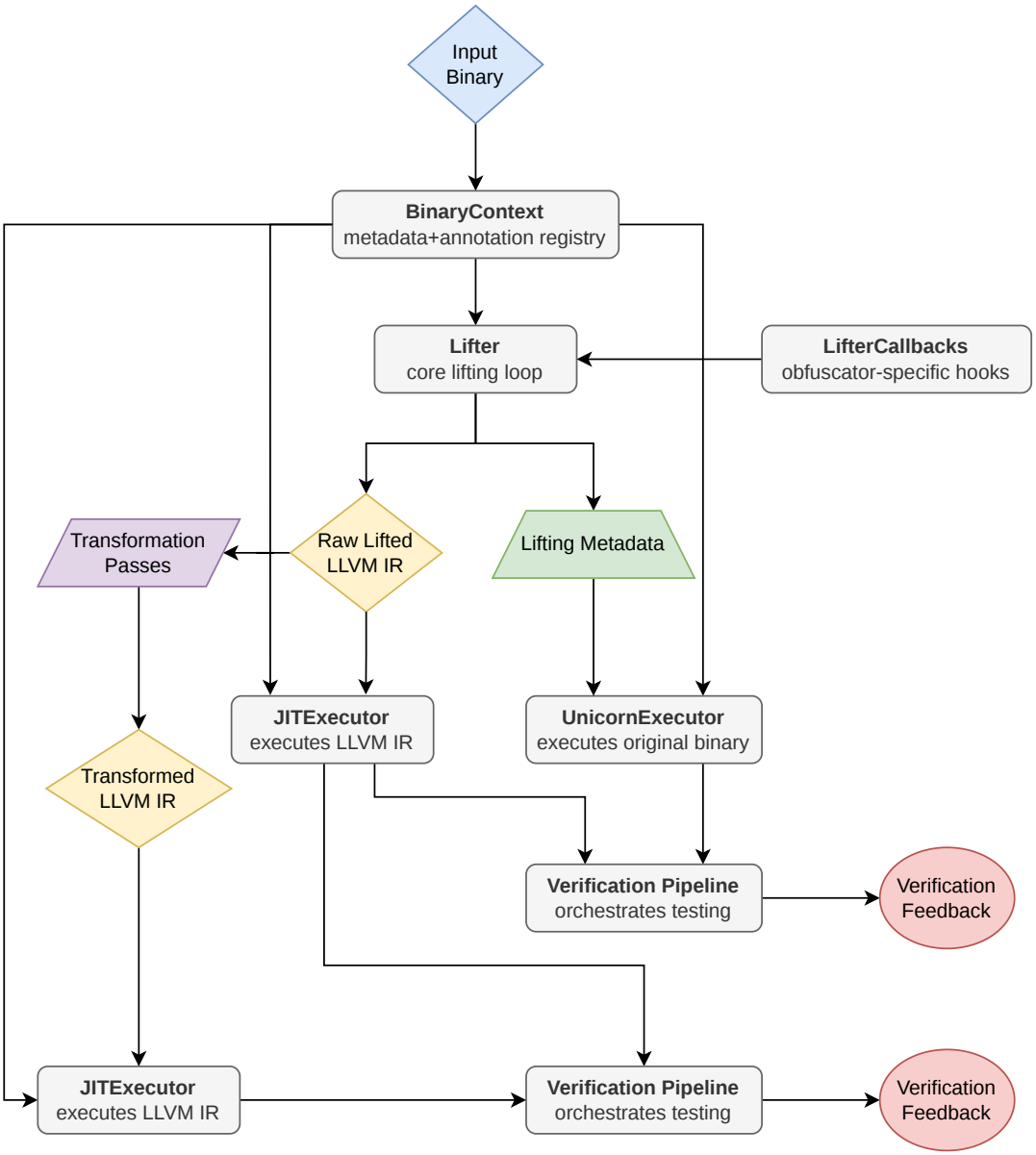


Figure 3.1: High-level architecture of LIFTED. A binary is lifted into LLVM IR, transformed at the IR level, and verified using through appropriate executors across successive stages. The same architecture also produces developer-facing artefacts which aid them in debugging the verification verdict, while `BinaryContext` supplies the annotation context required for later call handling.

After the initial lifting stage, the user may want to apply IR-level transformations, such as deobfuscation passes, to the raw lifted IR. This yields a *transformed (LLVM) IR* artefact, which should still be semantically equivalent to the original binary, but may differ in its structure. A transformation pass may preserve the overall function semantics while modifying the intermediate structure, such as rearranging memory accesses or the control-flow structure. For this reason, LIFTED allows for *configurable* verification profiles. Earlier stages can use a stricter verification profile, synchronising more often and comparing more dimensions because the raw lifted IR should still closely resemble the source execution structure. Later stages can

use a more relaxed profile that checks the semantic dimensions that should remain invariant after structure-changing passes. This enables the architecture to still provide informative feedback, while not inhibiting the performance of transformation passes.

As established in section 2.4, concrete testing samples the function state. For this reason, we provide a verification *pipeline*, divided into *stages*. Each stage runs a user-specified number of individual tests (*runs*), and has several configurable properties. This is done in order to make the system practical in discovering bugs by maximising coverage of the function's state space.

A run does not end with a simple pass/fail verdict—the pipeline records structured reports of which stage failed, which inputs triggered discrepancy, and what categories of behaviour differed. When required, a discrepancy can be replayed in a deep-dive mode that records chronological execution detail on both sides of the comparison, allowing the user to inspect where the two executions first separate. Similarly, the lifted function's CFG can be visualised, making it easier to interpret which regions of the function were explored and executed while testing.

The remainder of this chapter examines the two halves of this architecture in detail. section 3.2 describes the lifting framework, including the annotation-driven call-handling model and the mechanisms needed to recover and update the CFG. section 3.3 then describes the verification framework, including the executor model, the checkpoint system, staged pipelining, and the feedback mechanisms used to diagnose discrepancies.

3.2 The Lifting Framework

3.2.1 Lifting Algorithm

LIFTED's lifting algorithm broadly reimplements SATURN's iterative, worklist-based CFG recovery [33], using Remill [73] as the instruction-semantics engine. SATURN describes the core algorithm at a high level—the implementation details are entirely novel. Details of the exploration algorithm are omitted as they have already been described in subsection 2.3.3—we instead focus on our implementation decisions.

Contexts and caching. A key structural decision is the separation of two kinds of contexts in the *Lifter*. The core idea is that the *Lifter* object exists in a per-binary scope. This means that it contains information that can be persisted across lifts of different functions within that binary, allowing it to *cache* information that may be reused across multiple lifts, such as basic-block bodies. This is important because a basic block, particularly in obfuscated functions, can reappear across multiple functions (for example, shared tail-call targets) [25]. Relying on cached lifting avoids lifting the same instructions twice.

A separate, per-lift context is introduced: `lift_ctx_t`. This is an ephemeral structure that stores the information required for the lifting of a single function. Among other data, it contains the per-function CFG topology—such edge information between blocks—as well as the worklist and current state of the lifting loop (Figure 2.8). Following a successful lift, relevant information is extracted from this structure and provided in the form of a `lift_result_t`, containing the metadata required for later verification and transformation stages.

Exploration. The exploration loop replicates SATURN's design, as described in subsection 2.3.3. Lifting a function begins by seeding the worklist with the entry RVA. The exploration loop then processes blocks until two conditions are satisfied: the primary *worklist* is empty, and a secondary *reprove* queue is also empty. When the worklist drains but reprove is non-empty, all reprove entries are re-enqueued and the loop continues until the fixed-point is reached.

When a block is encountered for the first time, it is lifted by decoding machine instructions starting from its RVA via Remill. The termination policy is conservative, and differs from that described by SATURN: *any* control-flow instruction (e.g. *call*, *ret*, *jmp*, etc), this marks the end of the block. The resolution (and classification) of those successors is deferred to the target-resolution step. This is because, as described in section 2.1, we cannot assume any control-flow behaviour from the instructions alone: it is per-obfuscator behaviour, modelled by *LifterCallbacks*. For example, a `call` instruction might actually encode a direct branch, while a `jmp` might actually be an indirect call. Obfuscations can sever the relationship between instructions and control-flow intent.

The lifted block is assigned an ID consisting of its RVA, and its information is stored. The block's Remill-lifted semantics are materialised as a standalone LLVM IR function—named `BB_<BB_ID>`—and stored in the persistent *Lifter* cache. This function's signature is that of a standard Remill-lifted function—as previously described in section 2.2—taking `state`, `pc`, and `memory` as parameters.

```
define ptr @BB_10f9(ptr noalias nocapture %state, i64 %program_counter, ptr noalias
    ↪ readnone returned %memory) local_unnamed_addr {
  %RSP = getelementptr inbounds %struct.State.303, ptr %state, i64 0, i32 0, i32 6,
      ↪ i32 13, i32 0, i32 0
  %RBP = getelementptr inbounds %struct.State.303, ptr %state, i64 0, i32 0, i32 6,
      ↪ i32 15, i32 0, i32 0
  %PC = getelementptr inbounds %struct.State.303, ptr %state, i64 0, i32 0, i32 6,
      ↪ i32 33, i32 0, i32 0
  %1 = load i64, ptr %RSP, align 8
  %sub.i.i.i = add i64 %1, -8
  store i64 %sub.i.i.i, ptr %RSP, align 8
  store i64 %sub.i.i.i, ptr %RBP, align 8
  %2 = add i64 %1, -12
  %call.i.i = tail call i32 @__remill_read_memory_32(ptr poison, i64 noundef %2)
  %cf.i.i = getelementptr inbounds %struct.X86State.302, ptr %state, i64 0, i32 2,
      ↪ i32 1
  store i8 0, ptr %cf.i.i, align 1
  %conv.i.i.i.i = trunc i32 %call.i.i to i8
  %3 = tail call i8 @llvm.ctpop.i8(i8 %conv.i.i.i.i)
  %4 = and i8 %3, 1
  %conv.i.i.i = xor i8 %4, 1
  %pf.i.i.i = getelementptr inbounds %struct.X86State.302, ptr %state, i64 0, i32 2,
      ↪ i32 3
  store i8 %conv.i.i.i, ptr %pf.i.i.i, align 1
  %af.i.i.i = getelementptr inbounds %struct.X86State.302, ptr %state, i64 0, i32 2,
      ↪ i32 5
  store i8 0, ptr %af.i.i.i, align 1
  %cmp.i.i.i.i = icmp eq i32 %call.i.i, 0
  %conv5.i.i.i = zext i1 %cmp.i.i.i.i to i8
  %zf.i.i.i = getelementptr inbounds %struct.X86State.302, ptr %state, i64 0, i32 2,
      ↪ i32 7
  store i8 %conv5.i.i.i, ptr %zf.i.i.i, align 1
  %call.i.i.lobit = lshr i32 %call.i.i, 31
  %conv8.i.i.i = trunc i32 %call.i.i.lobit to i8
  %sf.i.i.i = getelementptr inbounds %struct.X86State.302, ptr %state, i64 0, i32 2,
      ↪ i32 9
  store i8 %conv8.i.i.i, ptr %sf.i.i.i, align 1
  %of.i.i.i = getelementptr inbounds %struct.X86State.302, ptr %state, i64 0, i32 2,
      ↪ i32 13
  store i8 0, ptr %of.i.i.i, align 1
  %5 = icmp slt i32 %call.i.i, 1
  %cond1.i.i.v = select i1 %5, i64 23, i64 17
  %cond1.i.i = add i64 %cond1.i.i.v, %program_counter
  store i64 %cond1.i.i, ptr %PC, align 8
  ret ptr %memory
}
```

Figure 3.2: Example of a single lifted basic-block (optimised with O2). The instructions lifted are `cmp DWORD PTR [rbp-0x4],0x0; jle 0x1110`. The lifted IR models all semantics of the block, including the memory read (via a call to `__remill_read_memory_32`), the flag updates, and the control-flow update (via a write to the instruction pointer).

CFG Stitching. The framework provides a facility for *stitching* these block functions into a single function-level CFG (Figure 3.3). There are two modes of stitching available: *forwards stitching*, which includes all blocks reachable from an initial block, and *backwards stitching*,

which includes all predecessor blocks which have a path to the specified block. Both share the same core algorithm, but differ in how the set of blocks to include is determined.

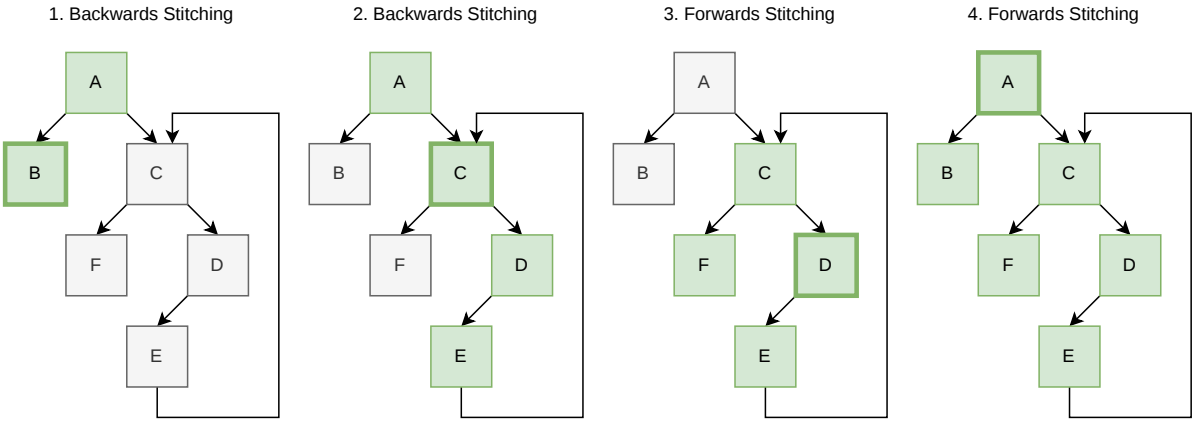


Figure 3.3: Backwards and forwards stitching algorithms. Thick border specifies which block is the target of the stitching, green blocks specify which are included in the stitched CFG. Backward stitching (1, 2) includes all blocks which control may pass through from the origin block (A) to the target block. Forwards stitching (3, 4) includes all blocks to which the target block may pass control to.

Stitching relies on the collected edges and blocks to dispatch successors (Figure 3.4). A new stitched LLVM IR function is first created, with standard Remill `state`, `pc` and `memory` parameters. Labels are then emitted in the function's IR for each lifted block. Each label issues a call to its respective basic-block function, which models the BB's semantics. After this call, the *instruction pointer* is extracted from the `State`, which now models the program's register state after the BB's execution. A switch statement is generated on this value, which enumerates each successor, and follows that successor's label. This allows for the function's control-flow to be modelled while knowing *only* the successors of each block.

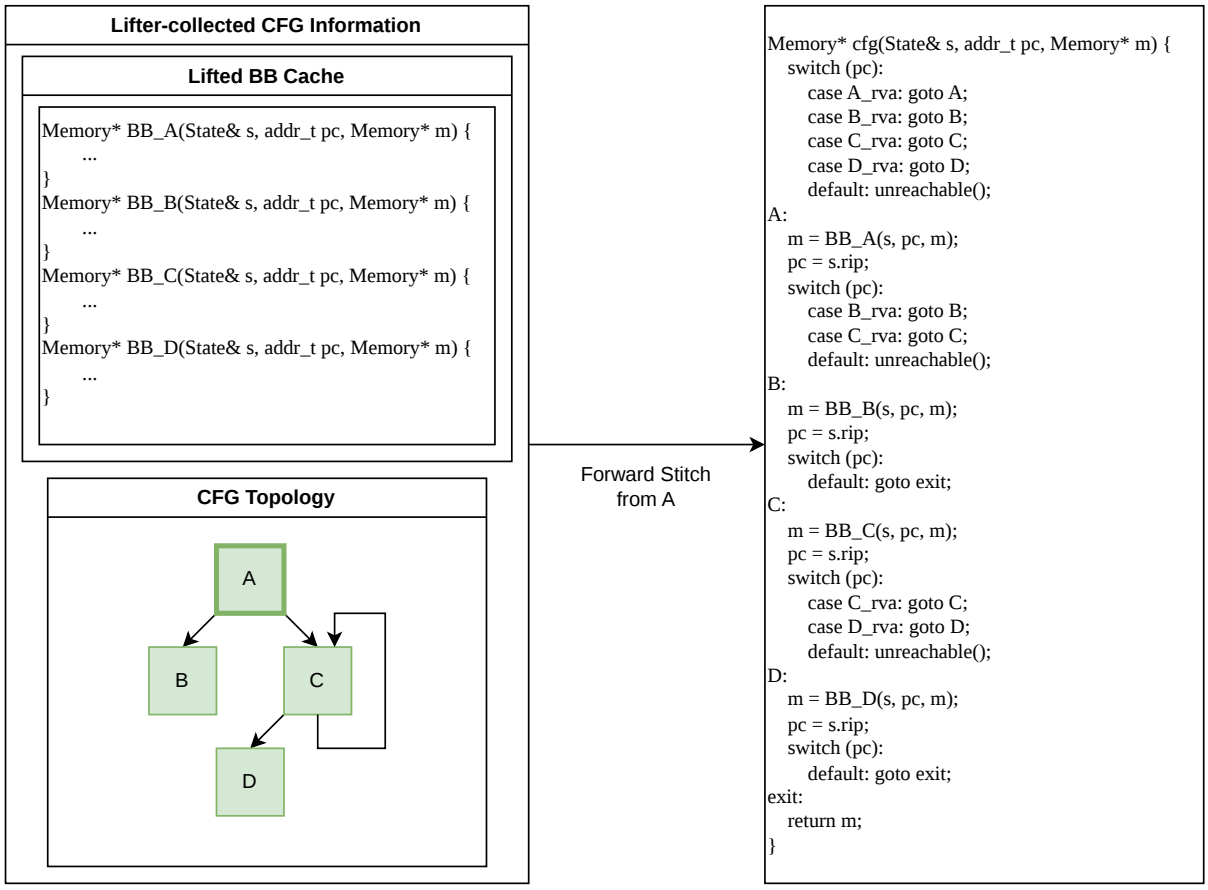


Figure 3.4: Example of a stitched CFG. The `stitch_cfg` function uses the lifter-collected CFG topology and the cached BB semantics functions to create a new function. This function enters an initial basic block based on the entry PC. Each basic block executes its semantics, then extracts the resulting instruction pointer. The value of this instruction pointer is what determines which successor is entered—modelled with a `switch` statement. This continues until a terminator block is reached, in which case the function returns. IR presented in C-like pseudocode for readability; the actual IR is in LLVM bitcode.

Target resolution. After obtaining a block, the lifter attempts to determine its successors. Two resolution paths are tried in order. The first is a cheap *advisory* pass using static metadata already known about the block. For direct jumps and direct calls, for example, the target is immediately available: control *must* transfer to the target. If the advisory pass cannot resolve the successors (for example, due to indirect control-flow as shown in Figure 2.4), the SATURN PC slicing process is used (subsection 2.3.3). The lifter constructs a backward-CFG from the current block, slices it for the final PC and provides the prepared slice to *LifterCallbacks*, which resolves the block's successors, passing them back to the *Lifter* and continuing exploration.

LifterCallbacks classifies resolved successors into one of four categories, each of which the lifter handles differently. This is a key innovation of LIFTED. Only *Internal* successors—blocks within the same function—become direct CFG edges, and the target is enqueued for

exploration if unseen, or marked for reproval if the edge is new. *Return-to-caller* blocks are recorded separately and translated into natural LLVM returns during CFG stitching. The remaining types—*External*, *Recursive* and *Indirect* successors—are handled in a special manner, and are described in detail in subsection 3.2.3.

Post-processing. Once all blocks in the worklist and reprove queues have been processed, the lifter produces a forwards stitch of the function, beginning at the initial BB. This produces an LLVM IR function which, if correct, perfectly models the function's semantics. At this point, we also apply a lightweight post-processing pass to clean up the lifted IR and prepare it for downstream transformations. This pass includes rewriting all memory access intrinsics into operations on an opaque `RAM` global array (Figure 3.5). This is essential for transformations which perform memory reordering and alias analysis, as it allows LLVM passes to reason about memory accesses as operations on a shared memory state, rather than opaque calls to unknown functions.

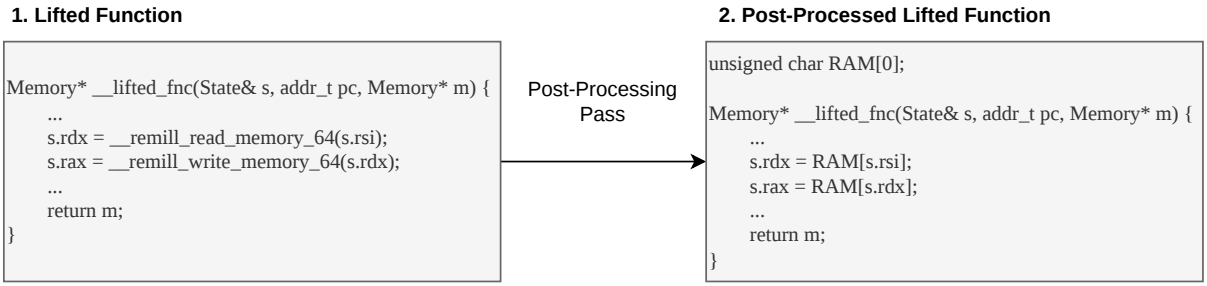


Figure 3.5: RAM lowering. The final lifted function (1) is subject to some post-processing cleanup passes, including lowering of all memory accesses from intrinsic form to global RAM access form (2).

The sections that follow describe the components that extend or specialise this core loop.

3.2.2 Annotations

Every function call encountered during lifting raises a question the lifter cannot answer from the binary alone: what does this callee do to the machine state? For internal edges within the lifted function, the answer emerges naturally from lifting the instructions: the semantics are modelled explicitly. For inter-procedural calls, however, the lifter has no callee body to examine. Yet to produce correct, optimizable lifted IR it must know, at minimum, which registers the callee may clobber (its calling convention), and where it stores its return address. Without this knowledge, LLVM's optimiser must assume the worst at every such site. In practice this pessimism defeats the very optimisation passes that SATURN relies upon to resolve opaque predicates and recover branch targets—it is not a tolerable default. As such, it is one of the innovations introduced by LIFTED.

The annotation model. LIFTED addresses this through a per-binary *annotation registry* owned by `BinaryContext`. An annotation is a record that captures the application binary interface (ABI) contract of one function: its calling convention, its memory behaviour, and details regarding its inputs and outputs. Annotations carry a stable numeric identifier and a human-readable name; these are the handles used throughout the rest of the system to refer to a particular callee contract.

The authoring surface is intentionally sparse. A user who only knows that a callee follows the System V AMD64 calling convention and reads global memory need only supply those two fields; all others default to their *pessimistic* values. This means the system is usable with partial knowledge: an incomplete annotation degrades gracefully to the same conservative assumptions that would be made with no annotation, rather than silently producing incorrect IR.

Resolution strategy. At each call site, `BinaryContext` resolves an annotation by consulting four sources in decreasing specificity. First, a *site-specific binding* may attach an annotation to one particular call-site address—useful when the same function is called in meaningfully different ways at different points in the binary. Second, a *destination binding* attaches an annotation to the callee's address, covering all call sites that reach it. Third, if a symbol name can be resolved for the callee (from the binary's import table or symbol metadata), a *canonical name lookup* is performed against a set of named annotations that encode standard library contracts. Finally, if all else fails, a *pessimistic auto-created fallback* is generated automatically. The system therefore requires no user input to function: annotations are opt-in refinements that improve analysis quality.

User-supplied annotations. The primary interface for supplying annotations is a JSON file loaded into `BinaryContext` at startup (Figure 3.6). The format distinguishes *canonical definitions*, which define reusable named contracts (e.g. `malloc`, `memcpy`), from *bindings*, which attach a canonical annotation to a specific destination or call-site address. This separation allows a single canonical definition to be shared across many binaries while still permitting binary-specific refinements without mutating the shared definition.

```
{
  "annotations": [
    {
      "name": "malloc",
      "cc": "systemv_x64",
      "reads_global_memory": true,
      "writes_global_memory": true,
      "arg_count": 1,
      "ret_size_bytes": 8,
      "ret_ptr": {
        "is_fresh_alloc": true
      }
    },
    {
      "name": "exit",
      "cc": "systemv_x64",
      "arg_count": 1,
      "no_return": true
    }
  ],
  "bindings": [
    {
      "annotation": "malloc",
      "rva": "0x1050"
    },
    {
      "annotation": "malloc",
      "site_rva": "0x4012ab"
    },
    {
      "annotation": "exit",
      "rva": "0x1070"
    }
  ]
}
```

Figure 3.6: An example of an annotation JSON file. Two functions are defined: `malloc` and `exit`, with their respective contracts. Three bindings are then defined: two for `malloc`—one for all calls to the destination at RVA `0x1050`, and one specific to the call site at RVA `0x4012ab`—and one for `exit`.

Downstream uses. Annotations remain attached to the lifted IR as custom attributes. They are propagated and maintained throughout the infrastructure, allowing them to be consulted by later transformation passes and optimisations. They also are a core component in the handling of intra-procedural calls, discussed in subsection 3.2.3. All consumers rely on the same `BinaryContext` instance, so there is a single source of truth for callee contracts throughout the entire pipeline.

3.2.3 Handling External, Indirect, and Recursive Calls

Non-trivial real-world binaries routinely contain inter-procedural calls, and any function-level lifter must be able to account for them without abandoning analysis at their boundaries. SATURN's documented algorithm does not describe how such cases are handled. Since LIFTED operates at the granularity of a single function, an explicit model of call-boundary semantics is required: continuing to lift without such a model would amount to recursively inlining every reachable callee.

A taxonomy of control-flow targets. LIFTED partitions all control-flow successors of a lifted basic block into five mutually exclusive categories. We call the potential successors of a block its *targets*, and the process of determining and categorising these targets is called *target resolution* and *classification*.

- **Internal** targets are those that lie within the same function, i.e. they are reachable from the entry block without passing through any inter-procedural calls.
- **Return-to-caller** targets are those that transfer control back to the caller of the lifted function.
- **External** targets are calls to functions that reside in the same binary but are not part of the function under analysis; the destination address is statically known.
- **Indirect** targets are those whose destination cannot be resolved until runtime—for example, targets derived from function arguments, computed data, or the import address table.
- **Recursive** targets are a specialisation of external targets in which the destination is the function currently being lifted.

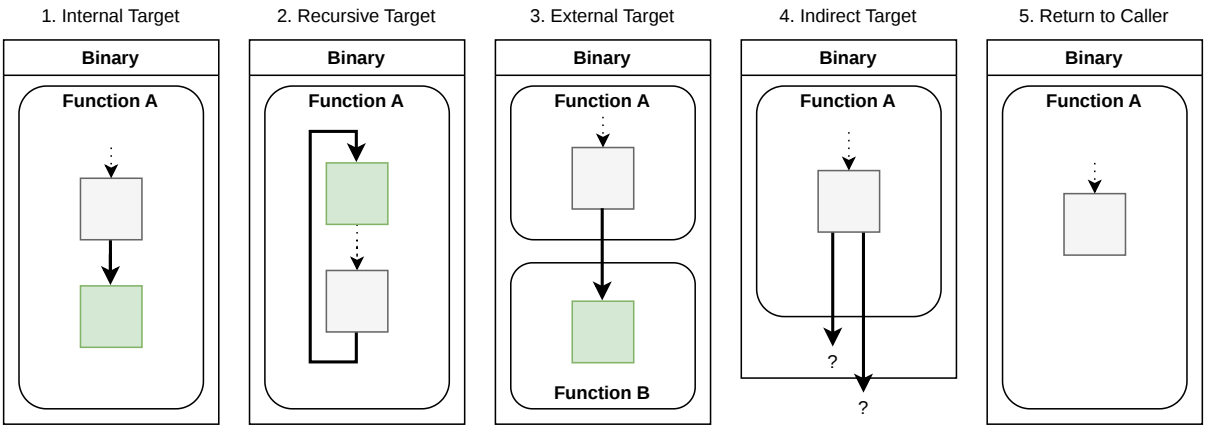


Figure 3.7: Types of basic-block targets. Squares denote basic-blocks. Green squares denote the statically-resolved target. For indirect calls, there are no known targets by definition—the destination can be any address inside or outside the binary. Return-to-caller targets denote that the block is a terminator block—hence also does not provide a statically-resolved target.

As established in section 2.1, obfuscating transformations can make the type of a control-flow transfer ambiguous. Instruction semantics alone are insufficient: a `ret` instruction, for example, may redirect control flow to a target of any of the above categories. Furthermore, a single block may terminate with a branch that resolves to multiple targets of *different* types simultaneously. The only structural exception is indirection: a block's destination is either statically known or it is not, and a block classified as having an indirect target cannot, by definition, carry any additional resolved targets under this model.

Because target resolution and classification are inherently obfuscator-specific, this responsibility is delegated to *LifterCallbacks*. For each resolved target, *LifterCallbacks* assigns one of the four statically-resolveable classifications: internal, recursive, external, or return-to-caller. Otherwise, it can instead specify that the block's successor is *indirect*, specifying that it is only known at runtime. This delegation keeps the *Lifter* core obfuscator-agnostic—it operates uniformly over whatever classification the callbacks supply.

The role of annotations. Classification alone is insufficient. Any inter-procedural call must satisfy a well-defined ABI contract: which registers the callee treats as arguments, which it may clobber, whether it modifies global memory, and whether it returns. At the binary level, this information is not syntactically present—it was discarded during compilation. This is the primary motivation for the annotation system described in subsection 3.2.2. For each external or indirect call site, *LifterCallbacks* is expected to supply an annotation alongside the target classification; this annotation informs both how the call is modelled during lifting and what optimisation assumptions LLVM's passes may safely apply at that site.

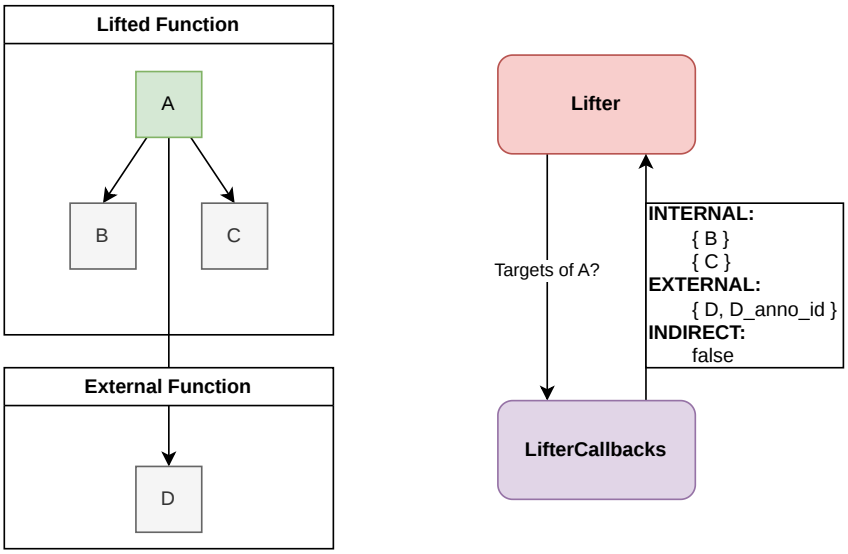


Figure 3.8: Example of `LifterCallbacks` returning multiple targets. The green square A denotes the block for which the `Lifter` (by delegation to `LifterCallbacks`) is resolving targets—it has two internal targets and one external target. The external target has an `annotation_id` associated with it.

Call stubbing. With targets resolved, classified and annotated, a representational question remains: how should these inter-procedural transitions be materialised in the lifted IR? Continuing the standard lifting algorithm is not viable for any of the three inter-procedural categories. For external calls, doing so would inline the callee, conflating two distinct functions. For recursive calls, it would introduce a conventional CFG edge back to the entry block, which misrepresents the call semantics. For indirect calls, no destination address is available from which to resume lifting.

LIFTED resolves this by *stubbing* non-internal calls (Figure 3.9). A *call stub* is an opaque function that models the semantics of an inter-procedural call without modelling the callee's body. Two opaque stub types are introduced—`__lifted_ext_call` (for external and indirect targets), and `__lifted_recursive_call` (for recursive targets)—each generated as an LLVM IR function declaration with the standard Remill signature: `state`, `pc`, and `memory`. The `pc` parameter carries the callee's address; for indirect stubs this is a non-constant expression, reflecting the runtime-dependent nature of the target.

A single lifted function can reach multiple distinct external callees and multiple independent indirect call sites, each subject to different annotations and ABI contracts. Stubs are therefore generated per-instance rather than per-type. External stubs are keyed by callee address, so each call destination receives its own declaration. Indirect stubs are keyed by the calling basic block, so each call site produces an independent stub. Recursive calls have a single well-defined destination and therefore require only one stub. Each stub declaration carries its associated annotation as a custom LLVM IR attribute and is recorded in the per-lift `lift_ctx_t`, pre-

serving the information needed for a future *brightening* pass to replace stubs with concrete IR call instructions.

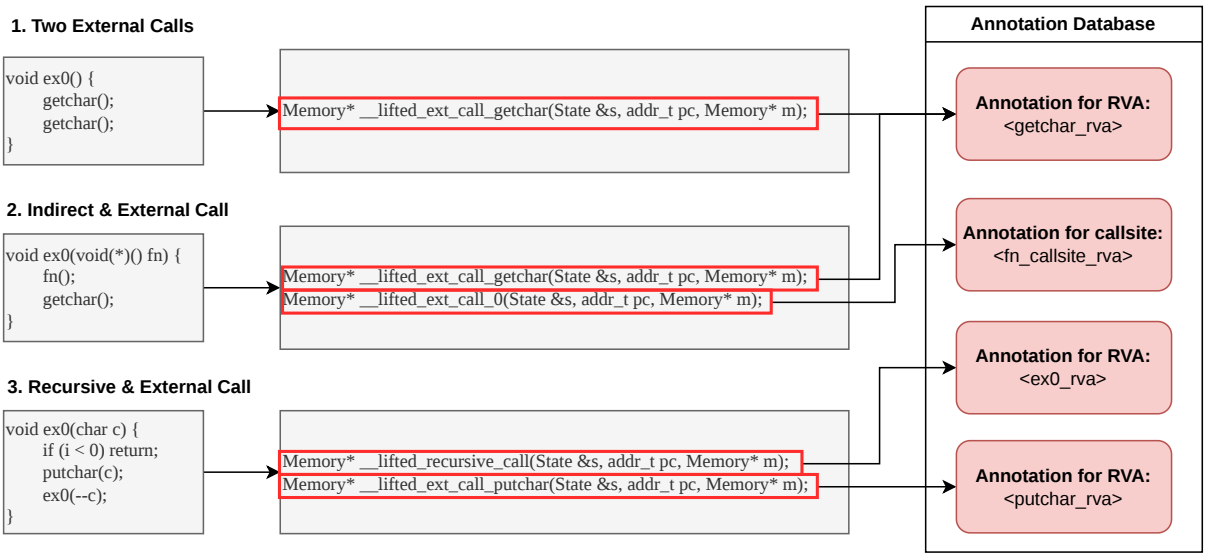


Figure 3.9: Example mappings of inter-procedural calls to stubs and annotations. The `Lifter`, upon encountering inter-procedural calls, creates stubs (red border). The two *external* calls share the same stub and definition, keyed by the call destination address (1); the indirect call and external call get unique stubs and annotations, keyed by the calling basic block address and destination address respectively (2); recursive calls are mapped to a single special stub, and inherit the lifted function's annotation (3).

Synthetic blocks. Introducing stubs exposes a structural issue with the CFG stitching mechanism described in section 3.2. Stitching dispatches successors via a switch on the final instruction pointer value produced by each block (Figure 3.4). Inter-procedural stubs, however, do not emit a basic block—they are opaque declarations—so the switch has no target label to dispatch to. Indirect stubs compound this problem: the destination address is unknown at compile time and cannot appear as a switch case at all.

To resolve both problems, LIFTED introduces the concept of *synthetic blocks* (Figure 3.10). A synthetic block is a virtual block that has no counterpart in the binary; its sole role is to act as a dispatch target within the stitched function that issues the appropriate stub call before transferring control back to the stitching mechanism. When an external, indirect, or recursive call site is encountered, a per-callsite synthetic block is generated, and assigned a `BB_ID` drawn from a reserved range guaranteed not to collide with any real block address.

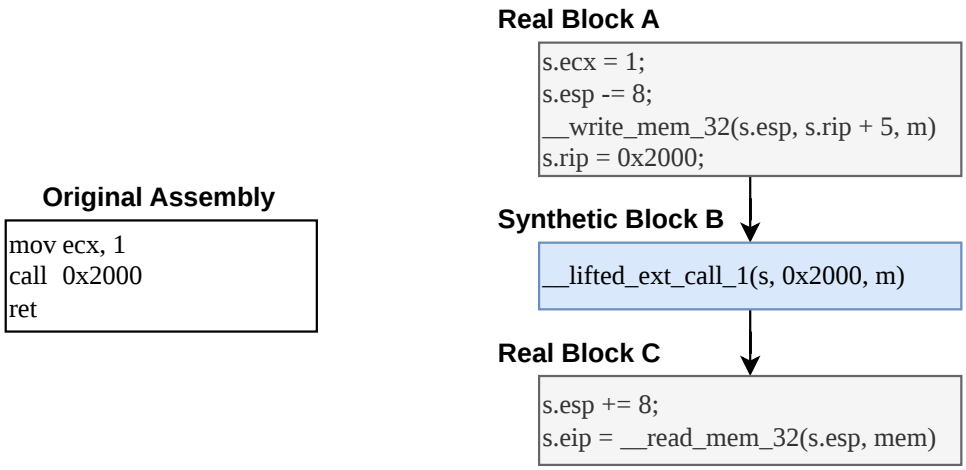


Figure 3.10: Example of a lifted synthetic block in LIFTED. Block A models the semantics of the `mov` and `call` instruction, and has an edge to the synthetic block B (blue). This synthetic block contains the call stub, which calls the generated stub for address `0x2000`. This block then has an edge to block C, which models the semantics of the `ret` instruction.

External and recursive call sites now have a concrete label to which the switch statement can dispatch: the synthetic block for that call site. Indirect call sites, whose target is unknown at lift time, are accommodated via the *default* case of the switch statement—a natural fit, since any block classified as indirect is, by the taxonomy above, free of other statically resolved successors. Generating synthetic blocks per-callsite rather than per-callee yields an additional structural benefit: two call sites targeting the same external function are represented as two distinct stub invocations rather than as a branch to a shared call site, preserving the correct call graph topology in the stitched IR (Figure 3.11).

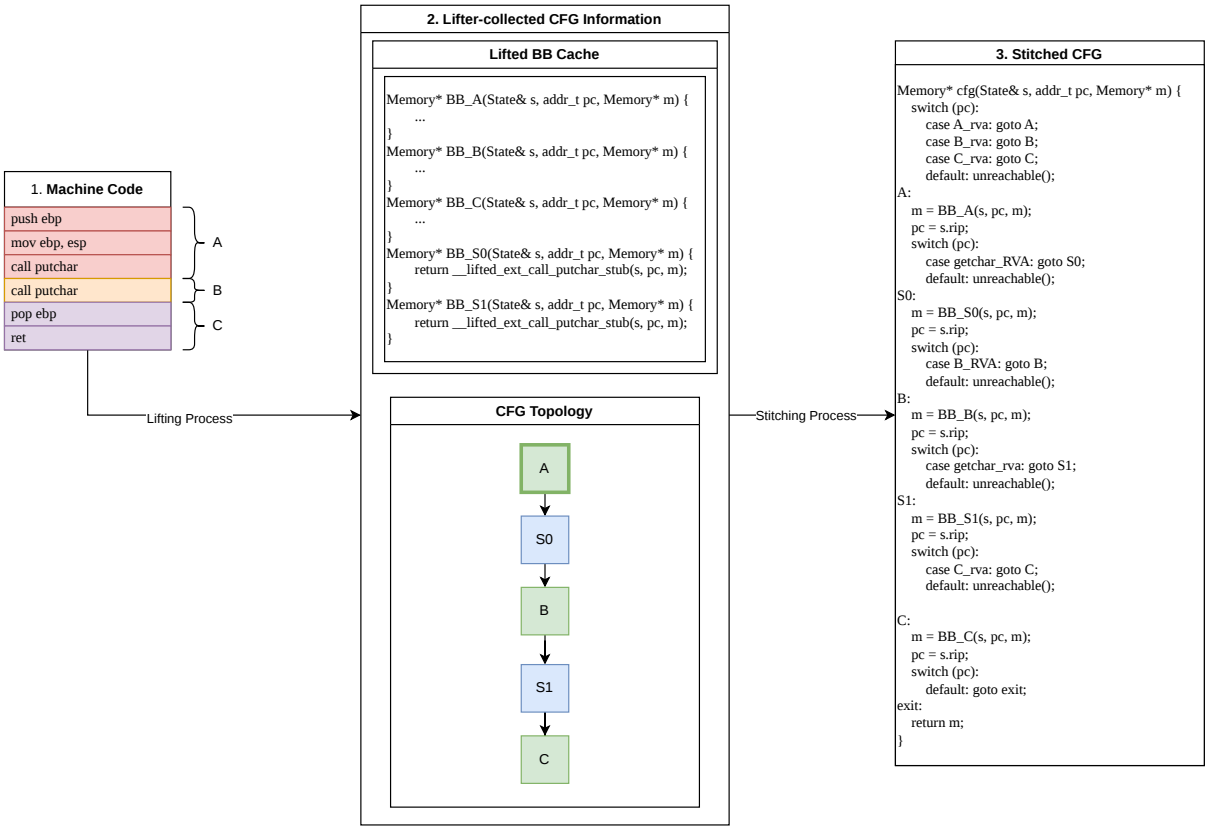


Figure 3.11: Stitched CFG with synthetic block dispatching, extending Figure 3.4. The different coloured blocks in (1) correspond to the CFG topology in (2). The synthetic blocks S0 and S1 (blue) issue calls to the `putchar` stub, which model the ABI contract of the function. The stitched CFG dispatches to these synthetic blocks based on the value of the instruction pointer after execution blocks A and B (the predecessors); if the resulting instruction pointer is equal to the RVA of `putchar`, the switch dispatches to the synthetic block.

Successors of Synthetic Blocks. Synthetic blocks are processed in the same manner as real ones in the lifting and target resolution algorithm; they are also subject to target resolution and PC slicing. This introduces another tension in the design: the synthetic blocks do not model the ABI's convention of returning control to the caller. This is per-ABI behaviour, which we must model to continue lifting—for example, the System V x64 ABI, used on Linux, specifies that the function's return address is pushed on the stack [54]. We must resume lifting at this address: it is the natural successor of the synthetic block.

This is conveniently already partially handled by our annotation system; recall that the inter-procedural call has an associated annotation. This annotation describes the ABI behaviour of the callee and, crucially, includes its calling convention and the location of the return continuation address. The Lifter uses this information to wrap the call-stub in yet another, calling-convention-aware, LLVM IR function (Figure 3.12). This function sets the program counter to its ABI-defined location, allowing the Lifter to continue exploration as normal. We also use this opportunity to save and restore registers that are non-volatile across the call,

specifying to LLVM which parts of the State remain unchanged.

```
Memory* __lifted_ext_call_putchar_stub(State& s, addr_t pc,
    ↪ Memory* m) {
    // Save all non-volatile registers (SysV ABI)
    uint64_t rbx = s.gpr.rbx;
    uint64_t rbp = s.gpr.rbp;
    uint64_t rsp = s.gpr.rsp;  // points to the return address
        ↪ pushed by `call`
    uint64_t r12 = s.gpr.r12;
    uint64_t r13 = s.gpr.r13;
    uint64_t r14 = s.gpr.r14;
    uint64_t r15 = s.gpr.r15;

    m = __lifted_ext_call_putchar(s, pc, m);

    // Restore all non-volatile registers
    s.gpr.rbx = rbx;
    s.gpr.rbp = rbp;
    s.gpr.r12 = r12;
    s.gpr.r13 = r13;
    s.gpr.r14 = r14;
    s.gpr.r15 = r15;

    // Simulate `ret`: read return address from [rsp], pop it,
        ↪ set rip
    uint64_t ret_addr = __remill_read_memory_64(m, rsp);
    s.gpr.rsp = rsp + 8;
    s.gpr.rip = ret_addr;

    return m;
}
```

Figure 3.12: Example of a call stub generated by the Lifter for the System V x64 calling-convention. The stub first saves all non-volatile registers, then issues a call to the external stub (which models the callee's semantics), then restores the non-volatile registers, and finally simulates the return by reading the return address from the stack and setting the instruction pointer to this value. This allows the lifter to resolve the targets of the synthetic block naturally.

This two-level indirection—annotation-driven stubs embedded within per-callsite synthetic blocks—allows LIFTED to model external, indirect, and recursive calls faithfully within the

SATURN-derived lifting infrastructure, without requiring changes to the core lifting loop or sacrificing the obfuscator-agnostic contract of the *Lifter*. The entire complexity of inter-procedural boundary handling is absorbed by *LifterCallbacks*, which is only required to classify targets and supply annotations; the structural consequences are handled uniformly by the core.

3.2.4 Block-Splitting

Another improvement over SATURN is the introduction of a block-splitting mechanism. This scenario must be considered when a target is discovered that is within the bounds of an already-lifted block. This may occur whenever there are fallthrough edges present in a function's CFG, a common pattern in functions containing loops.

During the exploration phase of our lifting algorithm, all blocks are terminated with a control-flow instruction. Depending on the order of exploration, which is arbitrary, we may end up in a scenario where a future lifted block has an edge *within* a previously lifted block (Figure 3.13). Because we model basic blocks as individual LLVM IR functions—where each function carries that block's semantics—our representation of basic-blocks is essentially *atomic*, meaning we cannot partially execute a block. This presents a problem for the lifting architecture.

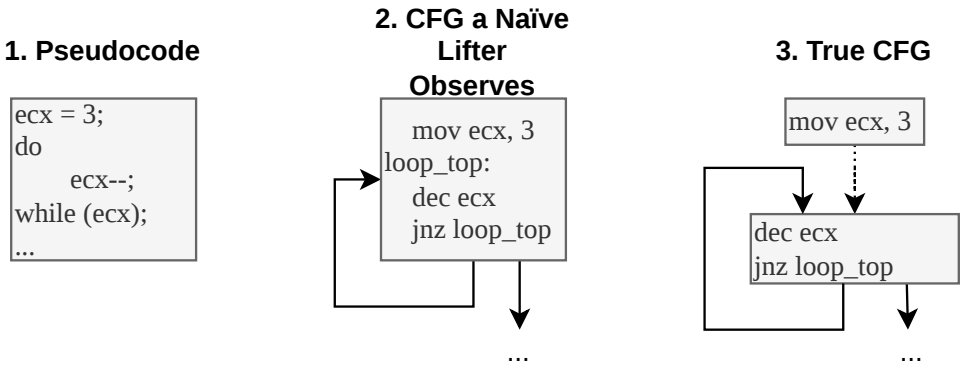


Figure 3.13: Block-splitting motivating example. Loops (1) often compile into machine code with *fallthrough edges*. Since our lifter explores the target CFG in an arbitrary order and continues lifting until it reaches a CFG instruction, it can lift both sides of the fallthrough edge as *single* block (2). The true CFG (3) contains this fallthrough edge, but it cannot be discovered by our lifter until it resolves the targets of the block terminated by the `jnz` instruction.

We solve this with a *block splitting* mechanism. When we encounter a target that is within the bounds of an already-lifted block in the lifting loop, we split this block at that address, relifting two individual blocks and linking them together. Consider an already-lifted block that spans RVAs $[A, B)$, and edge with destination D that was just discovered, where $A \leq D < B$. The algorithm is outlined below.

1. We remove the old block (spanning $[A, B)$) from the block cache.
2. We lift two new blocks: $[A, D)$ and $[D, B)$. This is done with the same method as the normal lifting algorithm, but specifying the maximum or final address of the block.
3. We rewrite the edges: for each edge that had destination $[A, B)$, we rewrite this destination to $[A, D)$. For each edge that had source $[A, B)$, we rewrite this source to $[D, B)$. We insert a new *fallthrough* edge from $[A, D)$ to $[D, B)$.
4. We schedule $[D, B)$ for reproval, as it gained a new predecessor.
5. We similarly rewrite our synthetic block information cache to account for this split.

This algorithm allows us to maintain the atomic representation of one LLVM IR function per basic block. It effectively handles scenarios such as the ones depicted above in Figure 3.13, where a target is discovered that is within an already lifted blocks. However, it does not consider the inverse problem: the fallthrough successor may already have been lifted before its fallthrough predecessor is encountered. In this scenario, our current algorithm, which terminates blocks exclusively on CFG instructions, would lift the fallthrough successor twice: once in isolation, and once as a continuation of its predecessor (Figure 3.14).

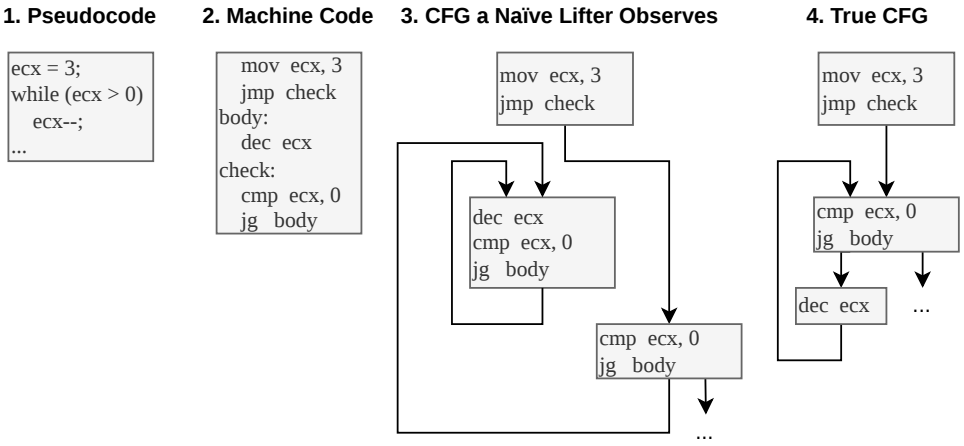


Figure 3.14: Block-splitting motivating example. Certain loops (1) compile into machine code where the target of the fallthrough edge is executed first (2). In this scenario, our naive lifter might explore this block first, leading to the *inverse* of the problem depicted in Figure 3.13 (3). When the lifter attempts to then lift the source of the fallthrough edge, it does not terminate at the already-lifted block, instead it continues until the CFG instruction.

We solve this problem by ensuring that, whenever we lift a block, if it would extend past a block that we have already lifted, we terminate our lifting prematurely and insert an edge between the two blocks. This ensures that we never lift a block that extends past an already lifted block—instead, we insert a natural fallthrough edge.

By combining these two methods, we give our lifting framework the flexibility to handle arbitrary CFG topologies, including those with fallthrough edges and loops, without sacrificing

the atomic block representation or the obfuscator-agnostic contract of the core lifting algorithm.

3.2.5 A Composable Architecture

The preceding sections describe our implementation of the core lifting algorithm and the two principal extensions that make it applicable to real-world binaries: the annotation-driven call-handling model and the block-splitting mechanism. Taken together, these components form a layered, developer-facing C++ library. This section demonstrates how they compose in practice, giving a concrete picture of the developer experience before the verification framework is introduced.

Extending the callbacks. The sole mandatory extension point is `LifterCallbacks`. A researcher targeting a specific obfuscator subclasses it and, at minimum, provides an implementation of `resolve_targets`—the hook responsible for classifying a block's successors from the prepared PC slice. All remaining hooks have working defaults and are overridden only where the obfuscator's behaviour warrants it:

```
struct MyObfuscatorCallbacks : public LifterCallbacks {
    // Mandatory: given the prepared backward-slice CFG,
       ↪ classify each successor.
    // Return INTERNAL, EXTERNAL, RECURSIVE, or
       ↪ RETURN_TO_CALLER; supply an
    // annotation_id alongside any inter-procedural target.
    resolved_targets_result_t resolve_targets(resolve_context_t
       ↪ &ctx) override;

    // Fast path: resolve trivially-known targets without
       ↪ building a slice.
    // Return std::nullopt to fall through to the full
       ↪ slice-based path above.
    std::optional<resolved_targets_result_t>
    hint_resolve_targets(resolve_context_t &ctx) override;

    // Called after each basic block is lifted into IR.
    // Override to run obfuscator-specific cleanup or
       ↪ observation passes.
    void process_bb(const lifted_bb_t &bb, llvm::Function *fn)
       ↪ override;

    // Called on the final stitched function before lift()
       ↪ returns.
    // Override to apply whole-function deobfuscation passes.
    void on_function_built(llvm::Function *fn, lift_context_t
       ↪ &ctx) override;

    // Notified when an already-lifted block is split at a
       ↪ mid-block target.
    void on_block_split(bb_id_t orig, bb_id_t first, bb_id_t
       ↪ second) override;

    // Return false to prune a block from exploration entirely.
    bool should_continue_exploration(
        bb_id_t id, const lift_context_t &ctx) const override;
};
```

Figure 3.15: Skeleton of an obfuscator-specific LifterCallbacks subclass. Only resolve_targets is mandatory; the remaining hooks are overridden where needed.

Setup and repeated lifting. With the callbacks defined, the remaining setup follows three steps: construct a `BinaryContext` from the parsed binary image; optionally load callee annotation contracts from a JSON file; and create the `Lifter`. Once constructed, the same `Lifter` instance can be used to lift any number of functions within that binary—basic-block bodies lifted on behalf of one function are cached and silently reused when a subsequent lift encounters the same block address:

```
// 1. Construct BinaryContext from a parsed binary image.
auto bin = LIEF::ELF::Parser::parse("target.elf");
BinaryContext bin_ctx(std::move(bin));

// 2. Load callee annotation contracts (optional).
//    Missing entries degrade gracefully to pessimistic
   ↪ defaults.
bin_ctx.import_annotations_from_json("annotations.json");

// 3. Create the Lifter once; Remill semantics are loaded here.
MyObfuscatorCallbacks cbs;
auto lifter = Lifter::create(arch, bin_ctx, &cbs);

// 4. Lift the first function -- all blocks are freshly lifted
   ↪ and cached.
auto result_a = lifter->lift(0x1200, entry_annotation_id);

// 5. Lift a second function -- blocks shared with the first
   ↪ are reused.
auto result_b = lifter->lift(0x1480, entry_annotation_id);

// 6. Each lift_result_t carries the stitched LLVM IR module,
   ↪ the public
//    entry function, and the reconstructed CFG topology.
result_a->module->print(llvm::outs(), nullptr);
```

Figure 3.16: End-to-end lifting of two functions from the same binary. The second call reuses cached basic-block bodies from the first, reflected in the `bbs_cached` field of the returned statistics.

The `lift_result_t` produced by each call is a self-contained artefact: its module holds the stitched IR function with all embedded annotation attributes, and its `basic_blocks`, `edges`, and `synthetic_blocks` fields capture the full CFG topology recovered during exploration. This is the shared artefact between the lifting and verification halves of LIFTED. It is consumed

directly by the verification framework described in the following section.

3.3 The Verification Framework

3.3.1 A Concrete Testing Approach

As established in section 2.4, approaches to equivalence checking occupy a spectrum between confidence and tractability. At the strongest end, fully verifying the transformation tool itself is not a practical goal here: even for conventional LLVM optimisation pipelines, proof-based validation at scale remains difficult, which is why tools such as Alive2 adopt bounded translation validation instead [52]. Lifted IR then adds further difficulties: it lacks the type, alignment, and pointer-provenance information that SMT-based validators rely on for tractable encodings [11, 44], while under-constrained calling context and path explosion further limit symbolic approaches [5, 59].

LIFTED therefore adopts the lightest point on that spectrum: a *concrete testing* approach. Two programs are executed with the same randomised initial states and their observable behaviour compared, requiring neither curated test inputs nor solver infrastructure [34].

In exchange for the scalability this affords, the framework sacrifices formal completeness: it can only produce *counter-examples* to equivalence, never proofs. This is a deliberate trade-off. The primary objective, as defined by IG3, is to give the deobfuscation developer fast, actionable feedback during development—identifying bugs such as that shown in Figure 1.3—rather than a sound verification guarantee.

Verification scope. In alignment with LIFTED's function-level lifting model, the verification framework operates on individual, self-contained functions. A verification target is identified by a user-supplied entry program counter; execution begins at that address and is considered complete upon a natural function return. This scope matches the granularity at which deobfuscation passes are applied, and it means that the function-level artefacts produced by the lifting framework can be verified directly.

Observable state. For this framework, the observable state of a function is modelled as a pair $S = (\mathcal{R}, \mathcal{M})$, where $\mathcal{R}$ is the register file and $\mathcal{M}$ is the memory state—the two components of programmer-visible architectural state on x86-64 [36]. A discrepancy between two functions is detected whenever they produce different output states S' from the same initial S; any such instance constitutes a definitive counterexample to equivalence.

Conversely, agreement on a given S provides no formal equivalence guarantee: the functions may simply happen to agree on this particular input while being discrepant. This is the fundamental limitation of sampling-based testing, and it is inherent to the approach. The

practical mitigation is to execute many randomised runs per verification session: each run that does not discover a discrepancy raises confidence, and a discrepancy found in any run is a concrete, reproducible failure the developer can inspect.

Underconstrained execution. A function extracted from a binary for verification is *underconstrained*: the valid input domain—which register values are meaningful, what memory regions are accessible, and what layout the stack is in—is not statically known [59]. Recovering this information in general would require analysing all call paths leading to the function in the original binary, which would require broader caller-context reconstruction across the binary and is outside the scope of this work.

As a consequence, the initial states S used in each run are not drawn from the function's true input domain. Instead, they are arbitrary, random combinations of register values and memory content. This means that individual runs may exercise execution paths that the function would never follow in its original binary context. This is not a defect unique to LIFTED; it is the standard operating condition of under-constrained analysis [59]. The function's operations on S remain well-defined and meaningful regardless of whether the initial state is realistic: a discrepancy is still a genuine discrepancy, and agreement is still a genuine agreement under the same arbitrary conditions applied equally to both sides of the comparison.

```c
// caller_a guarantees s is non-NULL
void caller_a(const char *input) {
    if (input != NULL)
        foo(input);
}

// caller_b guarantees s is null-terminated
void caller_b(char buf[256]) {
    buf[255] = '\0';
    foo(buf);
}

// mystrlen blindly walks memory until it finds '\0'.
// No checks -- it fully trusts its callers.
int mystrlen(const char *s) {
    int len = 0;
    while (s[len] != '\0')  // UB if s is not a valid readable
    ↪ pointer
        len++;
    return len;
}
```

Figure 3.17: Example of the under-constrained analysis problem. The function `mystrlen` is designed to be called by `caller_a` and `caller_b`, which guarantee that the input pointer is valid and null-terminated, respectively. However, when verifying `mystrlen` in isolation, we have no knowledge of these guarantees.

There are, however, certain constraints that can be applied to every run independently of binary knowledge in this context. First, the binary image is loaded at a fixed base address, and the instruction pointer is initialised to the user-supplied entry address; this is the entry-point of the function. Second, the stack pointer must satisfy the platform's alignment requirement. In our case, on x86-64, it must be 16-byte aligned prior to a call. These fixed parameters impose a minimal structural coherence on every run while leaving the remaining register and memory state fully randomised.

Together, the fixed entry point, stack alignment constraint, and fully randomised $(\mathcal{R}, \mathcal{M})$ define the launch conditions for every verification run.

3.3.2 A Randomised Memory Model

Providing a randomised initial register file $\mathcal{R}$ is straightforward: each general-purpose register is assigned a value sampled from a seeded generator, subject to the structural constraints

described in the preceding subsection. Constructing a coherent randomised memory state $\mathcal{M}$ is considerably more involved.

The difficulty is a direct consequence of the underconstrained execution model. Because the initial register values are arbitrary, any register may be dereferenced as a pointer at any point during execution. Moreover, a value read from an arbitrary memory address may itself be used as a pointer, chaining into further accesses. There is therefore no bounded region that can be safely left uninitialised: in principle, execution may reach any address in the process address space. Fully materialising this state in advance is infeasible—on a 64-bit platform it would require storage on the order of 2^{64} bytes.

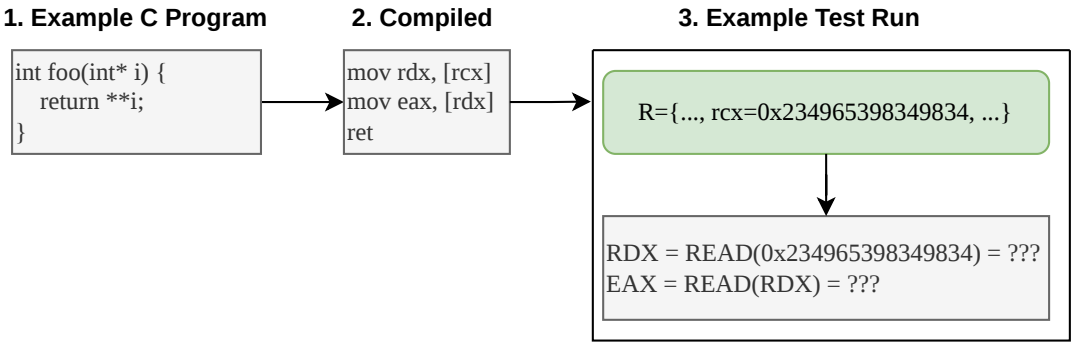


Figure 3.18: Example of the underconstrained memory problem. Since we are sampling random states and verifying arbitrary functions, we do not know what the function may use to access memory.

Lazy paged allocation. LIFTED resolves this through a *lazily-allocated, seeded* memory model. Rather than materialising the full address space upfront, memory is divided into fixed-size 4 KiB pages that are created on demand at the point of first access. This presents a semantically complete address space to the executing function—every address is accessible and no access produces a fault—while keeping memory consumption proportional to the pages actually touched during a run.

Each page is represented by a `page_t` structure carrying three components: a 4096-byte data array holding the page's current content, and two *bitset* fields—`read` and `written`—each tracking access at single-byte granularity. Every read sets the corresponding bit in `read`; every write sets the corresponding bit in `written` and stores the value. This is used for tracking which addresses have been written by the program.

Page content and seeding. Pages fall into one of two categories, determined at creation time:

- **Image pages** cover address ranges that map to loadable segments of the binary under analysis. On first access to such an address, the corresponding segment content is

copied into the page's data array, with dynamic relocations within that page applied in-place, mirroring the binary's state when actually executed in a real process.

- **Dynamic pages** cover all other addresses. Their content is generated by seeding a uniformly-distributed pseudo-random number generator (PRNG) with a page-specific seed derived from the page's base address and an initial master seed.

Because the page content is independent of the access order, the memory contents are essentially *deterministic*, based only on the initial seed provided.

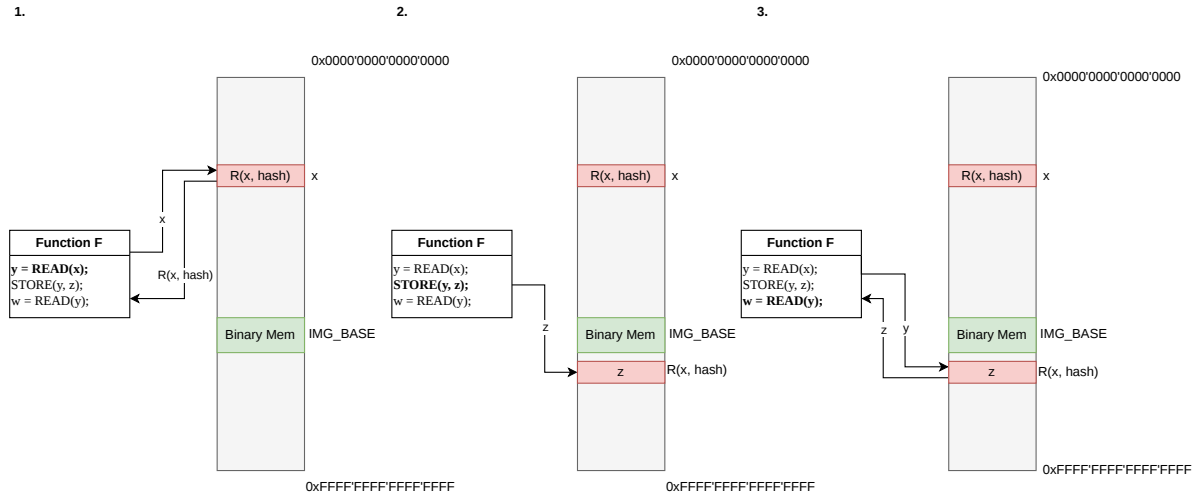


Figure 3.19: The LIFTED memory model. Green blocks denote image pages, red blocks denote dynamic pages. An uninitialised, dynamic read to `x` saves a deterministically-derived random value to be stored at that slot (1). That value can then be used to chain further reads and writes (2). Stored values are preserved, so when the program accesses a slot again, it retrieves the same value that was stored (2).

Write tracking and the rolling digest. Writes are handled uniformly across both page types. Each byte write records the new value in the data array, sets the corresponding `written` bit, and makes an incremental contribution to a pair of running hashes maintained by the model: a *written-set hash* and a *written-value hash*. Both hashes are maintained as commutative XOR accumulators. When a previously unwritten byte is first written, its address contribution is XORed into the written-set hash. On every write—including overwrites—the old value's contribution is removed and the new value's contribution is added to the written-value hash. The result is a pair of order-independent digests that can be compared in $O(1)$ as a fast equality check before a more detailed byte-level inspection is needed. Keeping the two hashes separate also allows the comparison to distinguish between the case where both executions wrote to the same set of addresses but with differing values, and the case where the write sets themselves differ, a distinction that is informative when diagnosing a discrepancy.

Comparison. The model materialises a memory *snapshot* after all memory accesses are collected. This snapshot consists of the content of each page touched during execution, the corresponding `written` bitsets, and the two hashes. A quick comparisons of the hashes can determine whether the write sets and written values are identical without needing to inspect the page content. A detailed comparison, which walks through the content and bitsets of each page, is only needed when a hash mismatch is detected. This results in a *memory diff result* that reports, per differing page, whether the mismatch was in the write set, the written values, or both, and collapses consecutive differing bytes into address ranges for human-readable reporting (Figure 3.20).

```
memory diff:
   equivalent = false
   1 page differs

   page 0x7fff0000 [BOTH], 2 differing ranges:
     0x7fff0018..0x7fff001c (4 B)
       snapshot A: 41 42 43 44
       snapshot B: 41 99 43 44

     0x7fff0030..0x7fff0032 (2 B)
       snapshot A: 10 20
       snapshot B: 00 00
```

Figure 3.20: Example of human-readable memory diff output. The diff reports that the two snapshots are not equivalent, and that one page differs. It then details the differing page, reporting that both the write set and the written values differ, and lists the two differing ranges within that page. For each range, it shows the content of that range in both snapshots.

In Practice. The aforementioned logic is wrapped in a self-contained `paged_memory_state` object—which provides these features in a high-level API—and is used by the downstream verification infrastructure. The following subsections describe how this memory model is shared between the two executors and how the behaviour recorded during execution is synchronised and compared.

3.3.3 The Executor Model

The preceding subsections establish what a verification run looks like in the abstract: two functions are driven with the same initial state $S = (\mathcal{R}, \mathcal{M})$ and their observable behaviour compared. As established in section 2.4, we cannot assume the correctness of the raw lifted IR, so the verification framework must be able to execute both LLVM IR and the binary code, and compare their behaviour in a common format. Realising this in practice requires confronting

a fundamental asymmetry. The lifted LLVM IR function and the original native binary cannot be executed in the same way; it is a compiler artefact that cannot be executed as-is. The binary is native machine code, opaque to the IR toolchain, requiring a mechanism that can intercept every memory access and control-flow transition without surrendering control to the executing code. For comparison to be possible, both must produce observations in a common form.

The *executor model* is the infrastructure that bridges this asymmetry: two execution mechanisms, each adapted to its artefact, both writing into a shared observation structure that the comparison layer acts on (Figure 3.21). Each executor receives an *artefact*—either the lifted IR module or the original binary image—and a reference to the shared observation structure, and is responsible for executing its artefact while recording the observed behaviour in that structure.

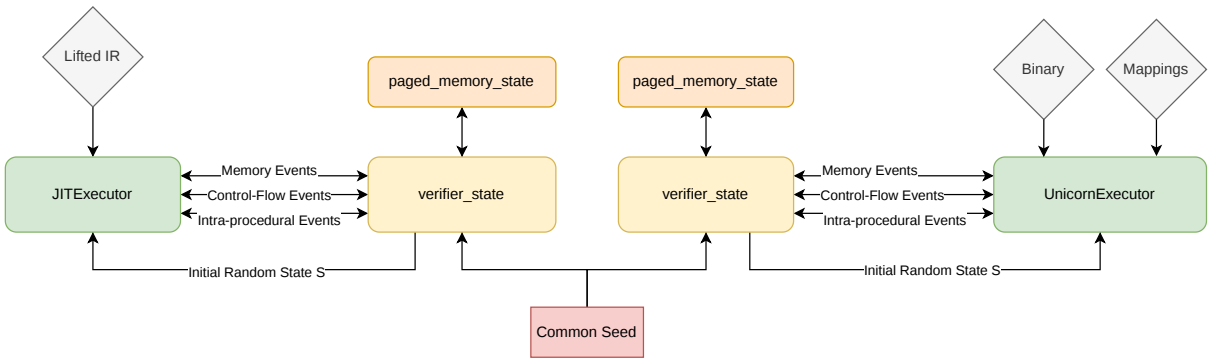


Figure 3.21: The LIFTED executor model. The `verifier_state` is initialised with a common seed and configuration. It orchestrates the verification runs of an executor, providing it with its initial state S. The executors abstract away the format of their input artefacts, feeding their verification logs into `verifier_state`, recording execution details and providing an interface into the `paged_memory_state`, allowing for a unified comparison layer that is agnostic to the source of the observations.

A Shared Observation Model

Before either executor can be described, the structure they both write into must be understood. Both executors generate *events*, observing three categories of behaviour during a run and report each through the same interface. These events are invoked into the `verifier_state`, which is the *observation interface*, responsible for accumulating the observed behaviour in a common format for comparison. Initialised with an initial master seed, it orchestrates verification runs, maintains the current state of execution, and records the events emitted by both executors in a common format.

The events emitted by each executor are categorised as follows:

- **Memory effects**: every load and store performed by the executing function, carrying the address, width, and value involved. These are passed by `verifier_state` into

the `paged_memory_state`, which maintains the memory model described in subsection 3.3.2.

- **Control-flow progress**: reported at each basic-block boundary as the function moves from one block to the next. Each event carries the current machine register state and the source and destination block addresses.
- **Inter-procedural boundaries**: reported when the executing function encounters an external or recursive call. These correspond directly to the call-boundary model introduced by the lifting framework (subsection 3.2.3).

Each event can also provide the executors with information; for example, whether the event should result in immediate termination of the current run, or the value to be returned by a memory read (by consulting the `paged_memory_state`). The later comparison layers operate entirely on what `verifier_state` has collected, independent of which executor produced it. A discrepancy is a discrepancy in the observation record, regardless of whether it originated from the raw lifted IR, a transformed version of that IR, or the machine code itself.

The JITExecutor

The lifting framework produces a stitched LLVM IR function: a complete IR artefact suitable for transformation by standard LLVM passes and analysis by the verification infrastructure (section 3.2). Executing this function is a matter of compiling and running real LLVM IR, a task for which LLVM provides mature infrastructure in the form of a JIT compiler.

JIT compilation is used for two primary reasons. First, it allows for the IR to be executed—and hence verified—programmatically, within the context of the process, sharing its address-space. This allows us to directly pass data into the function without requiring serialisation. Second, it is fast. JIT compilation results in native-speed executions of the lifted IR. This means that we can execute hundreds or thousands of runs in a relatively short period of time, a benefit demonstrated in chapter 4.

Preparation for verification. The lifted module cannot be submitted to the JIT compiler in its raw form. Three preparation steps are required before compilation can proceed (Figure 3.22).

The first step is memory intrinsic lowering. As established in section 2.2, after lifting, we model program memory through accesses to a global, opaque `RAM` array. This representation is essential during lifting and optimisation, as it allows LLVM to apply memory analysis and optimisation passes, but at execution time the `RAM` array has no runtime backing. Executing the IR directly would produce undefined behaviour on every load and store. A lowering pass walks the module and replaces every `RAM`-rooted memory access with a call to the corresponding typed memory helper—a declared but not yet defined intrinsic whose job is to

route the access through `verifier_state`. After the pass, no `RAM` uses remain; the setup verifies this as a hard precondition before proceeding.

The second step is linking the verification helper bitcode. At this point the module contains calls to a set of symbols that have no definition: the memory functions introduced by the lowering pass, the basic-block entry, external and recursive call intrinsics. A module with undefined symbols cannot be JIT-compiled, but more fundamentally, these symbols *are* the observation interface. Each one is a concrete bridge into `verifier_state`: they generate the events used for integrating the IR with the `verifier_state`. All of these are implemented in a companion translation unit compiled to LLVM bitcode (an example can be seen in Figure 3.23). Linking this bitcode into the module resolves every missing symbol and provides this observation interface in a single step.

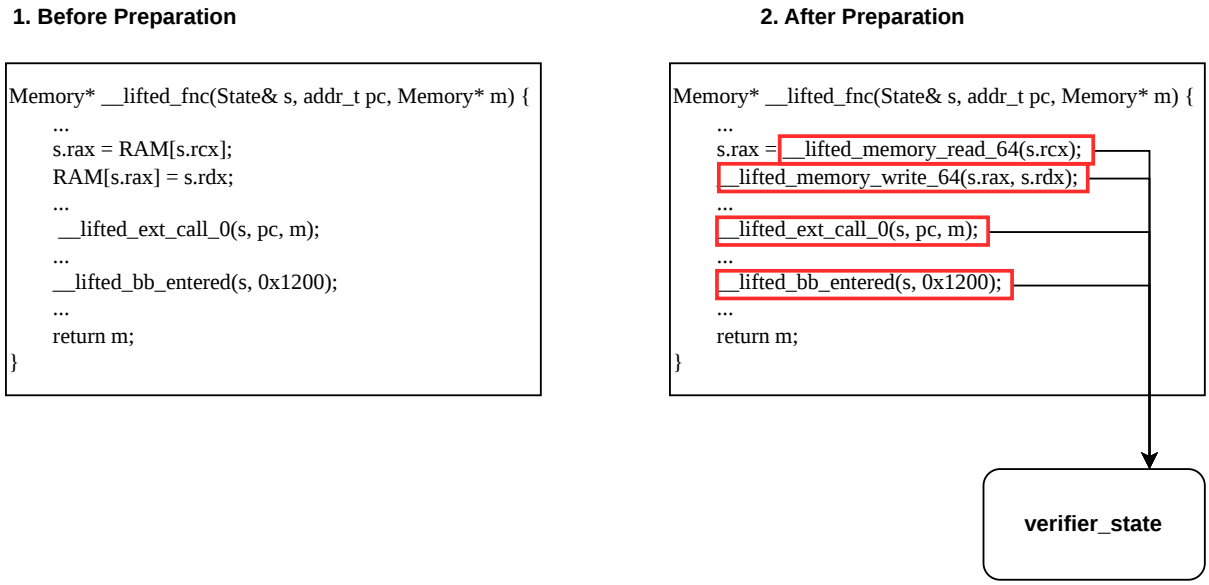


Figure 3.22: Preparation for JIT executor. Before preparation, the lifted module contains `RAM`-rooted memory accesses and calls to undefined intrinsics (1). After preparation, the memory accesses are raised to intrinsics, and the intrinsics now have definitions which call into `verifier_state` (2).

```
HELPER uint64_t __lifted_read_memory_64(addr_t a)
{
    uint64_t value;
    assert_exit(__ctx->verifier_context->read64(
        ↪ __ctx->verifier_context->context, a, value));
    return value;
}
```

Figure 3.23: The definition for 64-bit memory reads provided by the verification helper. After linking to the verification helper, all memory accesses (and external stubs, basic block entries, etc) are routed through similar functions. Internally, the call is routed through `verifier_state` to `paged_memory_state`, which records the read and provides the `value` at the memory address to the caller, which then continues execution. The `assert_exit` call redirects execution to the recovery point (described below) if necessary.

The bitcode module also defines the run entry point: a wrapper function that receives the run parameters (Figure 3.24). These include the random seed used to initialise the run, and invocation of the stitched function with the initial register state. Separating this entry point from the lifted IR means the module is prepared once and executed repeatedly with different inputs; there is no need to recompile for each run.

```
extern "C" void __lifter_verifier_begin(jit_executor_context_t
    ↪ *executor_ctx)
{
    // 1. Extract initial state as provided by the JITExecutor
    State state =
        ↪ executor_ctx->verifier_context->initial_reg_state;
    __ctx = executor_ctx;

    // 2. Register the recovery point; if we exit early, we
        ↪ jump to this point.
    int jump_val = setjmp(executor_ctx->exit_jmp);
    if (jump_val == 0)
    {
        // 3. Call the IR function that is being tested.
        __lifted_rebuilt_cfg_concrete(state, nullptr,
            ↪ LOAD_BASE);
    }

    // 4. Synchronize Remill's AFLAGS into x86-64 RFLAGS.
    // Remill models the x86-64 flags as a single AFLAGS
        ↪ register; we must extract the individual flags and
        ↪ update the corresponding bits in RFLAGS.
    synchronize_flags(&state);

    // 5. Report the final register state back.
    executor_ctx->verifier_context->final_reg_state = state;
}
```

Figure 3.24: The entry-point of the JIT verification intrinsics. The executor sets up the `jit_executor_context_t` with per-run parameters, and invokes this function. The function then executes, recording all run information into the `verifier_state`.

The third step is external call stub rewriting. The per-callee stub declarations introduced by the call-handling mechanism carry the callee's annotation identifier as an IR attribute. A rewriting pass extracts this identifier from each declaration and rewrites every call site to a single unified dispatch helper that forwards the annotation identifier as an explicit argument. The per-callee declarations are then erased. All external calls in the IR now funnel through one defined point, making the annotation visible to the observation layer at runtime.

Controlled execution. We want to be able to control when the execution is terminated and how it recovers from faults. The motivation behind early termination is described later

in subsection 3.3.4. We also need to recover from faults without crashing the process.

Both requirements are addressed through non-local termination. At the start of each run, a recovery point is established on the native stack via the `setjmp` mechanism. Every observation callback is instrumented so that the verifier can trigger execution early by performing a `longjmp` back to this point. Execution terminates cleanly, with the structured exit reason recorded at the point the stop was triggered, and the run finishes capturing the state at termination.

Hardware faults in JIT-compiled code, such as division-by-zero, cannot be caught by ordinary exception handling. Signal handlers are installed for the duration of each run to intercept such events and redirect them to the same recovery point, again producing a structured fault outcome.

The UnicornExecutor

Executing the native binary directly within the host process would surrender the control the verifier requires. Every memory access must be intercepted and answered from the shared memory model; every basic-block boundary must be observed at a granularity aligned with the lifted IR; every external call must be intercepted.

CPU emulation provides the necessary control. Unicorn is an emulator framework [21], derived from QEMU [10], that executes native machine code in an isolated environment and exposes a fine-grained hook interface for intercepting execution at translation-block, instruction, memory, and edge granularities. Every memory access, every control-flow transition, and every call boundary can be observed and overridden. This is what makes it possible to enforce the shared memory model on the binary side and to produce an event stream in the same form as the JIT executor.

Synchronisation with lifted execution. Unicorn, following QEMU's execution model, translates and executes native code in units called *translation blocks* (TBs): contiguous instruction sequences whose boundaries are determined by the emulator's internal translation logic. These boundaries do not in general coincide with lifted basic-block boundaries. Block splitting (subsection 3.2.4), for example, can divide a single contiguous instruction sequence into multiple lifted blocks; one emulator translation unit may therefore span several consecutive lifted blocks (Figure 3.25). Naively firing control-flow events at emulator block boundaries would misalign the observation stream with the JIT side. A further complication is that the emulator's translation mechanism can produce block-entry notifications that do not correspond to genuine control-flow edges (Figure 3.26).

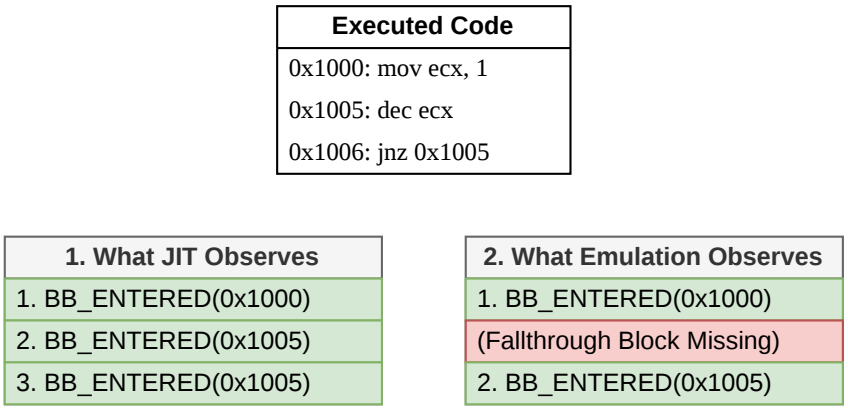


Figure 3.25: TB vs BB block-splitting example. The lifted module (1) knows the CFG of the function, and therefore fires BB-entry events a the first fallthrough point. The emulator (2) doesn't know this boundary in advance—so only fires TB events when the block is entered via the `jnz` instruction.

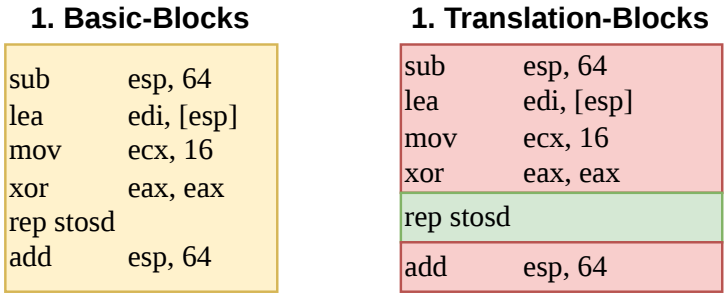


Figure 3.26: TB vs BB spurious TB example. The orange block denotes the BB. The green and red blocks denote the TBs. Unicorn translates `rep` instructions, which repeat an operation several times, into their own separate TBs. This means that the TB boundaries (2) do not necessarily align with the BB boundaries (1), and some TB events may be spurious.

We solve this problem by *normalising* TB events against the reconstructed, lifted CFG, provided by the lifter, using a two-level hook design (Figure 3.27). The first level fires at TB transitions. Its role is classification: it determines whether the transition represents a real control-flow edge, filtering spurious re-translation events using a companion edge-confirmation hook and disassembly of the preceding block's terminal instruction as a fallback. It then queries the set of lifted block start addresses that fall within the destination translation unit and queues them as pending events.

The second level fires at the exact first instruction of each known lifted block. When the emulator's program counter reaches a known lifted block start, this hook dequeues the next pending entry and fires a control-flow progress event with the current register state and the canonical block address. Because this fires at precisely the same logical point as the corresponding instrumentation in the JIT executor, a translation unit spanning two lifted blocks produces two correctly placed events in sequence—exactly matching the JIT side.

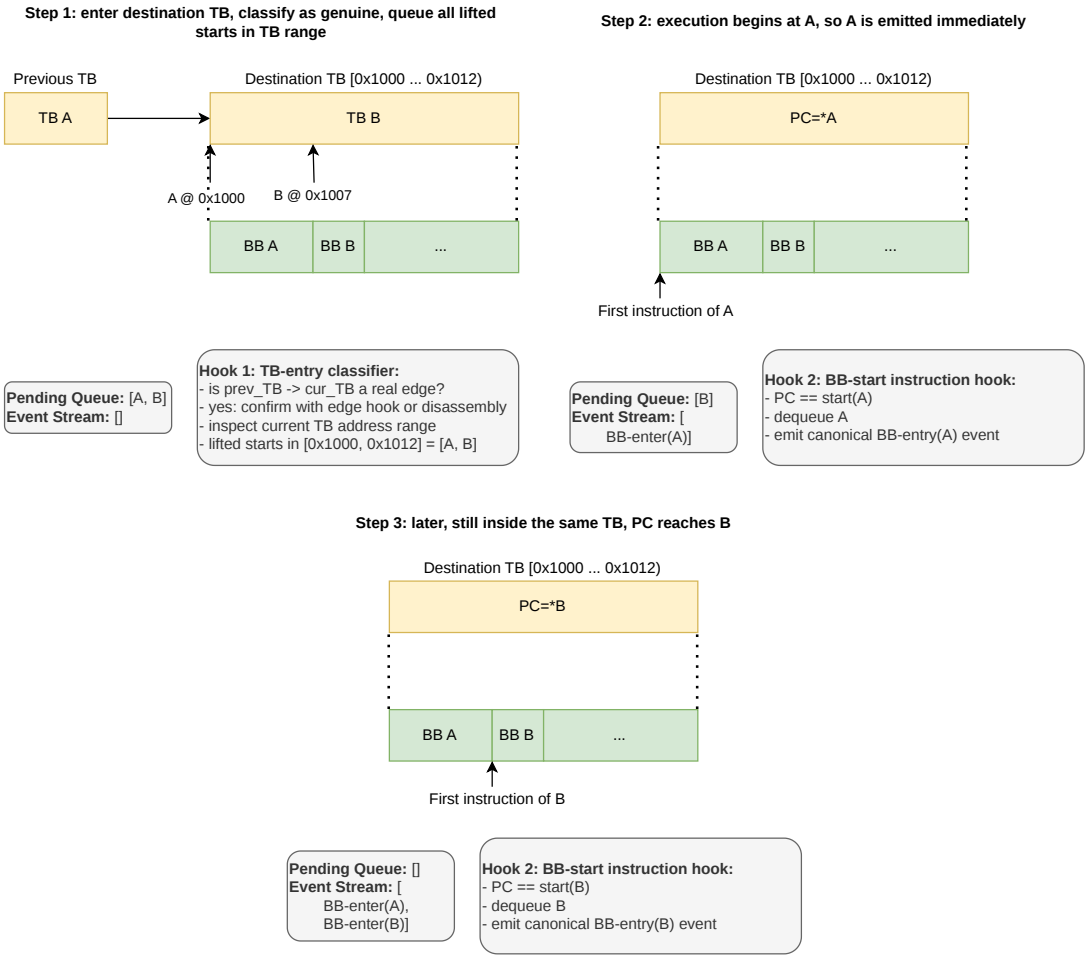


Figure 3.27: Two-level hook interaction in the `UnicornExecutor`. Unicorn/QEMU TBs are shown in yellow; the corresponding lifted BBs are shown in green. After a TB transition, the addresses of both TBs are classified to ensure they are real transitions, and then queued (Step 1). When execution then reaches any of the BBs' addresses (`0x1000` and `0x1007` in this example), the address is dequeued and a `BB-enter` event is emitted (Steps 2 & 3). This ensures that the event stream is aligned with the JIT side, even when TB boundaries do not align with BB boundaries.

External boundary reconstruction. External function calls present a synchronisation challenge on the binary side that does not arise on the JIT side. In the lifted IR, external call boundaries are explicit: the stub mechanism described in subsection 3.2.3 inserted stubs for every external call, and module preparation rewrote these into explicit verification events. When the JIT executor reaches an inter-procedural call stub, it fires an external call event and returns without entering the callee's body.

In the native binary, no such explicit boundaries exist. Unobfuscated binaries typically use import table or procedure linkage table entries to mark external calls, but obfuscated code may eliminate these markers entirely; statically linked code carries none; and indirect calls whose runtime targets resolve to external addresses are indistinguishable from indirect calls within the binary. There is no reliable way to identify external call boundaries from the

assembly alone, as shown by Figure 2.4. If the emulator were permitted to continue past such a boundary, it would continue execution *inline*, also modelling the semantics of all callees, producing a discrepancy relative to the JIT executor's event stream.

The lifting framework resolves this. As described in subsection 3.2.3, the lifter identifies every external call boundary during CFG recovery and records the callee addresses as part of the lift result. The `UnicornExecutor` loads this set at startup and intercepts any transition that reaches a known external callee address. At such a transition, an external call event is fired and the return address is read from the stack to resume emulation at the call's continuation, exactly as the JIT executor does via its synthetic block mechanism. The callee's body is never executed, and the event stream remains aligned.

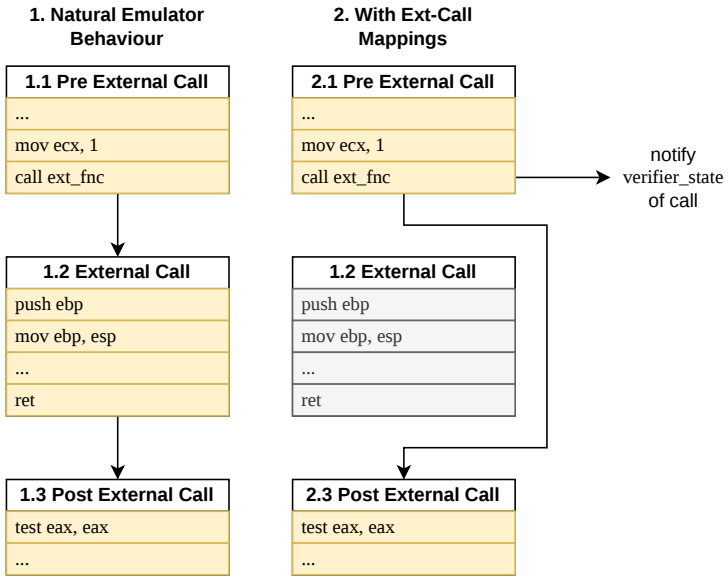


Figure 3.28: External boundary problem under emulation. Yellow denotes instructions emulated, grey denotes instructions not emulated. Under natural execution, the emulator will follow all external and indirect calls (1) and model their semantics as part of the function. We address this by providing it with mappings, so the emulator knows when it would enter an external call, so it instead can report it to the `verifier_state` and skip it (2).

Memory authority. On both sides, `verifier_state`, by owning and funnelling memory events to `paged_memory_state`, remains the sole authority for program memory. In the JIT path this holds by construction after module preparation: every memory access intrinsic defined by the verification helper bitcode routes these accesses to the `verifier_state`. On the emulator side, hooks intercept each memory access before the emulated CPU sees the result, query the shared memory model for the bytes at the requested address, and write the answer back into the emulator's address space. This means that memory accesses remain symmetrical across executors.

In practice.

For construction, the `JITExecutor` only requires an IR module to be provided. The `UnicornExecutor` requires the `BinaryContext`, representing the binary under execution, the entry RVA of the function, and mappings representing basic-block and inter-procedural call boundaries. Construction sets up all the necessary internal structures required for testing. This means that individual test runs remain fast and performant, enabling several runs to be executed quickly.

Executing matching runs also becomes trivial: by providing the same `run_params_t`—a structure describing a run's conditions, including the initial seed—we can execute two executors under the same conditions.

```
// JITExecutor wraps the lifted IR module; UnicornExecutor
   ↪ wraps the native binary.
// ext_call_info derives the external/recursive/indirect call
   ↪ topology from the lift.
auto jit = *JITExecutor::create(
    llvm::CloneModule(*lift_result.module),
    lift_result.lifted_function->getName().str());

auto unicorn = *UnicornExecutor::create(
    bin_ctx, ext_call_info_from(lift_result), entry_rva);

// Both executors are driven with identical initial conditions.
// The same seed produces the same register file and memory
   ↪ content on each side.
run_params_t params{
    .seed      = run_seed,
    .bin_ctx   = &bin_ctx,
    .entry_rva = entry_rva,
};

auto ir_result  = *jit->run_single(params, verif_params);
auto bin_result = *unicorn->run_single(params, verif_params);

// A single call classifies the pair.
auto verdict = compare_runs(ir_result, bin_result);
```

Figure 3.29: Creating and executing a matched executor pair from a single `lift_result_t`. Both executors receive identical `run_params_t`, ensuring the same deterministic initial state $S = (\mathcal{R}, \mathcal{M})$ is applied to both the lifted IR and the native binary. The asymmetry in construction is fully hidden at the comparison layer. `verif_params` is a structure describing *early termination conditions*, which do not affect the behaviour of the run, but allow the verifier to terminate early—features described in subsection 3.3.4 and subsection 3.3.5.

3.3.4 The Checkpoint System

We have now described how two artefacts can be run under our random concrete-testing model, and their final states compared. While this is sufficient to detect semantic discrepancies, it provides limited information about *where* that discrepancy occurred. A function that produces the wrong final register state may have become discrepant at the very first instruction or only in the last few operations; without intermediate comparison points this distinction is invisible. To address this, we introduce a *checkpoint* system that records and

compares the execution state of both artefacts at regular intervals throughout a run.

Checkpoints. A *checkpoint* is a compact record that identifies the state of an artefact under test at a specific point during execution (Figure 3.30). Recording the complete function state (i.e. register file and memory snapshot) at each event would be prohibitively expensive; instead, a checkpoint by default stores a set of *accumulated hashes*, each capturing a distinct semantic dimension of execution history up to that point. The ordered sequence of checkpoints produced over the course of a run constitutes the *checkpoint record* for that run.

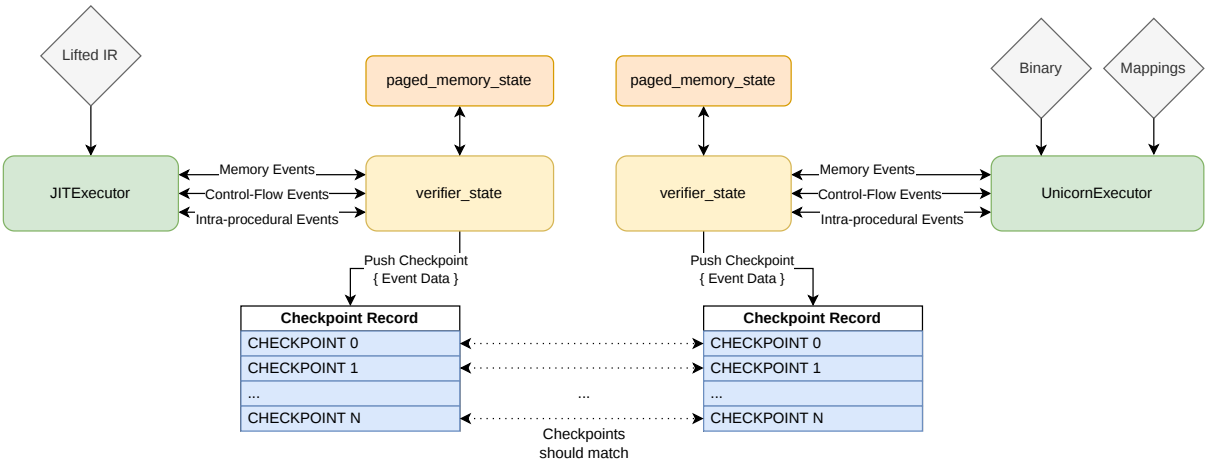


Figure 3.30: The checkpoint model. The `verifier_state` can record checkpoints at each event, accumulating intermediate information about the execution history in several independent hash dimensions. The checkpoint record is a chronological sequence of these checkpoints, which can be compared between runs to identify when and how discrepancies arise.

Hash dimensions. To support fine-grained discrepancy classification, the checkpoint system maintains several independent hash streams in parallel, each accumulating a different aspect of program behaviour. Because checkpoints are recorded chronologically, each accumulation step need only incorporate the data contributed by the current event; prior state is preserved in the running hash.

A checkpoint contains the following dimension hashes:

- `state_hash` accumulates the register file $\mathcal{R}$ at each event, covering the 16 general-purpose registers and the packed flags register.
- `bb_trace_hash` accumulates the sequence of basic-block entry RVAs, capturing the control-flow path taken through the function.
- `mem_write_hash` accumulates the write sequence in order; each write contributes its address, value, and width, so the hash reflects not just what was written but in what sequence.

- `memory_contents_hash` is sampled from `paged_memory_state` at each checkpoint; it is order- and width-insensitive, covering the current content of every written byte.
- `ext_call_hash` accumulates the sequence of external call target addresses.
- `read_hash` accumulates the sequence of memory reads and their values.

After each dimension hash is updated, a *combined hash* is computed by mixing all active dimension hashes together (Figure 3.31). This combined hash is a lightweight fingerprint of the artefact's full observable state at the time the checkpoint was recorded.

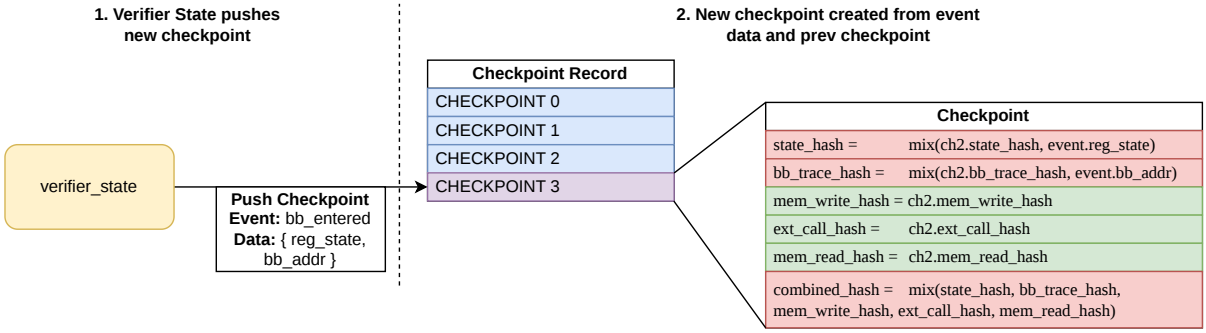


Figure 3.31: Example of checkpoint creation on event. Depending on the event, different dimension hashes are updated with the relevant data, and the combined hash is recomputed. The updated hashes (red) consist of the data of the previous checkpoint (checkpoint 2) mixed with the new data contributed by the current event. The unchanged hashes (green) are carried over without modification. The resulting checkpoint (checkpoint 3) reflects the full execution history up to that point.

Detailed mode. In normal operation, checkpoints record only the accumulated hash dimensions described above, which is sufficient for fast synchronisation and discrepancy detection. When a discrepancy has been identified and requires deeper investigation, checkpoints can operate in a *detailed mode*: in addition to the running hashes, every event is recorded in full, including complete register snapshots, accessed memory values, etc. This data is expensive to store and unnecessary during routine verification runs; detailed mode is therefore disabled by default. It is enabled selectively when replaying a discrepant seed in a deep-dive rerun, where the full chronological event record is necessary to isolate the root cause. The deep-dive mechanism that consumes this data is described in subsection 3.3.7.

Synchronisation events. All artefacts under test produce the same set of events through the observation interface described in subsection 3.3.3, but not all events need be synchronised. The framework is designed to operate with transformed IR as well as raw lifted IR, as depicted in Figure 3.1. When comparing the raw lifted IR against the original binary, every event on the IR side should correspond to a matching event under emulation, and all semantic dimensions are expected to agree. After a transformation pass has been applied, this correspondence may no longer hold. A pass that reorders memory accesses, merges basic blocks,

or eliminates redundant writes changes the event stream without changing the function's net effect; synchronising on every event would produce spurious mismatches.

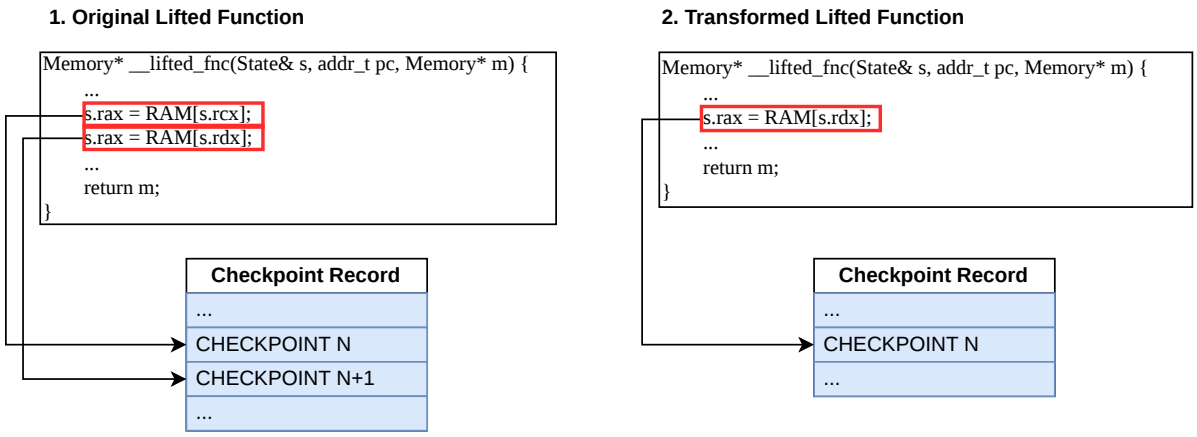


Figure 3.32: Example of event desynchronisation before and after an IR transformation. The `RAM` accesses are raised to intrinsics which generate events after preparation for verification (subsection 3.3.3). In this scenario, a transformation pass has killed the first read. In the original IR (1), two read events would be emitted; in the transformed IR (2), only one such event would be emitted. If the checkpoint system were configured to record on all events, this would produce a mismatch at the first read event.

The checkpoint system therefore exposes two independent configuration axes, which constitute the *checkpoint profile*:

- `record_on` specifies which event types trigger a checkpoint to be appended to the record.
- `compare_on` specifies which dimension hashes are incorporated into the combined hash.

Together these allow the user to tailor checkpoint granularity to the semantics a given transformation is expected to preserve.

We provide two checkpoint profiles by default (Figure 3.33):

- The RAW-STRICT profile records on all events and integrates all per-dimension hashes into its combined hash. It is designed for tests where the two artefacts are tightly synchronised, such as when comparing the raw lifted IR against the original binary.
- The OPTIMISED-RELAXED profile conservatively records on only *external call* events, incorporating the register state, memory contents and external call history hashes into its combined hash. It is designed for testing of artefacts across transformation passes, where there may no longer be an exact mapping of synchronisation events.

1. compare_on: ALL

Checkpoint
state_hash = ...
bb_trace_hash = ...
mem_write_hash = ...
ext_call_hash = ...
mem_read_hash = ...
combined_hash = mix(state_hash, bb_trace_hash, mem_write_hash, ext_call_hash, mem_read_hash)

2. compare_on: STATE, MEM_WRITE, EXT_CALL

Checkpoint
state_hash = ...
bb_trace_hash = ...
mem_write_hash = ...
ext_call_hash = ...
mem_read_hash = ...
combined_hash = mix(state_hash, mem_write_hash, ext_call_hash)

Figure 3.33: The RAW-STRICT and OPTIMISED-RELAXED default checkpoint profiles' effects on the `combined_hash`. By specifying different `compare_on` configurations, the two profiles produce different combined hashes at the same checkpoint. The RAW-STRICT profile (1) incorporates all dimension hashes, while the OPTIMISED-RELAXED profile (2) incorporates only a subset of dimension hashes.

Apart from these default profiles, users can create custom ones to encode precisely what they expect a given transformation to preserve.

Integration with the infrastructure. The checkpoint system is implemented as a layer within `verifier_state`, sitting between the executor event callbacks and the comparison infrastructure. When an executor fires an event through the observation interface, `verifier_state` first checks whether the event type appears in `record_on`. If so, it extracts the event data—register state, addresses, values—and feeds each field into the appropriate dimension hash. The combined hash is then computed, using dimensions enabled in `compare_on`, and the resulting checkpoint appended to the record. If the event type is not enabled, the event is still processed normally, but no checkpoint is recorded.

This placement is what makes the checkpoint system executor-agnostic. Neither the JIT executor nor the emulator has any direct knowledge of checkpointing; both simply fire events into the observation interface. The checkpoint configuration is held by `verifier_state` and applied uniformly regardless of which executor is active. The same checkpoint record structure is produced whether the artefact under test is JIT-compiled IR or an emulated binary.

A verification run can also be provided a *previous checkpoint record*—the checkpoint record of a prior run—as part of its parameters. This allows the run to be constrained by a prior execution's checkpoints: if the run reaches a checkpoint depth for which a previous record exists, it can compare its current checkpoint against the previous record's checkpoint at that depth, and trigger early termination (through the mechanisms described in subsection 3.3.3) if they mismatch. This allows runs to derive meaningful information even when they cannot be executed to completion, a process further described in subsection 3.3.6.

3.3.5 Handling Loops

A function that contains a loop presents a structural problem for any differential testing approach that drives execution with arbitrary initial state. The nature of this state—using random values for all inputs—means that certain types of programs can remain stuck in loops for an unfeasibly-high number of iterations, or even never terminate. Such is the case with loops which depend on a certain sentinel value to be satisfied before the loop is exited.

```
void fnc(char* ptr, uint64_t limit) {
    for (uint64_t i = 0; i < limit; i++)
        ptr[i]++;
}
```

Figure 3.34: A motivating example for a hard loop to break out of given random input due to bounding. The `limit` parameter is a random input, uniformly distributed between 0 and `0xFFFF'FFFF'FFFF'FFFF`. Hence we would, on average, expect this loop to execute for `0x7FFF'FFFF'FFFF'FFFF` iterations during our runs.

```
void fnc(uint64_t* ptr) {
    for (; *ptr != 0; ++ptr)
        *ptr++;
}
```

Figure 3.35: A motivating example for a hard loop to break out of given random input due to a sentinel check. Given our random memory model, the `*ptr` value will be uniformly distributed between 0 and `0xFFFF'FFFF'FFFF'FFFF`, each run would have a 1 in `0xFFFF'FFFF'FFFF'FFFF+1` chance to exit the loop.

Execution Limits

The first line of defence is hard execution limits imposed on each run. We allow the user to specify two independent limits for every verification run.

`bb_limit` caps the total number of basic-block entries across the entire run. If this limit is reached, the run terminates immediately. This is the coarsest possible limit and serves as a global safety net against any unbounded execution.

`loop_limit` caps the number of times any individual back-edge may be traversed. The verifier state applies a *revisit criterion* to classify an edge (*from*, *to*) as a back-edge: the edge is a back-edge if `to` was already present in the visited-block map when this particular traversal from `from` first occurred. A per-edge counter is incremented on each traversal; once it reaches `loop_limit`, the run terminates.

Provided that two runs reach the same checkpoint depth before either limit is hit, we can still compare their states at that point and issue a verdict. However, if one run hits a limit before the other, we cannot compare their states at that point—one run may have executed for much longer than the other—so we re-run the shorter run constrained by the longer run's checkpoint record, establishing a shared execution horizon before comparison.

Sentinel Injection

Hard limits truncate problematic executions, but they do not address the underlying problem. Our goal is to explore as much of the program's state space as possible. If a random input repeatedly drives execution into an early, hard-to-exit loop, later discrepancies may remain unobserved. We therefore also intervene at the source: the randomised inputs themselves.

Sentinel injection replaces selected randomised inputs with user-specified substitute values, each applied with configurable probability (Figure 3.36). This is particularly useful for loops which exit loops based on a specific value, as in Figure 3.35. The mechanism is implemented by a *sentinel oracle*. A *sentinel* specifies a substitution value, either an exact read width or a wildcard width, and an inverse-probability parameter `inv_prob`; `inv_prob` $= N$ corresponds to an injection rate of roughly one in N eligible queries. The oracle is consulted both when initialising the register file and when returning values from dynamically generated memory.

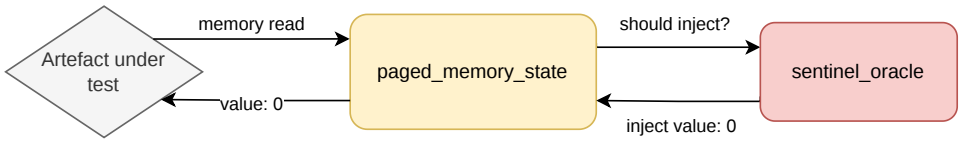


Figure 3.36: Simplified example of sentinel injection. During a run, when the artefact issues a memory read, it is routed to the `paged_memory_state`, which then consults the `sentinel_oracle` (configured with the user's sentinel and inverse-probability parameters) to determine whether it should substitute the otherwise randomly-generated value with a sentinel value. The sentinel oracle is also consulted for initial register values during run setup (not shown).

Symmetric injection. Sentinel injection preserves the differential-testing invariant. Although the executors do not share a single memory object, they are initialised from the same seed and query the same oracle using deterministic slot seeds. Consequently, a sentinel that fires on the JIT side also fires on the Unicorn side at the same logical input slot, yielding the same substituted value.

Value bounding & Alignment. Independently of sentinel injection, the framework can restrict generated inputs to a configurable interval $[min, max]$. This applies to both initial register values and dynamically generated memory contents. It is useful for functions whose

behaviour is only meaningful within a known range, such as the `limit` parameter in Figure 3.34.

The framework can also enforce an alignment constraint on generated inputs, again for both initial register values and dynamically generated memory contents. This is useful for programs that expect aligned pointers or aligned data.

These features aim to enable the verification pipeline to handle a diverse range of programs, including ones which exhibit control-flow schemes that are difficult for our randomised testing approach to sufficiently cover. Their configurability allows the user to adapt the verifier's behaviour to the criteria of the programs they are testing.

3.3.6 Pipelining

In the preceding sections, we described the wide verification feature set that the framework provides, and how these features may be configured. Now we will detail how they can be combined to create a *verification pipeline*, which instructs how exactly a verification *campaign* should be run.

Pipelines & Stages. A *pipeline* is responsible for running the verification *campaign* on two artefacts, and providing the user feedback on whether the two artefacts are equivalent or not (Figure 3.37). It consists of *stages*: each stage contains a master seed, which is used to derive the seeds for a series of runs under a particular configuration. We provide several configuration options to the user: related to handling loops (subsection 3.3.5), checkpointing (subsection 3.3.4), and more. For example, the first stage might run 500 tests with a low basic-block and loop limit, while another stage might run 100 runs with a low limit and sentinel-injection enabled. This is motivated by the fact that, as previously established, concrete testing samples the program's state space. As a result, we need several runs to gain confidence in the semantic equivalence of two artefacts, and different configurations may be better suited to exploring different regions of the state space.

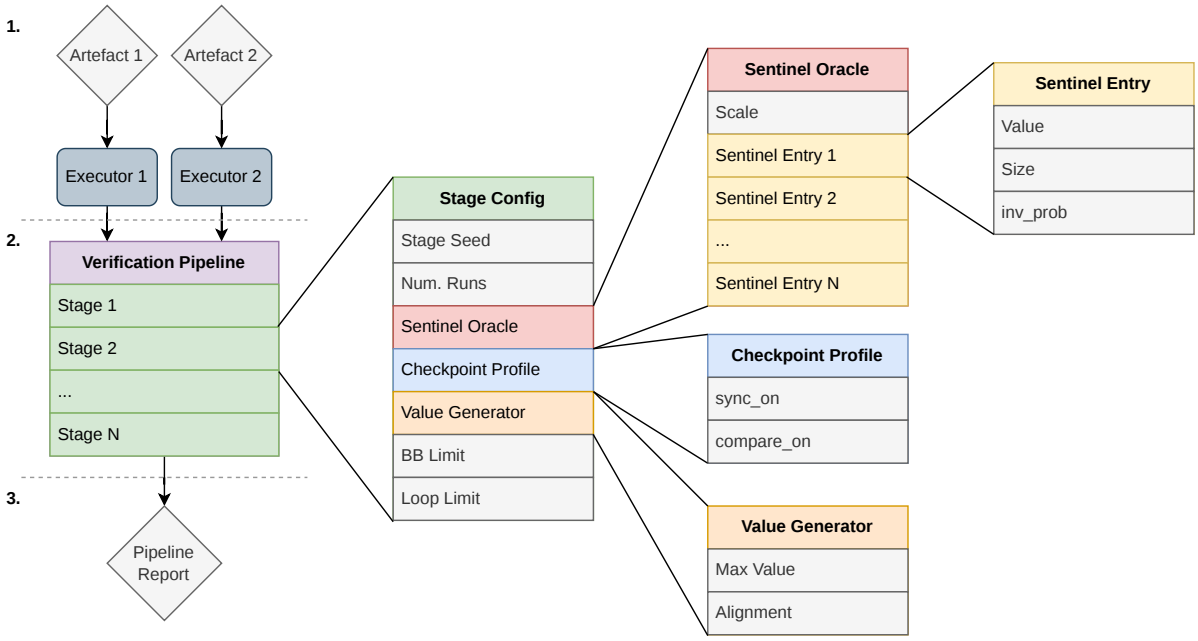


Figure 3.37: Example of a pipelined verification campaign. Executors are first created for the two artefacts under test (1); these are run through a pre-configured pipeline consisting of several stages (2), each consisting of the various configuration options described in the preceding sections. The end result is a pipeline report (3), which contains several pieces of information about the results of the campaign from which the user can derive insights (further explained in subsection 3.3.7).

This allows the user to create a single, reusable configuration collection for how they want the campaign to be verified.

Run Scheduling. Pipelines use checkpoints to get a verdict for each run. When comparing runs A (the *source* artefact) and B (the *target* artefact), the following process is used:

1. Run A executes unconstrained, producing a checkpoint record.
2. Run B then executes constrained by that record: it halts if its accumulated hash disagrees with the corresponding checkpoint, or when the record is exhausted.
3. If both runs terminated at the same checkpoint depth, a semantic comparison is performed and a verdict issued.
4. If the runs reached different depths—one side may have exited early due to a loop or block limit—the source is re-run constrained by the target's record, establishing a shared execution horizon before the final comparison.
5. If a discrepancy is confirmed, a structured diff is computed and stored, identifying which semantic dimensions first disagreed and at which checkpoint.

The benefit of this is that it allows the framework to derive meaningful information even from

runs that cannot be executed to completion, such as those which hit a loop or block limit. It should be noted that, even with checkpoints enabled, a final comparison is still performed on the final states of both runs, so a discrepancy that arises after the last checkpoint can still be detected.

Relation with Executors. A pipeline is executor-agnostic: it is merely a collection of configuration options, not related to any particular artefact. Once created, it can be re-used for any number of executors. For example, a single pipeline can be used for comparing the original lifted IR against the binary, and then comparing that IR against some transformed version.

The end result of a pipeline run is a *pipeline report*, which contains a summary of each stage, which runs failed for which initial seeds. This allows the user to recreate the exact conditions under which a discrepancy was detected, and to investigate the root cause of the problem, a matter we return to in the following subsection.

3.3.7 Feedback for the User

In order to be useful, the framework must provide feedback to the user on the results of a verification campaign. The most basic feedback is a binary verdict of whether a counterexample, or discrepancy, was found. However, this is not sufficient for diagnosing the root cause of a discrepancy; the user requires more information in order to debug the problem effectively. The framework provides three primary sources of information following a pipeline run: *pipeline reports*, *deep-dive reports*, and *CFG visualisations*. Examples of these, including a walk-through of their intended use, are provided in the case study in section 4.6.

Pipeline Reports. As previously described, a pipeline produces a *pipeline report* (Figure 4.4). This is a high-level summary of the results of the campaign, containing a breakdown which runs failed, and a brief explanation of the failure conditions. A pipeline report contains, for each stage: pass and fail counts, the distribution of run exit types (natural return, basic-block limit, loop limit, and checkpoint mismatch), and cumulative basic-block coverage. For any detected discrepancy, the report records the triggering seed and a structured diff that identifies its cause.

Deep-dive Reports. When a discrepancy is identified, the user can request a *deep-dive* on any discrepant seed (Figure 4.4). The framework reruns both executors on that seed with checkpoint detail recording enabled (subsection 3.3.4), then renders a side-by-side chronological table of execution events around the chosen discrepancy. The table presents all events which occurred during execution for each artefact up to the point where the discrepancy was

detected, and includes details for each event. This allows the user to pinpoint the exact point at which the two executions first separated.

CFG Visualisations. A pipeline's report can be overlaid on the lifted function's control-flow graph and exported as a visualisation, with nodes representing basic-blocks and edges representing CFG edges (Figure 4.8). Nodes are coloured by coverage status, with darker nodes indicating that they were executed more frequently. If the pipeline produced a discrepancy, the node where the two artefacts first became discrepant is highlighted in red, with the edges that lead to that node highlighted in blue. This complements the numerical coverage data in the pipeline report by making it immediately apparent which regions of the function were executed during testing, helping the user assess whether a discrepancy-free result might reflect equivalence or simply incomplete exploration of the function's reachable state space.

3.3.8 A Verification Campaign in Practice

The preceding subsections describe the verification machinery in isolation. In normal use, however, the developer combines them into a single campaign: starting from a `lift_result_t`, the lifted module becomes a `JITExecutor`, while the recovered CFG and call-boundary metadata are reused to build the matching `UnicornExecutor`. A checkpoint profile is then selected, the executors are wrapped in a pipeline, and the seeded campaign is run; if any stage reports a discrepancy, the same pipeline can immediately re-run the first failing seed in deep-dive mode. This is the same workflow reused throughout the following chapter.

```
llvm::ExitOnError die;

auto lifted_name = lift_result.lifted_function->getName().str();
auto ext_info = ext_call_info_from(lift_result);
auto bb_sync =
   ↪ eval::make_bb_sync_info(lift_result.basic_blocks);

auto jit = std::shared_ptr<verification::Executor>(die(
    verification::JITExecutor::create(
        llvm::CloneModule(*lift_result.module),
        lifted_name)));

auto native = std::shared_ptr<verification::Executor>(die(
    verification::UnicornExecutor::create(
        bin_ctx,
        std::move(ext_info),
        entry_rva,
        verification::UnicornExecutor::Flags::NONE,
        std::move(bb_sync))));

pipeline::Pipeline campaign(make_default_pipeline(
    "EG1 raw/native",
    lifted_name,
    jit,
    native,
    PipelineCheckpointProfile::RAW_STRICT));

auto report = die(campaign.run(bin_ctx, entry_rva));

for (const auto &stage : report.stages)
{
    if (stage.divergences.empty()) continue;

    auto dive = die(campaign.deep_dive(stage.divergences[0],
       ↪ bin_ctx, entry_rva));
    break; // inspect the first discrepant seed in detail
}
```

Figure 3.38: Running a practical verification campaign from a single `lift_result_t`. The same lifted artefact provides the IR module for the `JITExecutor` and the structural metadata needed to align the `UnicornExecutor` with the lifted CFG. The resulting pipeline produces a high-level campaign report, and any discrepant seed can then be re-run immediately in deep-dive mode.

4 Evaluation

LIFTED's scope spans lifting correctness, algorithmic improvements, and verification infrastructure. Evaluating these three dimensions requires distinct experiments; no single test adequately covers all three implementation goals. This chapter begins with a description of the experimental setup then defines a set of concrete evaluation goals tied to the implementation goals of subsection 3.1.1, and addresses each in turn. A final discussion synthesises the results across all experiments.

4.1 Evaluation Infrastructure & Setup

This section details the environment and artefacts used throughout the experiments.

Environment. All experiments were conducted on a machine with an Intel Core Ultra 7 155H CPU, 32 GB of RAM, and a 1 TB NVMe SSD, running Ubuntu 22.04 under WSL2.

Reproducibility. LIFTED is an open source project available at `https://github.com/0xnobody/lifted` (evaluation commit hash `efc61df1c8c6afa7b46f5d57e6f9e10be2b539e4`; DOI `10.5281/zenodo.19555527`). Detailed test results, as well as the commands required to reproduce them, are available in the LIFTED repository under the `eval/` directory.

Artefacts. Evaluating LIFTED requires a corpus of functions that is: (1) *self-contained*, compilable without third-party dependencies beyond the GCC standard library; (2) *representative*, covering arithmetic, loops, complex control-flow, external calls, indirect calls, and recursion; and (3) *standardised*, drawn from sources used in prior deobfuscation and program-analysis evaluations. A small set of hand-crafted *synthetic* functions was also developed as minimal, targeted unit tests for specific LIFTED features. The two primary corpora satisfying the above criteria are the TUM obfuscation benchmarks and a selection of modules from GNU Coreutils.

TUM. The TUM obfuscation benchmarks are an open-source collection of C programs maintained by TU Munich's Chair for Software and Systems Engineering [6], used as evalua-

tion artefacts in obfuscation research [7, 28]. We use three sub-suites: *tum-basic-algorithms* (63 functions), comprising CS101-style programs that cover loops, recursion, and moderate control-flow complexity; *tum-simple-hash-functions* (21 functions), implementing non-cryptographic hash functions with tight arithmetic loops and bitwise computation patterns; and *tum-small-programs* (48 functions), a set of parameterised programs whose control-flow complexity is varied.

GNU Coreutils. GNU Coreutils is a widely used collection of command-line utilities for Unix-like operating systems, maintained by the GNU Project [29]. Rather than the utilities themselves, we use *gnulib* [30], the portability library underlying Coreutils, specifically its hashing and checksum modules: `gnulib-md5`, `gnulib-sha1`, `gnulib-sha256`, `gnulib-sha512`, and `gnulib-crc`, yielding 22 functions in total. These modules were selected for three reasons: (1) they are relatively self-contained, allowing for integration with the test suite infrastructure without complex build-system modifications; (2) they are used in existing program analysis and deobfuscation evaluations [15, 65], and (3) they exhibit code patterns representative of real-world algorithmic complexity that exercise the lifting and verification infrastructure beyond trivial cases.

The Test Suite Infrastructure. To support reproducible evaluation across multiple suites and obfuscator configurations, a test suite infrastructure was developed and integrated into LIFTED's CMake build system. Each *target* corresponds to a single source file compiled to a position-independent shared library; a post-build script extracts the library's exported function symbols and their entry addresses into a CSV file. A *suite* is a named collection of targets; when an obfuscator is registered, the build system additionally compiles each target under that obfuscator, producing obfuscated binaries alongside their own function-address CSVs. The suite is then described by a flat *suite CSV* listing all (binary, function CSV, obfuscator) triples, which serves as the sole runtime input to the evaluation tooling.

Table 4.1 summarises all suites used in this chapter.

Table 4.1: Test suites used in the evaluation. Function counts reflect individually tested entry points. Obfuscated variants are compiled and evaluated separately.

Suite	Description	Functions
`synthetic`	Hand-crafted unit tests targeting specific LIFTED features (external calls, indirect calls, loops, switches, stack arguments)	13
`tum-basic-algorithms`	CS101-style algorithms: sorting, searching, recursion, arithmetic	63
`tum-small-programs`	Parameterised programs of varying control-flow complexity	48
`tum-simple-hash-functions`	Non-cryptographic hash function implementations	21
`gnu-coreutils`	Production gnulib hashing and checksum algorithms (MD5, SHA-1, SHA-256, SHA-512, CRC-32)	22
Total		**167**

Obfuscators Used. LIFTED is designed to handle both plain and obfuscated binaries via the `LifterCallbacks` interface (section 3.2). Each suite that includes an obfuscated variant is therefore evaluated twice: once with `UnobfuscatedLifterCallbacks` on the plain binary, and once with `PolarisLifterCallbacks` on the obfuscated binary. Plain binaries are compiled at O0 with stack protection and auto-vectorisation disabled to ensure that the compiled binary faithfully reflects the source code. Obfuscated variants are produced by Polaris [4], a machine-IR-level obfuscation framework for modern LLVM that extends the Obfuscator-LLVM design [38].

Polaris is a fork of clang that provides several obfuscation passes exposed during compilation on top of the LLVM infrastructure. We enable three: *indirect calls* (`indcall`), which replaces direct external calls with indirect variants; *indirect branches* (`indbr`), which replaces direct branches with indirect variants; and *opaque predicates*, which inserts dead branches throughout the code. Together, these passes represent the primary obfuscation surface that LIFTED's lifting and target-resolution logic is designed to handle (section 2.3). The remaining Polaris passes require custom analysis and deobfuscation logic to be applied during PC slicing. This is outside LIFTED's current scope and is instead intended as an extension point for end-users.

The LIFTED evaluation and command-line interfaces. All evaluations are driven by `lifted-eval`, LIFTED's evaluation CLI tool. It supports multi-threaded execution of

lifting and verification campaigns across all suites and obfuscator configurations, with results aggregated into several report and CSV files for subsequent analysis. A companion CLI tool, `lifted-cli` allows for *ad-hoc* lifting and verification of individual functions.

4.2 Evaluation Goals

This chapter evaluates LIFTED against the three implementation goals of subsection 3.1.1. We define four concrete, testable *evaluation goals* (EGs), each derived from one or more implementation goals, and structure the remaining sections around them. A final discussion (section 4.7) synthesises the results and presents them as a whole.

EG1: Lifting Correctness Across Suites. EG1 evaluates both the correctness of LIFTED's lifting algorithm and the diagnostic utility of the verification infrastructure. We assess whether the lifter correctly handles the test suites under both callback implementations, and whether the verifier's output, when it detects a discrepancy, is meaningful and attributable to specific, identifiable causes. *(Targets IG 1, IG 2, IG 3).*

EG2: Deobfuscation Pass Correctness. We evaluate whether the verification infrastructure correctly reports no semantic discrepancies when using a trusted deobfuscation pass, using LLVM's O2 optimisation pipeline as a concrete test case. *(Targets IG 3).*

EG3: Fault Injection Study. We evaluate the verifier's sensitivity and specificity: its ability to detect manually injected semantic discrepancies, and its ability to report no discrepancy on semantically equivalent artefacts. *(Targets IG 3).*

EG4: Case Study. We present an end-to-end walkthrough of lifting a function with a reported discrepancy from section 4.3 and diagnosing the fault using the verification infrastructure. *(Targets IG 1, IG 2, IG 3).*

Table 4.2 summarises the mapping between evaluation goals and implementation goals.

Table 4.2: Mapping of evaluation goals to implementation goals.

EG	Description	IG 1	IG 2	IG 3
EG1	Lifting Correctness Across Suites	✓	✓	✓
EG2	Deobfuscation Pass Correctness			✓
EG3	Fault Injection Study			✓
EG4	Case Study	✓	✓	✓

Together, these experiments provide coverage across all three implementation goals, with particular depth on IG 3, reflecting its centrality as LIFTED's most novel contribution. The remainder of this chapter addresses each evaluation goal in turn.

4.3 Lifting Correctness Across Suites

EG1 spans all three implementation goals and is the broadest experiment in this chapter. Its first aim is to establish that LIFTED's lifting algorithm correctly reconstructs arbitrary binary functions into well-formed LLVM IR, and that the modular `LifterCallbacks` interface enables the same lifting core to operate correctly under both plain and obfuscated inputs through substitutable callback implementations, without modification to the algorithm itself (IG1). Second, EG1 evaluates whether LIFTED's algorithmic improvements—specifically the handling of fallthrough edges and inter-procedural calls, which SATURN's published algorithm does not address—operate correctly across a diverse set of real functions (IG2). Third, for any discrepancies that the verification campaign surfaces, EG1 asks whether those discrepancies reflect genuine semantic differences between the lifted IR and the original binary, and whether the deep-dive mechanism provides diagnostic information sufficient to identify their root cause (IG3).

4.3.1 Methodology

Each function in the test suites described in section 4.1 is lifted in two variants and subsequently subjected to a verification campaign. A 300 second lifting timeout was used.

Unobfuscated lifting. In the first variants, functions are compiled at O0 without obfuscation and lifted using `UnobfuscatedLifterCallbacks`, a `LifterCallbacks` implementation tailored to unobfuscated binaries. It assumes that direct branches are not obfuscator-introduced opaque predicates, and that indirect calls are truly indirect rather than obfuscator-injected indirections. It only uses full PC slicing as a fallback. This variant establishes a correctness baseline, isolating the core lifting and CFG-reconstruction algorithm from any obfuscation-induced complexity.

Obfuscated lifting. In the second variant, functions are compiled under Polaris (as described in section 4.1) and lifted using `PolarisLifterCallbacks`, which uses full PC slicing for all blocks. `PolarisLifterCallbacks` additionally applies a lightweight constant-folding pass to address a specific Polaris behaviour: in our evaluation samples, constants that are in reality read-only were placed into writable data sections by the obfuscator, which would otherwise prevent the LLVM's optimisation infrastructure from correctly resolving targets during PC slicing.

Verification campaign. Following each lift, a verification campaign is conducted comparing the raw lifted IR against the original binary using LIFTED's differential testing infrastructure (section 3.3). The campaign serves two purposes. First, it provides an objective measure of lifting correctness: a function is considered correctly lifted if no discrepancies are reported across all runs. Second, for any function that yields a discrepancy, the cause of that discrepancy is investigated manually using the process outlined in section 4.6; all identified discrepancies are taxonomised by root cause and summarised.

Note: this experiment relies on the verifier to assess lifter correctness; the verifier's own reliability is independently established in EG3 (section 4.5).

Pipeline configuration. The verification pipeline comprises two stages (subsection 3.3.6), each executing a sequence of independently seeded runs under a fixed configuration. Table 4.3 summarises the per-stage parameters. The two-stage configuration is deliberate: the wide sweep is intended to explore a broad input range to surface discrepancies that only manifest on large values or specific alignments, while the small sweep is intended to break out of hard-to-exit loops (subsection 3.3.5). Both stages use sentinel injection of `0` and `0xFFFF'FFFF'FFFF'FFFF` to handle sentinel-bounded loops, such as checks for null terminators in strings.

Table 4.3: Verification pipeline stages used in EG1 (RAW-STRICT checkpoint profile).

Stage	Runs	Input max	Alignment	BB limit	Loop limit
Wide sweep	50	$2^{64} - 1$	8 bytes	5,000	100
Small sweep	100	65,536	1 byte	2,000	50

Checkpoint configuration. The RAW-STRICT checkpoint profile (subsection 3.3.4) is applied throughout this experiment, as the raw lifted IR is expected to be semantically identical to the original binary at every observable synchronisation point.

We consider EG1 satisfied if the majority of functions lift without discrepancy across both configurations, and if all reported discrepancies can be attributed to a bug or issue in the lifter or callback implementation.

4.3.2 Results

Table 4.4: EG1 lifting and verification results (raw lifted IR vs binary) across all test suites and obfuscator configurations.

Suite	Fns	Pass %	Avg. Cov %	Lift (s)	Test (s)
Unobfuscated					
synthetic	13	92.3	88.4	0.206	0.545
tum-basic-algorithms	64	92.2	81.1	0.254	0.863
tum-small-programs	48	93.8	73.7	0.136	0.313
tum-simple-hash-functions	21	100	74.6	0.102	0.426
gnu-coreutils	22	95.5	68.0	1.633	3.697
Weighted avg.	—	**94.0**	**76.9**	**0.378**	**0.998**
Polaris					
synthetic	13	92.3	91.7	6.721	0.275
tum-basic-algorithms	58[a]	91.4	88.5	155.077	1.157
tum-small-programs	48	89.6	84.2	24.769	0.537
tum-simple-hash-functions	21	95.2	75.2	8.277	0.530
gnu-coreutils	18[b]	94.4	72.4	31.092	0.312
Weighted avg.	—	**91.8**	**83.8**	**69.647**	**0.716**

[a] Polaris-compiled *tum-basic-algorithms* contains fewer exported functions than the unobfuscated build due to compiler differences.

[b] In Polaris-compiled *gnu-coreutils*, four functions timed out during lifting and are therefore omitted from the reported counts.

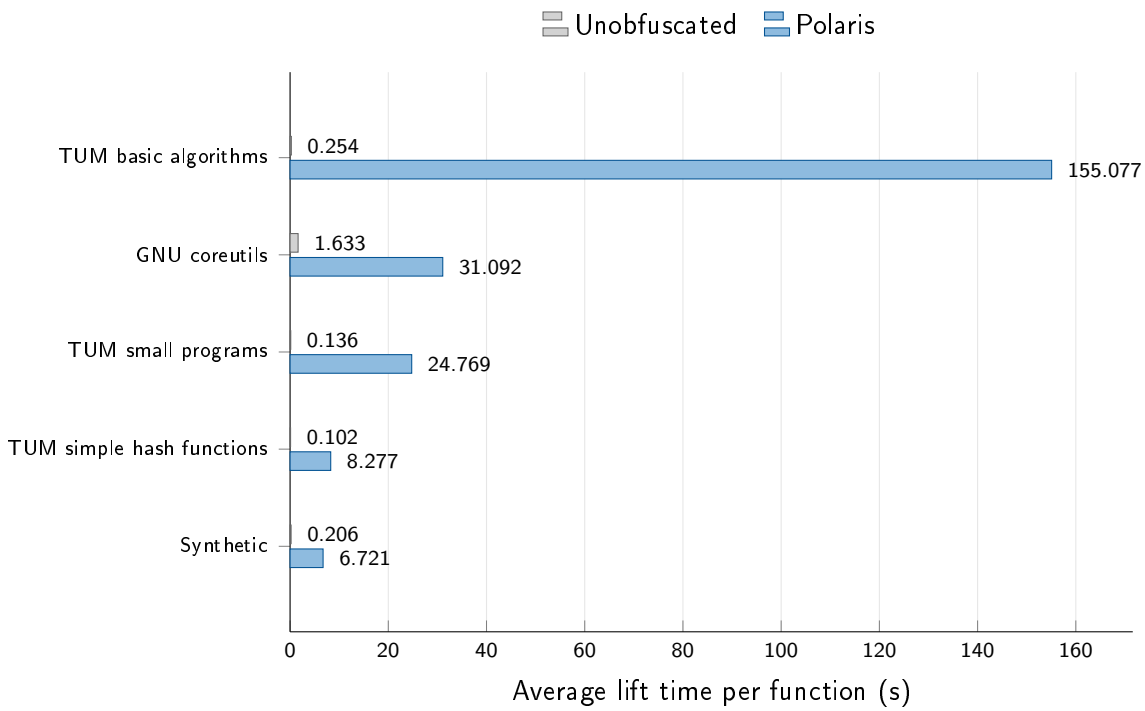


Figure 4.1: Average EG1 lift time per function for unobfuscated and Polaris binaries, from Table 4.4. The Polaris *GNU coreutils* bar reflects completed lifts only, matching the table and excluding four Polaris timeouts.

Table 4.5: EG1 discrepancy taxonomy. Each row is a single non-passing (target, function) pair. The taxonomy is derived from manual analysis of each discrepancy, attributing each to a specific root cause.

Obfuscator	Function	Taxonomy
synthetic		
None	`indirect-call::test`	CFG recovery
Polaris	`indirect-call::test`	CFG recovery
tum-basic-algorithms		
None	`binarymult::main`	Undefined flags
None	`factorialrec::fact`	Undefined flags
None	`factorialrec::fact.localalias`	Undefined flags
None	`multtable::main`	Undefined flags
None	`numtowords::main`	CFG recovery
Polaris	`comparestrings::stringCompare`	CFG recovery
Polaris	`minmaxarray::main`	Stack overflow
Polaris	`multtable::main`	Undefined flags
Polaris	`numtowords::main`	CFG recovery
Polaris	`romannumerals::main`	CFG recovery
tum-simple-hash-functions		
Polaris	`rshash::RSHash`	Undefined flags
tum-small-programs		
None	`16b-1-1-1-0-dc-1-1-0::main`	Undefined flags
None	`16b-1-1-1-1-dc-1-1-0::main`	Undefined flags
None	`1b-1-2-1-1-eq127-1-0-1::main`	Undefined flags
Polaris	`16b-1-1-1-0-dc-1-1-0::main`	Undefined flags
Polaris	`1b-1-2-1-1-gt127-1-0-1::main`	Undefined flags
Polaris	`1b-1-1-2-2-gt127_eq63_dep-0-0-0::main`	CFG recovery
Polaris	`1b-1-2-2-1-gt127-0-0-0::main`	CFG recovery
Polaris	`1b-1-2-2-2-gt63_gt127_dep-0-0-0::main`	CFG recovery
gnu-coreutils		
None	`gnulib-sha1::sha1_finish_ctx`	Undefined flags
Polaris	`gnulib-sha512::sha512_process_bytes`	JIT materialisation
Polaris	`gnulib-md5::md5_process_block`	Lift timeout
Polaris	`gnulib-sha1::sha1_process_block`	Lift timeout
Polaris	`gnulib-sha256::sha256_process_block`	Lift timeout
Polaris	`gnulib-sha512::sha512_process_block`	Lift timeout

Discrepancy root-cause summaries are outlined below.

CFG Recovery. This failure mode represents our `LifterCallbacks` failing to resolve a target (successor) of a BB. This can happen for several reasons: for example, we do not currently implement heuristics for switch statements, which derive their successor addresses from data rather than the instructions themselves. This case is explored in detail in section 4.6.

For the remaining cases, the cause is attributed to insufficient optimisation and deobfuscation passes being applied to the sliced PC function. The optimiser does not have sufficient information—regarding, for example, memory aliasing—to fully optimise the sliced PC function and extract constant targets. This results in certain block successors being missed, resulting in a hole in the resulting explored CFG.

Undefined Flags. Certain x86-64 instructions define certain flags and registers architecturally undefined—that is, after executing the instruction, the value of a certain flag or register can be *anything*. Such is the case for the `shl` instruction for example, as stated in the Intel Software Developer's Manual [36]:

> The CF flag contains the value of the last bit shifted out of the destination operand; it is undefined for SHL and SHR instructions where the count is greater than or equal to the size (in bits) of the destination operand.

Our verifier has no such concept. Our program state either matches in both runs, or it does not.

In this particular case, our Remill lifting implementation arbitrarily writes 0 to all architecturally-undefined values, but the Unicorn emulator exhibits different behaviour, modelling the behaviour CPU that the emulator is running on, which in this case, differs from Remill.

Lift Timeout. The `*_process_block` hash-related functions are particularly large, ranging from 7,052 to 22,700 instructions. This means that our when we rebuild our partial CFG, and optimise it in an attempt to resolve targets, we are operating on a very large function, which takes a significant amount of time. This is a limitation in our slicing pipeline in `PolarisLifterCallbacks`.

Stack Overflow. `minmaxarray::main` contains a stack buffer overflow. The function allocates `int a[10]` on the stack and fills with no upper bound, indexing with `argc`. When the random input yields `argc > 11`, the loop writes past `a[9]` into the stack frame, eventually overwriting the saved return address, causing the emulator to return to a spurious address. This is a bug in the verification infrastructure itself, not in the lifted IR; the lifted IR faithfully models the original binary's behaviour, including the buffer overflow.

JIT Materialisation. `gnulib-sha512::sha512_process_bytes` contains vector memory operations—for example `movups`. LIFTED in its current state does not support floating-

point instructions. This emits Remill intrinsics that are not supported by the infrastructure, causing the JIT executor to report an error.

4.3.3 Discussion

EG1 is partially satisfied. Both variants achieve pass rates above 90% and every non-passing case has been attributed to a root-cause.

Unobfuscated lifting is fast, averaging 0.38 s per function; the low cost reflects the fact that `UnobfuscatedLifterCallbacks` defaults to using instruction-provided basic block targets. Polaris lifting is substantially more expensive, averaging 69.6 s, because `PolarisLifterCallbacks` applies full PC slicing to every basic block rather than using it as a fallback. The contrast is shown visually in Figure 4.1. This demonstrates that callbacks can be used to implement assumptions present in a particular function variant, tailoring the lifting algorithm to the expected artefact structure. The four lift timeouts in *gnu-coreutils* under Polaris, however, suggest that algorithmic and optimisation improvements are required for very large functions.

Basic-block coverage averages 76.9% (unobfuscated) and 83.8% (Polaris), indicating that the pipeline explores the majority of reachable paths for most functions, suggesting that the pipeline configuration is appropriate for this experiment. The higher Polaris figure is expected, and occurs because the obfuscator splits single blocks into multiple ones, so one basic block in the deobfuscated variant can correspond to several in the obfuscated one.

Undefined-flag discrepancies account for the largest share of failures and are a key current limitation of the verification infrastructure. Similarly, Stack-overflow and JIT-materialisation failures are also attributable to limitations in the verification infrastructure rather than discrepancies present in the lifted IR. CFG-recovery failures reflect the PC-slice optimiser's inability to fully resolve certain indirect-jump targets—a limitation of the current optimisation passes and heuristics in `LifterCallbacks`, not of the core algorithm.

The taxonomy confirms that the verification infrastructure reliably surfaces real lifting defects and provides sufficient diagnostic information to distinguish them. However, it also presents several false-positives: bugs in the verification infrastructure itself rather than real discrepancies in the artefacts under test.

4.4 Deobfuscation Pass Correctness

EG2 targets IG3, examining a question distinct from EG1: can the verification infrastructure correctly compare a transformed IR artefact against an untransformed baseline without generating spurious discrepancies attributable to the transformation itself rather than to a

real semantic change? This property is a prerequisite for the lift → verify → deobfuscate → verify workflow: a verifier that cannot tolerate the structural changes introduced by a semantics-preserving transformation is not usable in this manner.

4.4.1 Methodology

Use of LLVM O2 as a proxy transformation. As discussed in subsection 3.1.1, the scope of LIFTED does not extend to developing bespoke deobfuscation passes; these are intended as user-supplied extensions via the `LifterCallbacks` interface. To evaluate the verification infrastructure in this context, we employ LLVM's built-in O2 optimisation pipeline as a proxy for a deobfuscation transformation. Like the SATURN-style workflow, O2 aggressively rewrites LLVM IR while aiming to preserve observable behaviour, so it provides a useful first test of whether the verifier accepts semantics-preserving structural change [33, 52].

Experimental corpus. This experiment operates exclusively on functions that passed EG1 without discrepancy. Functions that already exhibit a discrepancy between the raw lifted IR and the original binary provide no meaningful baseline: any further comparison would conflate pre-existing defects with the effect of the transformation. For each qualifying function, the raw lifted IR module is cloned; the O2 pipeline is applied to the clone; and the verification campaign then runs on the two IR artefacts.

Pipeline and checkpoint configuration. The same two-stage seed-sweep pipeline used in EG1 (Table 4.3) is retained, as the input generation and execution bounds remain appropriate. The checkpoint profile, however, is changed to OPTIMISED-RELAXED (subsection 3.3.4), to account for the structural changes that O2 introduces.

We consider EG2 satisfied if every function in the experimental corpus verifies as equivalent under O2, confirming that the OPTIMISED-RELAXED checkpoint profile correctly admits a trusted semantics-preserving transformation without false positives.

4.4.2 Results

The results present the same data as EG1, but instead comparing the raw lifted IR against the raw lifted IR after the LLVM O2 optimisation pass had been applied. We omit lifting times as they are identical to EG1 (the same lifting process is used for both experiments).

Table 4.6: EG2 lifting and verification (raw lifted IR vs O2 IR) results across all test suites and obfuscator configurations.

Suite	Fns	Pass %	Avg. Cov %	Test (s)
Unobfuscated				
synthetic	12	100	88.4	0.059
tum-basic-algorithms	59	100	81.1	0.134
tum-small-programs	45	100	73.7	0.058
tum-simple-hash-functions	21	100	74.6	0.098
gnu-coreutils	21	100	68.0	0.506
Weighted avg.	—	**100.0**	**76.9**	**0.151**
Polaris				
synthetic	12	100	91.7	0.051
tum-basic-algorithms	53	100	88.5	0.322
tum-small-programs	43	100	84.2	0.170
tum-simple-hash-functions	20	100	75.2	0.183
gnu-coreutils	17	100	72.4	0.099
Weighted avg.	—	**100.0**	**83.8**	**0.209**

4.4.3 Discussion

EG2 is satisfied. Every qualifying function verifies as equivalent under the O2 transformation, with a 100% pass rate across all suites and both obfuscator configurations (Table 4.6).

The OPTIMISED-RELAXED checkpoint profile correctly tolerates the structural changes O2 introduces without generating false positives. This confirms that the profile is a sound basis for the lift → verify → deobfuscate → verify workflow: the verifier accepts a trusted semantics-preserving transformation.

Coverage figures match EG1, as expected: the experimental corpus is a strict subset of the EG1 passing functions and the pipeline parameters are unchanged. Verification times are markedly lower than in EG1 (unobfuscated: 0.15 s vs 1.00 s; Polaris: 0.21 s vs 0.72 s), reflecting the absence of Unicorn emulation overhead in the JIT-vs-JIT mode.

Methodology limits, however, should be acknowledged. Fully evaluating the lift → verify → deobfuscate → verify workflow would require the development of several passes that are representative of real-world deobfuscations. As stated in subsection 3.1.1, this is outside of LIFTED's scope. As a result, it remains unclear how well checkpoint profiles can tolerate the wide variety of structural changes that such passes can introduce, and how effective the verification infrastructure is at discovering bugs in such passes. This gap is partially filled with the *fault injection* study (EG3), outlined in the following section.

4.5 Fault Injection Study

EG3 establishes verifier confidence independently of the lifting experiments. EG1 and EG2 relied on the verifier as an instrument to assess lifter and transformation correctness; EG3 evaluates the verifier directly. We measure two complementary properties: *sensitivity* (whether the verifier reliably detects injected semantic discrepancies) and *specificity* (whether it correctly refrains from reporting discrepancies when none exist).

4.5.1 Methodology

Approach. Mutation testing [23] is an established technique for evaluating the quality of verification and testing infrastructure [37]. A *mutant* is produced by applying a single, semantics-altering transformation to a known-correct artefact; the verification pipeline is then run against the mutant. A mutant that causes the verifier to report a discrepancy is said to be *killed*; one that passes undetected has *survived*. Sensitivity is quantified by the *kill rate*: the fraction of all tested mutants that are killed. Specificity is established separately through a baseline check on the unmodified artefact before any mutations are applied (described below).

In this context, each mutation simulates an error that a binary lifter or deobfuscation pass might introduce at the IR level. The technique is well-suited to this problem because the fault space is finite and enumerable at the IR level, and the ground truth (the original lifted IR) is always available for comparison. Kim et al. [39] applied a structurally similar approach to discover semantic bugs in state-of-the-art binary lifters, establishing mutation testing as a credible methodology for evaluating binary analysis infrastructure.

Mull and libIRM. Mull [24] is an LLVM-based mutation testing framework that applies IR-level semantic mutations directly to compiled programs and evaluates whether an existing test suite detects them. It operates by cloning the LLVM bitcode of the program under test, applying a mutation operator to produce a modified version, JIT-compiling the result, and running tests against it. Its underlying mutation operator library, libIRM [20], defines the taxonomy of integer-arithmetic IR mutations that Mull applies. Mull cannot be used directly in this work: it targets a fixed LLVM API version that is incompatible with the LLVM version used by LIFTED. We therefore reimplement the same mutation operator taxonomy, applying the operators directly to lifted LLVM functions rather than at compilation time. The eight mutation categories are listed in Table 4.7, together with representative examples.

Table 4.7: Mutation categories used in EG3. Categories are adapted from the libIRM taxonomy [20, 24].

Category	Description	Example
`ARITHMETIC`	Substitute arithmetic binary opcode	`add` → `sub`
`CONSTANT_REPL`	Replace a constant operand with 0	`add %x, 5` → `add %x, 0`
`BITWISE`	Substitute bitwise binary opcode	`and` → `or`
`COMPARISON_NEG`	Negate a comparison predicate	`icmp eq` → `icmp ne`
`OPERAND_SWAP`	Swap LHS and RHS of a non-commutative op	`sub a, b` → `sub b, a`
`DROP_COMPUTATION`	Replace instruction result with an operand	`add a, b` → `a`
`NEGATE_TRUNC`	Negate a boolean truncation to `i1`	`trunc x to i1` → `xor t, 1`
`COMPARISON_BDRY`	Off-by-one shift of a comparison predicate	`icmp sgt` → `icmp sge`

Strategy. The mutation campaign follows four steps for each function in the experimental corpus.

1. **Pre-processing.** The function is lifted using `UnobfuscatedLifterCallbacks` and the resulting module is passed through `always_inline` followed by the LLVM O2 optimisation pipeline. Inlining exposes all IR instructions to the mutation enumeration pass; O2 eliminates dead code and redundant computations. This step is critical, as without dead-code elimination, mutations applied to unreachable instructions would not change the function's observable behaviour.

2. **Baseline verification.** A verification campaign is run on the unmodified pre-processed module against the original native binary via Unicorn emulation, using both RAW-STRICT and OPTIMISED-RELAXED checkpoint profiles. Only functions that pass the baseline are admitted to the mutation campaign: a pre-existing discrepancy would prevent clean classification of mutant outcomes. This step simultaneously serves as the *specificity check*: a clean baseline confirms that the verifier does not report false positives on two identical artefacts.

3. **Enumeration and sampling.** All valid (instruction, operator) pairs in the pre-processed function are enumerated across all eight categories. The total pool is then

sampled down to a budget of 200 mutation points using *operator-stratified sampling with proportional remainder*: each active operator class receives an equal initial allocation, with any remaining budget filled uniformly from the unselected pool. We stratify by operator class so rare mutation classes are not lost under a fixed budget, while still sampling a mutant set that reflects meaningful operator-level heterogeneity [13]. The budget of 200 mutation points is a pragmatic compromise: large enough to sample the available mutation space broadly, but small enough to keep per-function evaluation runtime tractable.

4. **Mutant execution.** Each sampled mutant is tested independently. The mutant IR is JIT-compiled and compared against the original native binary using the same pipeline as in EG1. The outcome is classified as *killed* if a discrepancy is reported. The primary metric is the *kill rate* (killed / total mutants).

We consider EG3 satisfied if the kill rate is high across all suites and categories and the baseline passes without false positives.

4.5.2 Results

Table 4.8: Per-suite mutation test specificity results (O2 vs O2 self-test).

Suite	Fns	Pass %		
		Strict	Relaxed	Δ
synthetic	12	100.0	100.0	0.0
tum-basic-algorithms*	59	100.0	100.0	0.0
tum-small-programs	45	100.0	100.0	0.0
tum-simple-hash-functions	21	100.0	100.0	0.0
gnu-coreutils	21	100.0	100.0	0.0
Weighted avg.	—	**100.0**	**100.0**	**0.0**

Table 4.9: Per-suite mutation test sensitivity results.

Suite	Mutants	Killed %		
		Strict	Relaxed	Δ
synthetic	1404	71.9	60.6	-11.3
tum-basic-algorithms*	10738	74.3	61.1	-13.2
tum-small-programs	7465	66.5	55.0	-11.5
tum-simple-hash-functions	2639	69.5	62.0	-7.5
gnu-coreutils	2777	67.0	58.8	-8.2
Weighted avg.	—	**70.5**	**59.1**	**-11.4**

Table 4.10: Per-category mutation test sensitivity results across all test suites.

Category	Mutants	Killed %		
		Strict	Relaxed	Δ
ARITHMETIC	4497	65.7	64.2	-1.5
CONSTANT_REPL	22	54.5	36.4	-18.2
BITWISE	6957	76.7	51.4	-25.3
COMPARISON_NEG	1759	92.8	66.7	-26.1
OPERAND_SWAP	1957	68.9	50.8	-18.1
DROP_COMPUTATION	9155	66.2	65.0	-1.2
NEGATE_TRUNC	0	0.0	0.0	0.0
COMPARISON_BDRY	676	44.5	29.9	-14.6
Weighted avg.	—	**70.5**	**59.1**	**-11.4**

4.5.3 Discussion

EG3 is partially satisfied. Specificity is confirmed at 100% across all suites (Table 4.8): the O2-vs-O2 self-test produces no false positives under either checkpoint profile, establishing that the verifier does not report discrepancies on identical artefacts.

Overall kill rates of 70.5% (RAW-STRICT) and 59.1% (OPTIMISED-RELAXED) are reasonably high but leave a material fraction of mutants undetected (Table 4.9). The 11.4 pp gap between the two profiles is informative: mutants that survive under OPTIMISED-RELAXED but are killed under RAW-STRICT caused program state to become discrepant at some intermediate point and then re-converge by the next synchronisation point. Relaxed profiles therefore trade sensitivity for tolerance of structural transformation—a deliberate design choice

for the deobfuscation use-case, but one that leaves a class of short-lived discrepancies undetected.

Manual investigation of surviving mutants reveals a consistent pattern: despite the O2 preprocessing step, the lifted IR retains several instructions that do not influence the function's observable output. A representative example is residual PC-update computations emitted by Remill for tracking the instruction pointer (e.g. `%block_pc5.i = add i64 %2, 4828`). These instructions are byproducts of the lifting process that the O2 pass fails to eliminate. Mutations applied to such instructions are therefore invisible to the verifier regardless of checkpoint granularity. The measured kill rates are thus a conservative lower bound on true sensitivity; a more aggressive dead-code elimination pass tailored to Remill IR would reduce this dead-code pool and improve the score.

The per-category breakdown (Table 4.10) supports this interpretation. `COMPARISON_NEG` achieves the highest RAW-STRICT kill rate (92.8%), consistent with branch-condition negations immediately redirecting control flow and being caught at the first basic-block checkpoint. `COMPARISON_BDRY` and `CONSTANT_REPL` show the lowest rates (44.5% and 54.5% respectively). Off-by-one mutations often only manifest on narrow boundary inputs; here, their low kill rate is consistent with the known difficulty random testing has in reaching such cases, motivating the hybrid symbolic/concolic extensions discussed in section 5.3 [15, 68, 71]. Constant replacements frequently affect instructions whose output does not reach any observable state, such as the instruction pointer updated previously described.

Finally, it should be noted that the libIRM mutation taxonomy was designed to evaluate test-suite quality for compiled programs; to our knowledge, we are not aware of prior work validating libIRM's mutation taxonomy specifically as a proxy for lifter or deobfuscation-pass faults. The kill rates reported here should therefore be interpreted as a characterisation of the verifier's sensitivity to a well-defined class of IR-level mutations, not as a direct prediction of fault-detection capability in deobfuscator development.

4.6 A Case Study

EG4 turns one EG1 discrepancy into a worked debugging session. The goal is to show how one focused `lifted-cli fn` invocation produces enough evidence to move from a failing seed to a specific lifting defect, and describe the process of taxonomising the discrepancies presented in EG1.

4.6.1 Case Selection

We choose `numtowords::main` from the *tum-basic-algorithms* suite because it is one of the clearest EG1 discrepancies in an unobfuscated binary, yet the underlying source is simple: a

decimal-to-word converter with a normal `switch(digit)` (Figure 4.2). That combination makes it a good demonstration of the workflow promised in section 3.1: starting from one failing seed, the verifier should make it easy to determine what kind of lifting defect produced the discrepancy.

```
...
switch(digit){
    case 0: word[i++] = "zero"; break;
    case 1: word[i++] = "one"; break;
    case 2: word[i++] = "two"; break;
    case 3: word[i++] = "three"; break;
    case 4: word[i++] = "four"; break;
    case 5: word[i++] = "five"; break;
    case 6: word[i++] = "six"; break;
    case 7: word[i++] = "seven"; break;
    case 8: word[i++] = "eight"; break;
    case 9: word[i++] = "nine"; break;

}
...
```

Figure 4.2: A snippet of the `switch` statement from `numtowords::main`. The full function is available at Appendix A1.

4.6.2 Diagnosing the Discrepancy

We rerun the lifting and verification process with the same experimental setup used in EG1. The single command in Figure 4.3 isolates one function, executes the staged pipeline described in subsection 3.3.6, and emits the developer-facing artefacts described in subsection 3.3.7: a pipeline report (Figure 4.4), a deep-dive trace of the first reported discrepancy (Figure 4.5), and a CFG diagram (Figure 4.8). The tool's output is immediately actionable: lifting succeeds, but the raw lifted IR fails verification against the original binary.

```
$ ./build/src/lifted-cli/lifted-cli fn --suite
   ↪ tum-basic-algorithms \\
  --target numtowords --func main \\
  --output-dir eval/lifted-numtowords-case-study \\
  --render-cfg pdf
--- [1/1] libeval-numtowords.so :: main @ 0x0000000000001139 ---
  [LIFTING]  OK
  [STAGE 1]  FAIL  (raw lifted IR vs. Unicorn -- 47
     ↪ divergence(s))
  [STAGE 2]  N/A  (Stage 1 failed -- skipped)
  [RESULT]   FAIL
```

Figure 4.3: `lifted-cli fn` invocation and output. One invocation immediately reveals 47 discrepant runs. Note that *stages* in the output refer to *evaluation stages*: raw lifted IR vs the original binary (Stage 1) and O2-optimised lifted IR vs raw lifted IR (Stage 2); they are distinct from the pipeline stages described in subsection 3.3.6.

Inspecting the *DIFF* shown in the pipeline report (Figure 4.4), we immediately see the problem: the emulated binary (`Src`) and the lifted JIT (`Tgt`) execute the same sequence of 13 basic blocks, but at the next step, the Unicorn run enters a block at `0x1211` that the JIT does not. This is further supported by the exit types: `Src` exits with `NATURAL_EXIT` (i.e. it reaches the end of the function as expected), while `Tgt` exits with `MISSING_BLOCK` (i.e. it tries to execute a block that was not lifted).

```
=== Pipeline: EG1 raw/native[main] ===

[Stage 1] SeedSweep(50 runs, base_seed=0x1338)
  Comparison outcomes:
    PASS:                    42  (84.0%)
    FAIL_DIVERGED:            8  (16.0%)
    FAIL_INCOMPARABLE:        0  (0.0%)
    NEEDS_RERUN triggered:    0  (0.0%)
    NEEDS_RERUN -> PASS:      0  (0.0%)
  Src (final) exit types:  NATURAL_EXIT=50
  Tgt exit types:  NATURAL_EXIT=42  MISSING_BLOCK=8
  Coverage:  11 unique blocks
  Corpus:    50 entries added
  Timing (wall clock, executor run_single only):
    Src:      3.0 ms total,    0.06 ms/run avg
    Tgt:     17.4 ms total,    0.35 ms/run avg

  DIVERGENCES (8):
    Run #7  seed=0xabbc7c36c106aca1
      Src exit: NATURAL_EXIT   Tgt exit: MISSING_BLOCK
[DIFF]
  exit_type: r1=NATURAL_EXIT  r2=MISSING_BLOCK
  bb_trace: r1_len=5  r2_len=6  (1 divergences):
    [5] r1=<absent>  r2=0x1211
  checkpoints: r1_events=13  r2_events=14  (1 divergences):
    [14] field=state
      last 5 BBs: 0x1139  0x1174  0x1337  0x1198  0x11ed    <-
         ↪ state diverged at this BB entry
      r1: state=0x0 bb=0x0 mem=0x0 mem_write=0x0 read=0x0
         ↪ ext_call=0x0
      r2: state=0x7db6e0bdabc101e6 bb=0x84f90ceee6c4b259
         ↪ mem=0x47c6991bc0eb9357 mem_write=0xad911c2d7578085b
         ↪ read=0x0 ext_call=0x0
    Run #12  seed=0x21cdfd9258f82f25
      Src exit: NATURAL_EXIT   Tgt exit: MISSING_BLOCK

...
```

Figure 4.4: An extract from the pipeline report for `numtowords::main`. All reported discrepancies are enumerated (only one shown here; the full report is available under the `eval/` directory in the open-source repository.), and the cause of the discrepancy is presented: a BB trace mismatch. The Unicorn run entered a BB that the JIT did not.

The deep-dive report (Figure 4.5) further confirms our suspicions. Both executions remain synchronised for the first 13 checkpoints. At the next step Unicorn enters `0x1211` while the lifted JIT has no corresponding continuation. That immediately rules out an early arithmetic error in the preceding blocks: the first observable defect is a missing successor after BB `0x11ed`.

```
  [DEEP DIVE S1 1/1]
Status: FAIL_DIVERGED -- first divergence at checkpoint 14

  Chk  Src (JIT)                                                                  Tgt (Unicorn)                                                                  Status
-------------------------------------------------------------------------------------------------------------------------------------------------------------------
    1  BB   rva=0x1139  rax=0xffffffffffffffff rcx=0xffffffffffffffff rdx=0x7  (same)                                                                         [=]
    2  W64 addr=0x7ffeffffff8  val=0x9a59a8ed73714660                          (same)                                                                         [=]
    3  W64 addr=0x7ffefffeff8  val=0x6424c4cc61b209d8                          (same)                                                                         [=]
    4  W32 addr=0x7ffefffe0a4  val=0x1fb9a650                                  (same)                                                                         [=]
    5  W64 addr=0x7ffefffe098  val=0x65f79b3ebe8a0d70                          (same)                                                                         [=]
    6  BB   rva=0x1174  rax=0xffffffffffffffff rcx=0xffffffffffffffff rdx=0x7  (same)                                                                         [=]
    7  W32 addr=0x7ffeffffff0  val=0x0                                         (same)                                                                         [=]
    8  W32 addr=0x7ffeffffff4  val=0x78                                        (same)                                                                         [=]
    9  BB   rva=0x1337  rax=0x78 rcx=0xffffffffffffffff rdx=0x7035e8ec1cdb342  (same)                                                                         [=]
   10  BB   rva=0x1198  rax=0x78 rcx=0xffffffffffffffff rdx=0x7035e8ec1cdb342  (same)                                                                         [=]
   11  W32 addr=0x7ffeffffffe8  val=0x0                                        (same)                                                                         [=]
   12  W32 addr=0x7ffeffffff4  val=0xc                                         (same)                                                                         [=]
   13  BB   rva=0x11ed  rax=0xc rcx=0x0 rdx=0xc rsi=0x0 rdi=0xf8a53e9e1fb9a65  (same)                                                                         [=]

>>> FIRST DIVERGENCE AT CHECKPOINT 14 <<<

   14  <absent>                                                                BB   rva=0x1211  rax=0x7ffff7001211 rcx=0x0 rdx=0x7ffff7002038 rsi=0x0  [!]
```

Figure 4.5: The deep-dive report for the first reported discrepancy in `numtowords::main`. A chronological record of checkpointed events are shown, alongside information for each event. The first 13 checkpoints are synchronised; at checkpoint 14, Unicorn enters a basic block that the lifted JIT does not.

Source and binary inspection explain why `0x11ed` is the right place to look. In the original C program the loop body contains a plain ten-way `switch(digit)` over values `0..9` (Figure 4.2). The compiled binary lowers this to the usual jump-table pattern: it bounds-checks `digit`, loads a 32-bit displacement from a table in the read-only data section, adds the table base back, and executes `jmp rax` (Figure 4.7). Inspecting that table yields the legal case-entry RVAs, shown in Figure 4.6. Notice that the missing block RVA, `0x1211`, is one of these entries.

```
.rodata:2038 jpt_120E        dd offset loc_1211 - 2038h
.rodata:203C                 dd offset loc_1230 - 2038h
.rodata:2040                 dd offset loc_124F - 2038h
.rodata:2044                 dd offset loc_126E - 2038h
.rodata:2048                 dd offset loc_128D - 2038h
.rodata:204C                 dd offset loc_12AC - 2038h
.rodata:2050                 dd offset loc_12C8 - 2038h
.rodata:2054                 dd offset loc_12E4 - 2038h
.rodata:2058                 dd offset loc_1300 - 2038h
.rodata:205C                 dd offset loc_131C - 2038h
```

Figure 4.6: The switch table from the `numtowords::main` binary. Extracted with IDA Pro [35] and adapted for readability. The table is located at 0x2048 in the read-only data section; the entries are 32-bit displacements from that base, their destinations indicated by loc_<destination>.

```
.text:11ED          mov     eax, [rbp+digit]
.text:11F0          lea     rdx, ds:0[rax*4]
.text:11F8          lea     rax, jpt_120E
.text:11FF          mov     eax, ds:((jpt_120E) - 2038h)[rdx+rax]
.text:1202          cdqe
.text:1204          lea     rdx, jpt_120E
.text:120B          add     rax, rdx
.text:120E          db      3Eh ; switch jump
.text:120E          jmp     rax
```

Figure 4.7: The `switch` dispatcher instructions from the `numtowords::main` binary. Extracted with IDA Pro [35] and adapted for readability. The offset calculated from `num` is in the `rdx` register; the jump-table, located at 0x2048, is indexed by `rdx` and loaded into `rax` before the indirect jump.

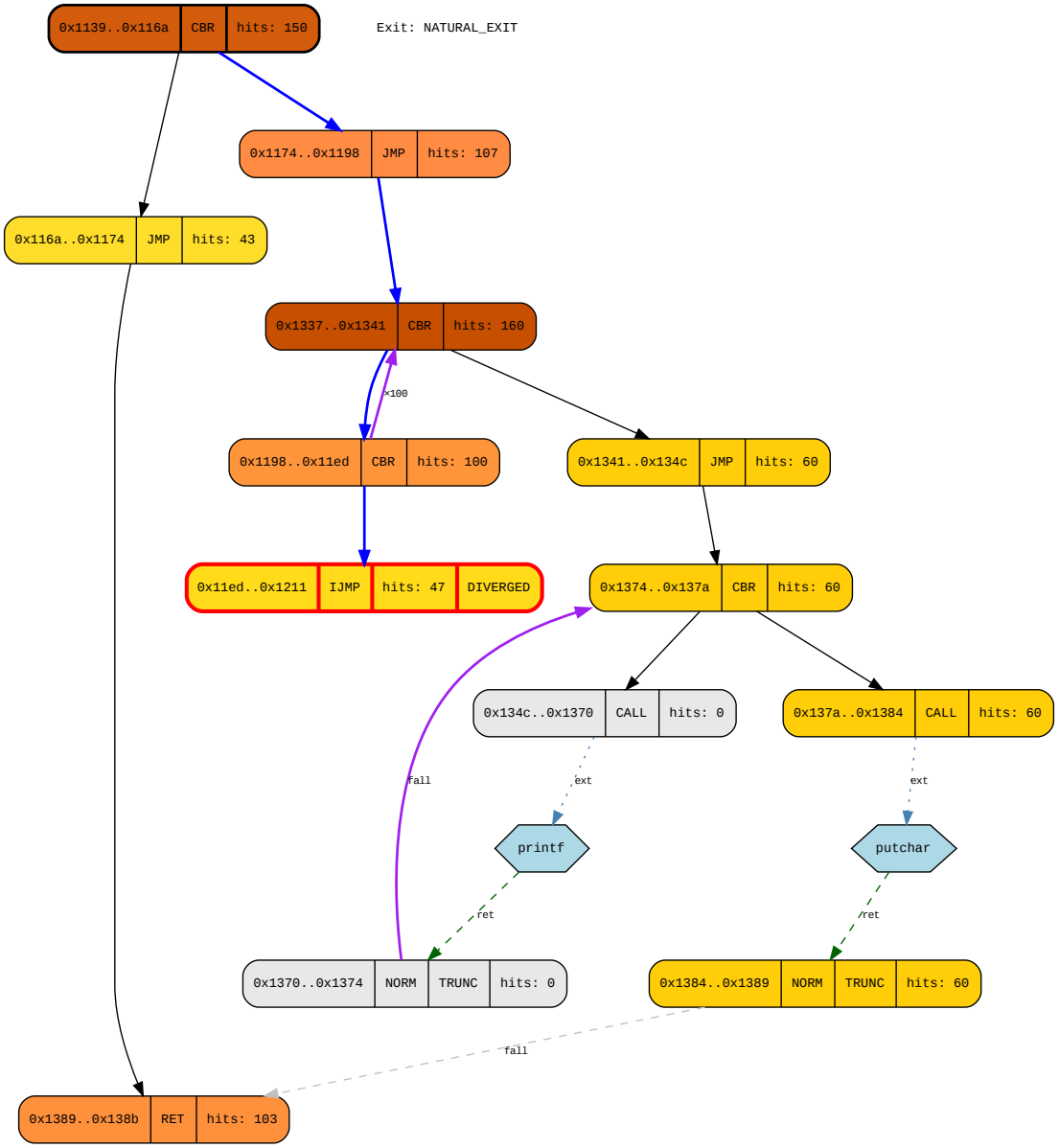


Figure 4.8: Recovered CFG for `numtowords::main` with execution-frequency overlay. The discrepant site is the indirect jump at `0x11ed..0x1211`; the recovered graph stops at that node instead of enumerating the switch-arm successors.

The CFG overlay (Figure 4.8) shows this graphically. The verifier-highlighted node is `0x11ed..0x1211`, labelled `IJMP` (indirect `JMP`) and marked as discrepant, but the recovered graph contains no outgoing edges to any of the ten case blocks. The combination of report, deep-dive, and CFG view therefore reduces the search space to a single question: why did the lift recover the dispatch arithmetic, but not the legal successors?

4.6.3 Root-Cause Attribution

The stitched IR answers the first half of that question. As shown in Figure 4.9, the wrapper around `BB_11ed` executes the lifted block, extracts the next program counter, computes `%next_rva8`, and then branches straight to `dead_end`, which returns from the function. In other words, the local block semantics are present, but the stitched CFG described in section 3.2 has no recovered successors.

```
bb_11ED:
  %24 = call ptr @BB_11ed(ptr %state, i64 %22, ptr %21)
  %25 = call i64 @__extract_rip(ptr %state)
  %next_rva8 = sub i64 %25, %base
  br label %dead_end
```

Figure 4.9: Stitched wrapper around `BB_11ed`. The block computes the next PC, but the rebuilt CFG has no switch over the jump-table successors.

The optimised slice for `BB_11ed` (Figure 4.10) reads the `digit`-dependent table entry from `@RAM`, sign-extends it, adds the jump-table base, and returns the result as the next PC. The missing piece is successor enumeration inside `UnobfuscatedLifterCallbacks::extract_indirect_targets`: the current logic does not recognise compiler-generated jump table, and fails to provide its entries as targets. Once that callback returns no internal targets, the stitcher has no successors to emit and the function simply returns. This is therefore not a verifier false positive; it is a real defect in the current target-resolution path.

```
...
%9 = add nuw nsw i64 %8, 8248
%arrayidx.i186.i.i.i.i = getelementptr inbounds [0 x i8], ptr
   ↪ @RAM, i64 0, i64 %9
%v.0.copyload.i187.i.i.i.i = load i32, ptr
   ↪ %arrayidx.i186.i.i.i.i, align 1, !noalias !78
%conv.i.i1.i155.i.i.i.i = sext i32 %v.0.copyload.i187.i.i.i.i
   ↪ to i64
%add.i.i.i156.i.i.i.i = add nsw i64 %conv.i.i1.i155.i.i.i.i,
   ↪ 8248
ret i64 %add.i.i.i156.i.i.i.i
```

Figure 4.10: Snippet of optimised slice of `BB_11ed`. The value returned (`%add.i.i.i156.i.i.i.i`) represents the successor blocks' RVAs; `%8` is the offset into the jump table. The constant 8248 (`0x2048` in hexadecimal) is the jump-table base. The slice reads the entry at `RAM[%8 + 0x2048]` which contains the digit-dependent offset, sign-extends it, adds the base back, and returns the result as the next PC. The full slice is available under Appendix A2.

The classification becomes clear: native execution reaches valid switch-arm targets such as `0x1211`, but the lifted CFG never models those edges, so the discrepancy is a *CFG recovery* failure caused by missing successors at an indirect jump. In this case the missing-successor site is the compiler-generated jump table for the source-level `switch(digit)`, so the defect lies in switch-target recovery.

This is the classification recorded for the case in Table 4.5, and it illustrates the intended developer workflow: one focused run points to the failing seed, the first bad basic block, the missing successor set, and the callback logic responsible for the omission.

4.7 Discussion

Taken together, the four evaluation goals of Table 4.2 give a positive but qualified verdict on the three implementation goals of subsection 3.1.1. They show that LIFTED delivers a workable SATURN-derived lifting infrastructure and a practically useful verification workflow for iterative deobfuscation development. At the same time, the evidence does not justify a stronger claim of completeness: EG1 and EG3 both expose current boundaries in the lifting process and fault-detection sensitivity. The chapter therefore supports a claim of practical viability rather than full generality.

Table 4.11: Compact verdict on the evaluation goals.

EG	Verdict	Short synthesis takeaway
EG1	Partially satisfied	Lifting succeeds for the large majority of functions in both unobfuscated and Polaris configurations, and the remaining failures are attributable, but recovery, modelling, and scalability limits remain (Table 4.4, Table 4.5).
EG2	Satisfied	The relaxed verifier profile accepts a trusted semantics-preserving LLVM transformation, supporting the intended transformation-validation workflow (Table 4.6, Figure 4.1).
EG3	Partially satisfied	Specificity is strong, but sensitivity is incomplete; the verifier is useful as a practical oracle rather than a proof system (Table 4.8, Table 4.9).
EG4	Satisfied	The diagnostic workflow can narrow a failing run to a concrete lifting defect and the responsible subsystem (Figure 4.4, Figure 4.5, Figure 4.8).

Infrastructure and practical lifting (IG1, IG2). EG1 and EG4 show that LIFTED is more than a proof-of-concept reimplementation. The same core architecture (Figure 3.1) operates across both unobfuscated and Polaris binaries through user-provided callback implementations, while still achieving high pass rates of 94.0% and 91.8% respectively (Table 4.4). This is important because the gap identified in section 2.5 was not only the absence of a generally-available SATURN implementation, but the absence of reusable infrastructure that remains usable on realistic binaries. This was also explicitly labelled as future work by SATURN [33].

Every non-passing case has been attributed to a concrete root cause (Table 4.5), and EG4 (section 4.6) demonstrates that the tooling provides an effective workflow for this. In the present state, however, unresolved indirect-target recovery, undefined-flag modelling, and scalability remain the main practical limits. The evidence therefore supports IG1 and IG2 as claims of practical applicability and extensibility, albeit with implementation-related limitations that must be addressed in future work.

Verification as a development oracle (IG3). Taken together, EG2, EG3, and EG4 show that the JIT-vs-Unicorn and JIT-vs-JIT comparison model, checkpoint system, staged pipelines, and feedback loop described in subsection 3.3.4, subsection 3.3.6, and subsection 3.3.7 work together as intended for development-time confidence building. EG2 shows that the relaxed profile can validate a trusted semantics-preserving transformation, with a

100.0% pass rate across the qualifying corpus under O2 (Table 4.6). EG3 shows that this does not come from indiscriminate permissiveness: specificity remains at 100.0% (Table 4.8), but sensitivity falls to 70.5% under RAW-STRICT and 59.1% under OPTIMISED-RELAXED (Table 4.9, Table 4.10).

EG4 then shows that, when a discrepancy is found, the framework's diagnostics are actionable enough to narrow it from a failing seed to the first bad block and then to the responsible callback logic. The verifier therefore succeeds in its goal of providing a practical translation-validation and differential-testing workflow for lifted IR and deobfuscation development, though the end-to-end lift→verify→deobfuscate→verify pipeline (EG2) was exercised only against LLVM O2; generalisation to more aggressive or obfuscator-specific passes remains an open question.

Integrated thesis-level takeaway. Taken together, these results support the central claim of the thesis: LLVM-based deobfuscation can be made more practical, extensible, and trustworthy through an integrated lifting-plus-verification infrastructure. Relative to the three problems posed in Chapter 1 and the literature gaps summarised in section 2.5, LIFTED contributes the missing substrate: an open SATURN-derived framework, lifting extensions that make realistic cases tractable, and an automated validation workflow that makes failures observable and debuggable.

The evaluation results do not show that LIFTED solves binary deobfuscation in full generality or that its verifier establishes formal semantic equivalence. Instead, they show that each transformation step can be empirically tested against the original binary, discrepancies can be traced to their source, and the remaining failure modes are documented and attributable rather than unknown. The next chapter builds on this verdict by turning from evaluation evidence to an explicit discussion of the system's present limitations and the most important directions for future work.

5 Conclusion

This thesis asked how LLVM-based binary deobfuscation could be made more practical, extensible, and trustworthy. The question arose from three concrete gaps in the literature: the absence of a publicly available implementation of SATURN's lifting-based deobfuscation paradigm, the under-specification of its algorithm for real-world cases, and the absence of automated verification infrastructure for deobfuscated artefacts (section 2.5). This chapter answers that question by summarising the system that was delivered, examining its present boundaries, and identifying the most important directions for future work.

5.1 Summary of Contributions

LIFTED demonstrates that SATURN's core insight—recovering obfuscated machine code into LLVM IR and using compiler infrastructure to undo obfuscation—can be realised as an open, reusable framework with integrated verification. The result is an extensible framework for deobfuscator development, one in which each transformation step can be empirically tested against the original binary and failures can be traced to their source. Chapter 1 introduced this problem through a simple password-validation function (Figure 1.1) in which a sequence of deobfuscation steps could silently transpose the function's branches, producing an incorrect result that was indistinguishable from a correct one without further tooling or manual analysis (Figure 1.3). The contributions summarised below make exactly this class of silent regression observable and diagnosable.

An open, extensible lifting framework. The first problem identified in Chapter 1 was that no open implementation of SATURN existed, and its extensible architecture remained unrealised. LIFTED addresses this directly: it reimplements SATURN's iterative CFG-recovery algorithm, wrapping it in a modular callback interface that decouples the obfuscator-agnostic core from obfuscation-specific target resolution and per-block processing (subsection 3.1.1). The same architecture operates across unobfuscated and Polaris-obfuscated binaries through user-provided callback implementations, achieving pass rates of 94.0% and 91.8% respectively (section 4.3). This moves SATURN from an unavailable research artefact to reusable infrastructure that subsequent work can build upon.

Algorithm extensions for realistic binaries. The second problem was that SATURN's published algorithm does not describe how to handle inter-procedural, indirect, or recursive calls, patterns that are prevalent in real-world software. LIFTED introduces a callee-annotation system that allows exploration to continue past such call sites, alongside a block-splitting mechanism for newly discovered branch targets that fall within already-lifted basic blocks (subsection 3.1.1). The evaluation confirms that these extensions make realistic cases tractable: the evaluation corpus includes real-world binaries with such patterns. These improvements do not resolve all real-world complications—certain scenarios must still be handled by bespoke per-obfuscator logic—but they substantially broaden the class of binaries that the SATURN-derived pipeline can process.

Integrated verification infrastructure. The third and most consequential problem was the absence of any mechanism for checking whether a deobfuscation step preserves semantics. LIFTED's verification framework addresses this through a JIT-vs-Unicorn and JIT-vs-JIT comparison model, a checkpoint-based synchronisation system, and a staged, configurable pipeline that allows for flexible verification strategies (section 3.3, subsection 3.3.4, subsection 3.3.6). The evaluation establishes that this workflow is practically useful: the relaxed verifier profile validates a trusted semantics-preserving transformation (LLVM O2) with a 100.0% pass rate (section 4.4); specificity is 100.0%, meaning valid transformations are never falsely rejected (section 4.5); and when a discrepancy is found, the diagnostics narrow it to the first failing block and the responsible callback logic (section 4.6). The verifier is a practical development oracle, not a proof system, but it makes the class of silent semantic regressions illustrated in Figure 1.3 detectable and attributable.

Taken together, the three contributions close the loop with the problems posed in Chapter 1. The motivating gaps are each addressed by an evaluated component of the delivered system. The evidence supports a claim of practical viability rather than full generality: LIFTED makes deobfuscator development more systematic and more testable, within the boundaries documented below.

5.2 Limitations

The evaluation establishes that LIFTED is practically useful, but it also exposes clear boundaries. This section distinguishes between intentional scope decisions and empirically observed shortcomings, so that future work can target the areas of greatest impact.

Intentional scope boundaries. LIFTED deliberately targets the early stages of the deobfuscation pipeline: recovery of lifted IR from an obfuscated binary, transformation at the IR level, and validation of those transformations (subsection 3.1.1). It does not provide a

full end-to-end deobfuscation pipeline and does not perform recompilation back to a binary; these later stages presuppose a fully finalised deobfuscated function and are explicitly outside scope. Similarly, architecture and ISA coverage is restricted to x86-64, with no support for floating-point, SSE, or AVX instructions. These are real constraints on applicability, but they reflect deliberate scoping rather than design failures.

Lifting and scalability limits. The evaluation of section 4.3 reveals three categories of lifting failure that currently limit coverage. First, undefined-flag modelling: the lifter inherits Remill's precise modelling of x86 flag semantics, but cases remain in which flag values depend on undefined architectural behaviour, producing non-deterministic results during verification. Second, scalability: aggressive PC-slicing and iterative optimisation impose per-function costs that lead to lift timeouts on the largest functions in the Polaris and Coreutils corpora. These constitute the main boundary on the practical lifting contributions (IG1, IG2) as evaluated: the infrastructure works, but its reach is limited by CFG-recovery completeness and the high cost of iterative optimisation.

Verification-model limits. LIFTED's verifier deliberately occupies the lightest-weight point on the assurance spectrum discussed in section 2.4: concrete randomised testing alongside checkpoint-based synchronisation. This design can produce counterexamples but it cannot produce proofs of equivalence. The relaxed checkpoint profile trades sensitivity for tolerance of semantics-preserving transformations, meaning that some injected faults survive when checkpoint comparison is coarsened. Additionally, false positives can arise from verifier-infrastructure bugs rather than genuine IR defects; a small number of such cases were observed and attributed in section 4.3. These limitations are inherent to the chosen verification strategy and its implementation, and they establish the verifier as a development oracle rather than a formal assurance tool.

Evaluation-methodology limits. The mutation-testing evaluation of section 4.5 provides useful sensitivity estimates, but with caveats. The mutation score is a lower bound: dead computations in the lifted IR can survive O2 preprocessing, rendering their mutations unobservable regardless of seed selection. The mutation taxonomy itself, adapted from Mull [24] and libIRM [20], is a proxy for real lifter and deobfuscator bugs, not a direct sample of them. The evaluation corpus, while substantial, does not exhaust all obfuscation families or ISA corner cases. LIFTED has demonstrated utility under its evaluated conditions; broader generality remains to be established by future evaluation and real-world use.

5.3 Future Work

This section outlines future directions in order of proximity to the current system: immediate engineering extensions that follow directly from the limitations above, broader research directions that extend the system's scope and verification strength, and a brief speculative note on AI-assisted development workflows.

Direct engineering extensions. The most immediate priorities are those exposed by the evaluation. Addressing undefined-flag modelling, and similar verifier bugs identified in chapter 4, would reduce non-determinism-induced verification noise. Reducing lift-time overhead, particularly the cost of iterative PC-slicing, would improve scalability on the large functions that currently time out. Broadening ISA coverage to include floating-point and SIMD instructions would extend the framework's reach to a wider range of production binaries. Finally, introducing Remill-aware dead-code elimination or normalisation passes before verification and mutation testing would improve sensitivity estimates by removing dead lifted computations that currently absorb mutations undetected.

Broader applicability beyond deobfuscation. Although LIFTED's verification infrastructure was designed for the lifted-IR deobfuscation setting, its underlying mechanism is not specific to deobfuscation. The same approach could be repurposed for other binary-transformation settings where semantic preservation matters. Post-link binary-rewriting workflows, for example, apply transformations to compiled binaries for performance or security hardening [57]; a checkpoint-synchronised comparison against the original binary could serve as a practical regression oracle in such pipelines.

Similarly, recent decompiler evaluation work has emphasised semantic fidelity and runtime-aware validation, suggesting that a future binary-vs-binary comparison mode could also be useful in decompiler pipelines [31]. These applications would require adaptation of the checkpoint placement strategy and the executor interfaces, but the core comparison engine and diagnostic workflow of section 3.3 would transfer directly. Investigating this broader applicability is a promising direction for extending the impact of the verification contribution beyond the deobfuscation domain.

Extending the deobfuscation pipeline and verification strength. Looking further ahead, the SATURN pipeline itself extends beyond what LIFTED currently implements. Code brightening and recompilation back to a runnable binary are the natural next stages [33]. The verification infrastructure could be extended to cover recompilation by comparing the recompiled binary against the original, closing the full deobfuscation loop. On the verification side, the most impactful extension would be hybrid strategies that combine LIFTED's concrete testing with targeted symbolic or concolic execution for paths that randomised seeds fail

to reach [15, 68, 71]. The mutation-testing results of section 4.5 suggest that most undetected mutations reside in hard-to-reach code—precisely the kind of path-coverage problem where symbolic and concolic methods have demonstrated strength [15, 68, 71]. Integrating such methods into LIFTED's staged pipeline could substantially improve sensitivity without abandoning the lightweight concrete baseline.

AI-assisted deobfuscation development. Finally, LIFTED's structured diagnostics and reproducible counterexamples create a natural oracle for AI-assisted development workflows, analogous to the verification-guided reinforcement-learning loop demonstrated by LLM-VeriOpt [27] for compiler optimisation. An LLM tasked with writing or repairing deobfuscation passes could submit candidate transformations to LIFTED's pipeline and iterate on concrete, localised failure signals rather than opaque outcomes. This direction is speculative and is not a contribution of this thesis, but the infrastructure it requires is precisely what LIFTED delivers.

LIFTED does not solve binary deobfuscation in full generality, nor does its verifier establish formal semantic equivalence. What it delivers is more modest but, we argue, more immediately useful: an open, extensible infrastructure that makes deobfuscator development more systematic, more testable, and more honest about its own limitations. The three gaps identified at the outset of this thesis are each addressed by a concrete component of the delivered system. The remaining boundaries are documented, the failure modes are attributable, and the infrastructure is available for the research community to extend.

Bibliography

[1] Alfred Aho et al. *Compilers. Principles, Techniques, and Tools (Second Edition)*. Jan. 2007.

[2] Adwa Alangari and Ohoud Alharbi. *A Systematic Review of Technical Defenses against Software-Based Cheating in Online Multiplayer Games*. 2025. arXiv: 2512.21377 [cs.CR]. URL: https://arxiv.org/abs/2512.21377. Pre-published.

[3] Dennis Andriesse et al. "An In-Depth Analysis of Disassembly on Full-Scale X86/X64 Binaries". In: Aug. 2016.

[4] Zhou Ang et al. "Polaris: MIR-level Obfuscation in LLVM for Efficient and Robust Decompiler Resistance". In: *Proceedings of the 48th IEEE/ACM International Conference on Software Engineering: Demonstrations (ICSE-demo)*. Rio de Janeiro, Brazil, 2026.

[5] Roberto Baldoni et al. "A Survey of Symbolic Execution Techniques". In: *ACM Comput. Surv.* 51.3 (May 2018). ISSN: 0360-0300. DOI: 10.1145/3182657. URL: https://doi.org/10.1145/3182657.

[6] Sebastian Banescu, Martín Ochoa, and Alexander Pretschner. *Obfuscation Benchmarks: A Set of C Programs for Benchmarking the Strength of Obfuscation*. 2015. URL: https://github.com/tum-i4/obfuscation-benchmarks.

[7] Sebastian Banescu et al. "Code Obfuscation against Symbolic Execution Attacks". In: *Proceedings of the 32nd Annual Conference on Computer Security Applications*. Acsac '16. Los Angeles, California, USA: Association for Computing Machinery, 2016, pp. 189–200. ISBN: 978-1-4503-4771-6. DOI: 10.1145/2991079.2991114. URL: https://doi.org/10.1145/2991079.2991114.

[8] Boaz Barak et al. *On the (Im)Possibility of Obfuscating Programs*. 2001. URL: https://eprint.iacr.org/2001/069.

[9] Sébastien Bardin, Robin David, and Jean-Yves Marion. "Backward-Bounded DSE: Targeting Infeasibility Questions on Obfuscated Codes". In: *2017 IEEE Symposium on Security and Privacy (SP)*. May 2017, pp. 633–651. DOI: 10.1109/SP.2017.36.

[10] Fabrice Bellard. "QEMU, a Fast and Portable Dynamic Translator". In: *2005 USENIX Annual Technical Conference (USENIX ATC 05)*. Anaheim, CA: USENIX Association, Apr. 2005, pp. 41–46. URL: https://www.usenix.org/conference/2005-

usenix-annual-technical-conference/qemu-fast-and-portable-dynamic-translator.

[11] Ryan Berger et al. "Translation Validation for Llvm's Aarch64 Backend". In: *Proc. ACM Program. Lang.* 9 (OOPSLA2 Oct. 2025). DOI: 10.1145/3763147. URL: https://doi.org/10.1145/3763147.

[12] Tim Blazytko et al. "Syntia: Synthesizing the Semantics of Obfuscated Code". In: *USENIX Security Symposium*. 2017.

[13] Leonardo Bottaci. "Efficiency of Mutation Operators and Selective Mutation Strategies: An Empirical Study." In: *Softw. Test., Verif. Reliab.* 9 (Dec. 1999), pp. 205–232. DOI: 10.1002/(SICI)1099-1689(199912)9:43.0.CO;2-X.

[14] David Brumley et al. "BAP: A Binary Analysis Platform". In: *Computer Aided Verification*. Ed. by Ganesh Gopalakrishnan and Shaz Qadeer. Berlin, Heidelberg: Springer Berlin Heidelberg, 2011, pp. 463–469. ISBN: 978-3-642-22110-1.

[15] Cristian Cadar, Daniel Dunbar, and Dawson Engler. "{KLEE}: Unassisted and Automatic Generation of {High-Coverage} Tests for Complex Systems Programs". In: 8th USENIX Symposium on Operating Systems Design and Implementation (OSDI 08). 2008. URL: https://www.usenix.org/conference/osdi-08/klee-unassisted-and-automatic-generation-high-coverage-tests-complex-systems (visited on 01/15/2026).

[16] C.S. Collberg and C. Thomborson. "Watermarking, Tamper-Proofing, and Obfuscation - Tools for Software Protection". In: *IEEE Transactions on Software Engineering* 28.8 (2002), pp. 735–746. DOI: 10.1109/TSE.2002.1027797.

[17] Christian Collberg, Clark Thomborson, and Douglas Low. "A Taxonomy of Obfuscating Transformations". In: *http://www.cs.auckland.ac.nz/staff-cgi-bin/mjd/csTRcgi.pl?serial* (Jan. 1997).

[18] Christian Collberg, Clark Thomborson, and Douglas Low. "Manufacturing Cheap, Resilient, and Stealthy Opaque Constructs". In: *Proceedings of the 25th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*. Popl '98. San Diego, California, USA: Association for Computing Machinery, 1998, pp. 184–196. ISBN: 0-89791-979-3. DOI: 10.1145/268946.268962. URL: https://doi.org/10.1145/268946.268962.

[19] Luigi Coniglio. "Combining Program Synthesis and Symbolic Execution to Deobfuscate Binary Code". In: 2019. URL: https://www.semanticscholar.org/paper/Combining-Program-Synthesis-and-Symbolic-Execution-Coniglio/9367bbeb37dc6d58773b (visited on 01/15/2026).

[20] Mull Project Contributors. *Libirm: Low-level IR Mutations for LLVM Bitcode*. 2019. URL: https://github.com/mull-project/libirm.

[21] Hoang-Vu Dang and Anh-Quynh Nguyen. *Unicorn: Next Generation CPU Emulator Framework*. Jan. 1, 2015.

[22] Sandeep Dasgupta et al. "Scalable Validation of Binary Lifters". In: *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation*. PLDI 2020. New York, NY, USA: Association for Computing Machinery, June 11, 2020, pp. 655–671. ISBN: 978-1-4503-7613-6. DOI: 10.1145/3385412.3385964. URL: https://dl.acm.org/doi/10.1145/3385412.3385964 (visited on 01/15/2026).

[23] Richard DeMillo, R.J. Lipton, and Fred Sayward. "Hints on Test Data Selection: Help for the Practicing Programmer". In: *Computer* 11 (May 1978), pp. 34–41. DOI: 10.1109/C-M.1978.218136.

[24] A. Denisov and S. Pankevich. "Mull It over: Mutation Testing Based on LLVM". In: *2018 IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW)*. Apr. 2018, pp. 25–31. DOI: 10.1109/ICSTW.2018.00024.

[25] Alessandro Di Federico, Mathias Payer, and Giovanni Agosta. "Rev.Ng: A Unified Binary Analysis Framework to Recover CFGs and Function Boundaries". In: *Proceedings of the 26th International Conference on Compiler Construction* (Austin, TX, USA). CC 2017. New York, NY, USA: Association for Computing Machinery, 2017, pp. 131–141. ISBN: 978-1-4503-5233-8. DOI: 10.1145/3033019.3033028. URL: https://doi.org/10.1145/3033019.3033028.

[26] Ninon Eyrolles, Louis Goubin, and Marion Videau. "Defeating MBA-based Obfuscation". In: *Proceedings of the 2016 ACM Workshop on Software PROtection*. SPRO '16. New York, NY, USA: Association for Computing Machinery, Oct. 28, 2016, pp. 27–38. ISBN: 978-1-4503-4576-7. DOI: 10.1145/2995306.2995308. URL: https://dl.acm.org/doi/10.1145/2995306.2995308 (visited on 01/15/2026).

[27] Xiangxin Fang et al. "LLM-VeriOpt: Verification-guided Reinforcement Learning for LLM-based Compiler Optimization". In: *2026 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*. 2026, pp. 740–755. DOI: 10.1109/CGO68049.2026.11395239.

[28] Zheyun Feng and Dongpeng Xu. "DEBRA: A Real-World Benchmark For Evaluating Deobfuscation Methods". In: *Proceedings of the 2025 Workshop on Software Understanding and Reverse Engineering*. SURE '25. New York, NY, USA: Association for Computing Machinery, Oct. 30, 2025, pp. 76–88. ISBN: 979-8-4007-1910-3. DOI: 10.1145/3733822.3764674. URL: https://dl.acm.org/doi/10.1145/3733822.3764674 (visited on 01/15/2026).

[29] Free Software Foundation. *GNU Coreutils: Core GNU Utilities*. 9.10. manual. Free Software Foundation, 2026. URL: https://www.gnu.org/software/coreutils/manual/coreutils.html.

[30] Free Software Foundation. *Gnulib – the GNU Portability Library*. manual. Free Software Foundation, 2026. URL: https://www.gnu.org/software/gnulib/.

[31] Zeyu Gao et al. *DecompileBench: A Comprehensive Benchmark for Evaluating Decompilers in Real-World Scenarios*. 2025. arXiv: 2505.11340 [cs.SE]. URL: https://arxiv.org/abs/2505.11340.

[32] Peter Garba. *Saturn_Results: Tests and Results for the SATURN Deobfuscation Framework*. 2019. URL: https://github.com/pgarba/Saturn_Results.

[33] Peter Garba and Matteo Favaro. *SATURN – Software Deobfuscation Framework Based on LLVM*. Sept. 5, 2019. DOI: 10.48550/arXiv.1909.01752. arXiv: 1909.01752 [cs]. URL: http://arxiv.org/abs/1909.01752 (visited on 01/15/2026). Prepublished.

[34] Alex Groce, Gerard Holzmann, and Rajeev Joshi. "Randomized Differential Testing as a Prelude to Formal Verification". In: *29th International Conference on Software Engineering (ICSE'07)*. 29th International Conference on Software Engineering (ICSE'07). May 2007, pp. 621–631. DOI: 10.1109/ICSE.2007.68. URL: https://ieeexplore.ieee.org/abstract/document/4222623 (visited on 01/15/2026).

[35] Hex-Rays SA. *IDA Pro Disassembler and Debugger*. 2025. URL: https://hex-rays.com/ida-pro.

[36] Intel Corporation. *Intel® 64 and IA-32 Architectures Software Developer's Manual*. manual. Intel Corporation, Feb. 2026. URL: https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html.

[37] Yue Jia and Mark Harman. "An Analysis and Survey of the Development of Mutation Testing". In: *IEEE Transactions on Software Engineering* 37.5 (Sept. 2011), pp. 649–678. ISSN: 1939-3520. DOI: 10.1109/TSE.2010.62.

[38] Pascal Junod et al. "Obfuscator-LLVM – Software Protection for the Masses". In: *2015 IEEE/ACM 1st International Workshop on Software Protection*. May 2015, pp. 3–9. DOI: 10.1109/SPRO.2015.10.

[39] Soomin Kim et al. "Testing Intermediate Representations for Binary Analysis". In: *2017 32nd IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 2017 32nd IEEE/ACM International Conference on Automated Software Engineering (ASE). Oct. 2017, pp. 353–364. DOI: 10.1109/ASE.2017.8115648. URL: https://ieeexplore.ieee.org/abstract/document/8115648 (visited on 01/15/2026).

[40] Patrick Kochberger et al. "SoK: Automatic Deobfuscation of Virtualization-Protected Applications". In: *Proceedings of the 16th International Conference on Availability, Reliability and Security*. Ares '21. Vienna, Austria: Association for Computing Machinery, 2021. ISBN: 978-1-4503-9051-4. DOI: 10.1145/3465481.3465772. URL: https://doi.org/10.1145/3465481.3465772.

[41] Kaiyuan Kuang et al. "Enhance Virtual-Machine-Based Code Obfuscation Security through Dynamic Bytecode Scheduling". In: *Computers & Security* 74 (2018), pp. 202–220. URL: https://api.semanticscholar.org/CorpusID:4774889.

[42] Tímea László and Ákos Kiss. "Obfuscating C++ Programs via Control Flow Flattening". In: vol. 30. Annales Universitatis Scientiarum Budapestinensis de Rolando Eötvös Nominatae. Sectio Computatorica. June 2007.

[43] C. Lattner and V. Adve. "LLVM: A Compilation Framework for Lifelong Program Analysis & Transformation". In: *International Symposium on Code Generation and Optimization, 2004. CGO 2004.* International Symposium on Code Generation and Optimization, 2004. CGO 2004. Mar. 2004, pp. 75–86. DOI: 10.1109/CGO.2004.1281665. URL: https://ieeexplore.ieee.org/abstract/document/1281665 (visited on 01/15/2026).

[44] Juneyoung Lee et al. "An SMT Encoding of Llvm's Memory Model for Bounded Translation Validation". In: *Computer Aided Verification*. Ed. by Alexandra Silva and K. Rustan M. Leino. Cham: Springer International Publishing, 2021, pp. 752–776. ISBN: 978-3-030-81688-9.

[45] Xavier Leroy. "Formal Verification of a Realistic Compiler". In: *Communications of The Acm* 52.7 (July 2009), pp. 107–115. ISSN: 0001-0782. DOI: 10.1145/1538788.1538814. URL: https://doi.org/10.1145/1538788.1538814.

[46] Huayi Li et al. "SymSem: Symbolic Execution with Time Stamps for Deobfuscation". In: *Information Security and Cryptology*. Ed. by Zhe Liu and Moti Yung. Cham: Springer International Publishing, 2020, pp. 225–245. ISBN: 978-3-030-42921-8. DOI: 10.1007/978-3-030-42921-8_13.

[47] Mingyue Liang et al. "Deobfuscation of Virtualization-Obfuscated Code Through Symbolic Execution and Compilation Optimization". In: *Information and Communications Security*. Ed. by Sihan Qing et al. Cham: Springer International Publishing, 2018, pp. 313–324. ISBN: 978-3-319-89500-0. DOI: 10.1007/978-3-319-89500-0_28.

[48] Cullen Linn and Saumya Debray. "Obfuscation of Executable Code to Improve Resistance to Static Disassembly". In: *Proceedings of the 10th ACM Conference on Computer and Communications Security*. Ccs '03. Washington D.C., USA: Association for Computing Machinery, 2003, pp. 290–299. ISBN: 1-58113-738-9. DOI: 10.1145/948109.948149. URL: https://doi.org/10.1145/948109.948149.

[49] Binbin Liu et al. "MBA-Blast: Unveiling and Simplifying Mixed Boolean-Arithmetic Obfuscation". In: 30th USENIX Security Symposium (USENIX Security 21). 2021, pp. 1701–1718. ISBN: 978-1-939133-24-3. URL: https://www.usenix.org/conference/usenixsecurity21/presentation/liu-binbin (visited on 01/14/2026).

[50] Zhibo Liu et al. "SoK: Demystifying Binary Lifters Through the Lens of Downstream Applications". In: *2022 IEEE Symposium on Security and Privacy (SP)*. 2022 IEEE Symposium on Security and Privacy (SP). May 2022, pp. 1100–1119. DOI: 10.1109/SP46214.2022.9833799. URL: https://ieeexplore.ieee.org/document/9833799 (visited on 01/15/2026).

[51] LLVM Project. *LLVM Language Reference Manual*. manual. LLVM Project, 2026. URL: https://llvm.org/docs/LangRef.html.

[52] Nuno P. Lopes et al. "Alive2: Bounded Translation Validation for LLVM". In: *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation* (Virtual, Canada). PLDI 2021. New York, NY, USA: Association for Computing Machinery, 2021, pp. 65–79. ISBN: 978-1-4503-8391-2. DOI: 10.1145/3453483.3454030. URL: https://doi.org/10.1145/3453483.3454030.

[53] William Mansky and Elsa Gunter. "A Framework for Formal Verification of Compiler Optimizations". In: vol. 6172. July 2010, pp. 371–386. ISBN: 978-3-642-14051-8. DOI: 10.1007/978-3-642-14052-5_26.

[54] Michael Matz et al. *System V Application Binary Interface: AMD64 Architecture Processor Supplement*. SUSE Labs / CodeSourcery, July 2012. URL: https://refspecs.linuxbase.org/elf/x86_64-abi-0.99.pdf.

[55] NaC-L. *Mergen: Deobfuscation via Optimization with Usage of LLVM IR and Parsing Assembly*. GitHub, 2023. URL: https://github.com/NaC-L/Mergen.

[56] Oreans Technologies. *Themida: Advanced Windows Software Protection System*. URL: https://www.oreans.com/Themida.php.

[57] Maksim Panchenko et al. *BOLT: A Practical Binary Optimizer for Data Centers and Beyond*. 2018. arXiv: 1807.06735 [cs.PL]. URL: https://arxiv.org/abs/1807.06735.

[58] A. Pnueli, M. Siegel, and E. Singerman. "Translation Validation". In: *Tools and Algorithms for the Construction and Analysis of Systems*. Ed. by Bernhard Steffen. Berlin, Heidelberg: Springer Berlin Heidelberg, 1998, pp. 151–166. ISBN: 978-3-540-69753-4.

[59] David A. Ramos and Dawson Engler. "Under-Constrained Symbolic Execution: Correctness Checking for Real Code". In: *Proceedings of the 24th USENIX Conference on Security Symposium*. SEC'15. USA: USENIX Association, Aug. 12, 2015, pp. 49–64. ISBN: 978-1-931971-23-2.

[60] Luc Reginato. *Updated Analysis of PatchGuard on Microsoft Windows 10 RS4*. Technical Report. Version 1.01. Tetrane, 2019. URL: https://blog.tetrane.com/downloads/Tetrane_PatchGuard_Analysis_RS4_v1.01.pdf.

[61] Benjamin Reichenwallner and Peter Meerwald-Stadler. "Efficient Deobfuscation of Linear Mixed Boolean-arithmetic Expressions". In: *Proceedings of the CheckMATE 2022 Workshop, Co-Located with the ACM Conference on Computer and Communication Security, CCS'22*. Los Angeles, CA, USA: ACM, November 7 - 11, 2022, 2022-11, pp. 19–28. DOI: 10.1145/3560831.3564256. URL: https://arxiv.org/abs/2209.06335.

[62] Benjamin Reichenwallner and Peter Meerwald-Stadler. "Simplification of General Mixed Boolean-Arithmetic Expressions: GAMBA". In: *Proceedings of the 2nd Workshop on Robust Malware Analysis, WORMA'23, Co-Located with the 8th IEEE European Symposium on Security and Privacy*. Delft, The Netherlands: IEEE, July 2023, pp. 427–438.

DOI: 10.1109/EuroSPW59978.2023.00053. URL: https://arxiv.org/abs/2305.06763.

[63] Rolf Rolles. "Unpacking Virtualization Obfuscators". In: *Proceedings of the 3rd USENIX Conference on Offensive Technologies*. WOOT'09. Montreal, Canada: USENIX Association, 2009, p. 1.

[64] Kevin A. Roundy and Barton P. Miller. "Binary-Code Obfuscations in Prevalent Packer Tools". In: *ACM Comput. Surv.* 46.1 (July 2013). ISSN: 0360-0300. DOI: 10.1145/2522968.2522972. URL: https://doi.org/10.1145/2522968.2522972.

[65] Jonathan Salwan, Sébastien Bardin, and Marie-Laure Potet. "Symbolic Deobfuscation: From Virtualized Code Back to the Original". In: *Detection of Intrusions and Malware, and Vulnerability Assessment*. Ed. by Cristiano Giuffrida, Sébastien Bardin, and Gregory Blanc. Cham: Springer International Publishing, 2018, pp. 372–392. ISBN: 978-3-319-93411-2. DOI: 10.1007/978-3-319-93411-2_17.

[66] Raimondas Sasnauskas et al. *Souper: A Synthesizing Superoptimizer*. 2018. arXiv: 1711.04422 [cs.PL]. URL: https://arxiv.org/abs/1711.04422.

[67] Sebastian Schrittwieser et al. "Protecting Software through Obfuscation: Can It Keep Pace with Progress in Code Analysis?" In: *ACM Comput. Surv.* 49.1 (Apr. 2016). ISSN: 0360-0300. DOI: 10.1145/2886012. URL: https://doi.org/10.1145/2886012.

[68] Koushik Sen. "Concolic Testing". In: *Proceedings of the 22nd IEEE/ACM International Conference on Automated Software Engineering*. ASE '07. New York, NY, USA: Association for Computing Machinery, Nov. 5, 2007, pp. 571–572. ISBN: 978-1-59593-882-4. DOI: 10.1145/1321631.1321746. URL: https://dl.acm.org/doi/10.1145/1321631.1321746 (visited on 01/15/2026).

[69] Yan Shoshitaishvili et al. "SOK: (State of) the Art of War: Offensive Techniques in Binary Analysis". In: *2016 IEEE Symposium on Security and Privacy (SP)*. May 2016, pp. 138–157. DOI: 10.1109/SP.2016.17.

[70] Colton Skees. *Dna: LLVM Based Static Binary Analysis Framework*. URL: https://github.com/Colton1skees/Dna.

[71] Nick Stephens et al. "Driller: Augmenting Fuzzing through Selective Symbolic Execution." In: *NDSS*. Vol. 16. 2016. 2016, pp. 1–16. URL: https://yancomm.net/papers/2016%20-%20NDSS%20-%20Driller.pdf (visited on 01/15/2026).

[72] Trail of Bits. *McSema: Framework for Lifting X86, Amd64, and AArch64 Program Binaries to LLVM Bitcode*. GitHub, 2014. URL: https://github.com/lifting-bits/mcsema.

[73] Trail of Bits. *Remill: Library for Lifting Machine Code to LLVM Bitcode*. GitHub, 2015. URL: https://github.com/lifting-bits/remill.

[74] S.K. Udupa, S.K. Debray, and M. Madou. "Deobfuscation: Reverse Engineering Obfuscated Code". In: *12th Working Conference on Reverse Engineering (WCRE'05)*. 12th Working Conference on Reverse Engineering (WCRE'05). Nov. 2005, 10 pp.–54. DOI:

10.1109/WCRE.2005.13. URL: https://ieeexplore.ieee.org/document/1566145 (visited on 01/15/2026).

[75] VMProtect Software. *VMProtect Software Protection*. URL: https://vmpsoft.com.

[76] Chenxi Wang et al. "Protection of Software-Based Survivability Mechanisms". In: *2001 International Conference on Dependable Systems and Networks*. July 2001, pp. 193–202. DOI: 10.1109/DSN.2001.941405.

[77] Hui Xu et al. "Layered Obfuscation: A Taxonomy of Software Obfuscation Techniques for Layered Security". In: *Cybersecurity* 3 (Apr. 2020), p. 9. DOI: 10.1186/s42400-020-00049-3.

[78] Babak Yadegari et al. "A Generic Approach to Automatic Deobfuscation of Executable Code". In: *2015 IEEE Symposium on Security and Privacy*. 2015 IEEE Symposium on Security and Privacy. May 2015, pp. 674–691. DOI: 10.1109/SP.2015.47. URL: https://ieeexplore.ieee.org/abstract/document/7163054 (visited on 01/15/2026).

[79] Yongxin Zhou et al. "Information Hiding in Software with Mixed Boolean-Arithmetic Transforms". In: *Information Security Applications*. Ed. by Sehun Kim, Moti Yung, and Hyung-Woo Lee. Berlin, Heidelberg: Springer Berlin Heidelberg, 2007, pp. 61–75. ISBN: 978-3-540-77535-5.

A1 numtowords::main Code

```
int main(int argc, char* argv[]) {
    if (argc < 2) return 1;
    int number, i = 0, j, digit;
    char * word[1000];

    number = argv[1][0];

    while(number) {
        digit = number % 10;
        number = number / 10;
        switch (digit) {
            case 0: word[i++] = "zero"; break;
            case 1: word[i++] = "one"; break;
            case 2: word[i++] = "two"; break;
            case 3: word[i++] = "three"; break;
            case 4: word[i++] = "four"; break;
            case 5: word[i++] = "five"; break;
            case 6: word[i++] = "six"; break;
            case 7: word[i++] = "seven"; break;
            case 8: word[i++] = "eight"; break;
            case 9: word[i++] = "nine"; break;
        }
    }
    for(j = i-1; j >= 0; j--){
         printf("%s ", word[j]);
    }
    printf("\n");
    return 0;
}
```

Figure A1.1: Full function code for `numtowords::main` [6]. Adapted slightly for readability.

A2 Full Optimised Slice of BB_11ed

```
define i64 @__slice_rip_concrete(ptr nocapture noundef readonly
   ↪ byval(%struct.State.10502) align 16 %0, i64 noundef %1,
   ↪ ptr noalias nocapture noundef readnone %2)
   ↪ local_unnamed_addr #19 {
bb_1174.i.i.i.i:
  %.sroa.85.0..sroa_idx = getelementptr inbounds i8, ptr %0,
     ↪ i64 2280
  %.sroa.85.0.copyload = load i64, ptr %.sroa.85.0..sroa_idx,
     ↪ align 8
  %.sroa.96.0..sroa_idx = getelementptr inbounds i8, ptr %0,
     ↪ i64 2296
  %.sroa.96.0.copyload = load i64, ptr %.sroa.96.0..sroa_idx,
     ↪ align 8
  %.sroa.118.0..sroa_idx = getelementptr inbounds i8, ptr %0,
     ↪ i64 2328
  %.sroa.118.0.copyload = load i64, ptr %.sroa.118.0..sroa_idx,
     ↪ align 8
  %state.i.i.sroa.540.2296.extract.trunc = trunc i64
     ↪ %.sroa.96.0.copyload to i32
  %sub.i.i.i.i.i.i.i.i = add i64 %1, -8
  %arrayidx.i213.i.i.i.i = getelementptr inbounds [0 x i8], ptr
     ↪ @RAM, i64 0, i64 %sub.i.i.i.i.i.i.i.i
  store i64 %.sroa.118.0.copyload, ptr %arrayidx.i213.i.i.i.i,
     ↪ align 1, !noalias !71
  %3 = add i64 %1, -8028
  %arrayidx.i210.i.i.i.i = getelementptr inbounds [0 x i8], ptr
     ↪ @RAM, i64 0, i64 %3
  store i32 %state.i.i.sroa.540.2296.extract.trunc, ptr
     ↪ %arrayidx.i210.i.i.i.i, align 1, !noalias !71
  %4 = add i64 %1, -8040
  %arrayidx.i211.i.i.i.i = getelementptr inbounds [0 x i8], ptr
     ↪ @RAM, i64 0, i64 %4
```

```
  store i64 %.sroa.85.0.copyload, ptr %arrayidx.i211.i.i.i.i,
     ↪ align 1, !noalias !71
  %sub.i.i29.i.i.i.i.i = add i32
     ↪ %state.i.i.sroa.540.2296.extract.trunc, -1
  %cmp.i.i.i.i40.i.i.i.i.i = icmp ne i32
     ↪ %state.i.i.sroa.540.2296.extract.trunc, 1
  %xor.i28.i.i.i48.i.i.i.i.i = lshr i32
     ↪ %state.i.i.sroa.540.2296.extract.trunc, 31
  %shr9.i.i.i.i49.i.i.i.i.i = xor i32 %sub.i.i29.i.i.i.i.i,
     ↪ %state.i.i.sroa.540.2296.extract.trunc
  %xor3.i.i.i.i50.i.i.i.i.i = lshr i32
     ↪ %shr9.i.i.i.i49.i.i.i.i.i, 31
  %add.i.i.i.i51.i.i.i.i.i = add nuw nsw i32
     ↪ %xor3.i.i.i.i50.i.i.i.i.i, %xor.i28.i.i.i48.i.i.i.i.i
  %cmp.i29.i.i.i52.i.i.i.i.i = icmp eq i32
     ↪ %add.i.i.i.i51.i.i.i.i.i, 2
  %tobool10.i.i.i.i.i.i.i = icmp sgt i32 %sub.i.i29.i.i.i.i.i, -1
  %cmp.i.i.i.i.i.i.i.i = xor i1 %tobool10.i.i.i.i.i.i.i,
     ↪ %cmp.i29.i.i.i52.i.i.i.i.i
  tail call void @llvm.assume(i1 %cmp.i.i.i.i40.i.i.i.i.i)
  tail call void @llvm.assume(i1 %cmp.i.i.i.i.i.i.i.i)
  %5 = add i64 %1, -16
  %arrayidx.i209.i.i.i.i = getelementptr inbounds [0 x i8], ptr
     ↪ @RAM, i64 0, i64 %5
  store i32 0, ptr %arrayidx.i209.i.i.i.i, align 1, !noalias !71
  %add.i.i.i.i.i.i.i.i = add i64 %.sroa.85.0.copyload, 8
  %arrayidx.i200.i.i.i.i = getelementptr inbounds [0 x i8], ptr
     ↪ @RAM, i64 0, i64 %add.i.i.i.i.i.i.i.i
  %v.0.copyload.i201.i.i.i.i = load i64, ptr
     ↪ %arrayidx.i200.i.i.i.i, align 1, !noalias !71
  %arrayidx.i.i.i.i.i.i = getelementptr inbounds [0 x i8], ptr
     ↪ @RAM, i64 0, i64 %v.0.copyload.i201.i.i.i.i
  %v.0.copyload.i.i.i.i.i.i = load i8, ptr %arrayidx.i.i.i.i.i.i,
     ↪ align 1, !noalias !71
  %conv.i12.i.i.i.i.i.i.i = sext i8 %v.0.copyload.i.i.i.i.i.i to i32
  br label %bb_1337.i.i.i.i

bb_1337.i.i.i.i:                                  ; preds =
   ↪ %bb_1337.i.i.i.i, %bb_1174.i.i.i.i
  %sub.i.i91.i.i.i.i.i278 = phi i32 [ %sub.i.i91.i.i.i.i.i,
     ↪ %bb_1337.i.i.i.i ], [ %conv.i12.i.i.i.i.i.i,
```

```
    ↪ %bb_1174.i.i.i.i ]
  %conv.i12.i181.i.i.i.i.i = sext i32 %sub.i.i91.i.i.i.i.i278
    ↪ to i64
  %mul.i.i155.i.i.i.i.i = mul nsw i64 %conv.i12.i181.i.i.i.i.i,
    ↪ 1717986919
  %shr.i160.i257.i.i.i.i.i = ashr i64 %mul.i.i155.i.i.i.i.i, 34
  %.neg = lshr i32 %sub.i.i91.i.i.i.i.i278, 31
  %conv.i.i90.i.i.i.i.i = trunc i64 %shr.i160.i257.i.i.i.i.i to
    ↪ i32
  %sub.i.i91.i.i.i.i.i = add nsw i32 %.neg,
    ↪ %conv.i.i90.i.i.i.i.i
  %add.i.i.i95.i.i.i.i.neg = mul i32 %sub.i.i91.i.i.i.i.i, -10
  %sub.i.i61.i.i.i.i.i = add i32 %add.i.i.i95.i.i.i.i.neg,
    ↪ %sub.i.i91.i.i.i.i.i278
  %6 = icmp ugt i32 %sub.i.i61.i.i.i.i.i, 9
  br i1 %6, label %bb_1337.i.i.i.i, label
    ↪ %__slice_rip.unfolded.exit

__slice_rip.unfolded.exit:                        ; preds =
   ↪ %bb_1337.i.i.i.i
  %7 = add i64 %1, -12
  %arrayidx.i208.i.i.i.i = getelementptr inbounds [0 x i8], ptr
    ↪ @RAM, i64 0, i64 %7
  %8 = add i64 %1, -24
  %arrayidx.i207.i.i.i.i = getelementptr inbounds [0 x i8], ptr
    ↪ @RAM, i64 0, i64 %8
  store i32 %sub.i.i91.i.i.i.i.i, ptr %arrayidx.i208.i.i.i.i,
    ↪ align 1, !noalias !71
  store i32 %sub.i.i61.i.i.i.i.i, ptr %arrayidx.i207.i.i.i.i,
    ↪ align 1, !noalias !71
  %conv.i.i4.i153.i.i.i.i = zext i32 %sub.i.i61.i.i.i.i.i to i64
  %9 = shl nuw nsw i64 %conv.i.i4.i153.i.i.i.i, 2
  %10 = add nuw nsw i64 %9, 8248
  %arrayidx.i186.i.i.i.i = getelementptr inbounds [0 x i8], ptr
    ↪ @RAM, i64 0, i64 %10
  %v.0.copyload.i187.i.i.i.i = load i32, ptr
    ↪ %arrayidx.i186.i.i.i.i, align 1, !noalias !71
  %conv.i.i1.i155.i.i.i.i = sext i32 %v.0.copyload.i187.i.i.i.i
    ↪ to i64
  %add.i.i.i156.i.i.i.i = add nsw i64 %conv.i.i1.i155.i.i.i.i,
    ↪ 8248
```

```
  ret i64 %add.i.i.i156.i.i.i.i
}
```

Figure A2.1: Full PC slice function of BB_11ed.

A3 AI Statement

Generative AI was used in the development of the project and in the writing of the thesis, in the form of large language models (LLMs): OpenAI's GPT 5.4 (`https://chatgpt.com/`) and Anthropic's Claude Sonnet / Opus 4.6 (`https://claude.ai/`). These were used for a variety of tasks, including for general queries, for coding assistance, and stylistic writing support. Details of the use of LLMs are outlined below.

LLM-based **chatbots** were used for general queries, including to aid in analysis of implementation and approach drawbacks, to help in understanding of existing research, and similar tasks.

Guided **coding agents** were used extensively for the implementation. The typical workflow consisted of the following steps: (1) an implementation specification was devised by the author; (2) this specification was provided to the agent, which provided a detail plan of implementation; (3) the author iterated on the implementation details and provided the agent with modifications; (4) the agent implemented the specification; (5) the author reviewed the implementation and made edits if necessary. All ideas and specifications were fully devised by the author.

LLMs were used for **writing support**, specifically to rephrase and reformulate certain passages to improve writing quality, and to aid in LaTeX formatting. LLMs were not used for bulk writing, and all research was done without LLM assistance.

LLMs were also used to aid in the **review process**, providing suggestions regarding modifications to be made to the thesis; these were then applied or rejected manually by the author.